Ước lượng cao độ F_0 kiểu Praat trên DE10-Standard

Từ file WAV 16 kHz qua HPS, FPGA và trở về HPS, so với Praat 7.0.03 trên laptop
(Intel Quartus Prime — hai đường: IP Catalog và HDL tự viết)

Bản v3 — viết lại theo đề cương chốt v2 (10/11/2026)

Mục lục

Chương 0. Bài toán, mục tiêu so sánh, và Praat thật sự làm gì **3**
0.1. Một file, hai câu trả lời . . . 3
0.2. Ý tưởng của Boersma: Chuẩn hóa bằng tự tương quan của cửa sổ . . . 5
0.3. Tám bước của Praat đúng như mã nguồn và các lựa chọn thiết kế phần cứng . . . 7
0.4. Kiến trúc hệ thống và bản đồ các chương . . . 12
0.5. Các phép đo đánh giá: GPE, VDE, FFE và sai số cent . . . 15
0.6. Ngân sách chu kỳ, hệ số thời gian thực RTF, và mạch phát cặp mẫu phần cứng . . . 17
0.7. Bản đồ kiểm chứng ba tầng và bộ golden v2 . . . 22
Bài tập cuối chương . . . 26

Chương 1. WAV trên HPS và cầu HPS–FPGA, chiều đi và chiều về **32**
1.1. Vấn đề truyền dữ liệu giữa hai miền HPS và FPGA . . . 32
1.2. Đọc tệp WAV đúng chuẩn và tính toán tham số toàn cục trên HPS . . . 33
1.3. Các phương án truyền dữ liệu, đánh đổi và lựa chọn . . . 39
1.4. Đường (A): Xây dựng bộ đệm bằng Intel On-Chip FIFO IP . . . 42
1.5. Đường (B): Hiện thực phần cứng bằng HDL tự viết . . . 44
1.6. Bản đồ thanh ghi và giao tiếp điều khiển lõi xử lý . . . 47
1.7. Phần mềm điều khiển trên HPS và kiểm chứng toàn diện . . . 49

Chương 0. Bài toán, mục tiêu so sánh, và Praat thật sự làm gì

▶ MỤC TIÊU HỌC TẬP

- Nắm vững bản chất bài toán ước lượng tần số cơ bản (F_0) và lý do vì sao hai phương pháp trực giác (đếm điểm qua không và tìm đỉnh phổ FFT) thất bại trên tín hiệu tiếng nói thực tế.
- Hiểu rõ nguyên lý cốt lõi của Boersma: chuẩn hóa hàm tự tương quan bằng cách chia cho hàm tự tương quan của cửa sổ Hanning để triệt tiêu độ suy giảm giả tạo ở các độ trễ lớn.
- Nắm tường tận 8 bước thuật toán trong mã nguồn Praat 7.0.03 cùng 10 tham số chuẩn, đối chiếu chính xác giữa các khâu phần mềm của Praat và các khối xử lý phần cứng trên FPGA.
- Hiểu rõ kiến trúc toàn hệ thống: luồng dữ liệu từ tệp WAV qua HPS, cầu nối Lightweight HPS-to-FPGA, lõi tính toán trên FPGA, và đường trả kết quả F0 về HPS để ghi tệp nhị phân.
- Nắm chắc định nghĩa 4 chỉ số sai số chuẩn trong so sánh cao độ: VDE, GPE, FFE và sai số cent mịn, cùng cách tính tay trên từng khung.
- Phân tích ngân sách chu kỳ từ nguyên lý đầu tiên: 50 MHz, 3.125 chu kỳ/mẫu, 500.000 chu kỳ/bước nhảy 10 ms; dẫn xuất chính xác 98.540 phép nhân-cộng MAC và đo thực tế 98.543 chu kỳ/khung, từ đó xác lập hệ số thời gian thực (RTF).
- Hiểu kiến trúc kiểm chứng ba tầng và vai trò của bộ mô hình tương đương (twin) số nguyên `golden_v2/fixed_twin.py` làm đặc tả phần cứng cho toàn bộ dự án.

0.1. Một file, hai câu trả lời

Một tệp âm thanh WAV chuẩn 16 kHz, kênh đơn (mono), định dạng mẫu PCM 16-bit có dấu được đưa vào hệ thống. Trên máy tính cá nhân, phần mềm Praat đọc tệp âm thanh này và xuất ra một đường cao độ F_0 với chu kỳ phân tích 10 ms. Trên bo mạch DE10-Standard, hệ thống vi xử lý HPS (Hard Processor System, Cortex-A9 lõi kép) đọc cùng tệp WAV đó từ thẻ nhớ SD, truyền các mẫu âm thanh qua cầu nối bus sang vùng logic FPGA (Cyclone V 5CSXFC6D6F31C6N). Lõi phần cứng trên FPGA tính toán cao độ từng khung và trả kết quả về HPS dưới dạng tệp nhị phân `pitch_f0.bin` ở định dạng số cố định Q12.4.

Câu hỏi trung tâm mà anh Tú cùng bất kỳ kỹ sư phần cứng nào cũng phải đối mặt là: hai đường cong cao độ đó lệch nhau bao nhiêu, và nguyên nhân vật lý hay số học nào tạo ra sự chênh lệch ở từng mili-giây? Để trả lời thấu đáo câu hỏi này, trước hết ta phải hiểu vì sao việc ước lượng cao độ tiếng nói lại khó, và vì sao những cách tiếp cận trực giác thông thường đều sụp đổ khi áp dụng vào thực tế.

Sự sụp đổ của phương pháp đếm điểm qua không (Zero-Crossing Counting)

Phương pháp đơn giản nhất mà người mới học xử lý tín hiệu thường nghĩ đến là đếm số lần tín hiệu đổi dấu qua mức 0 (zero-crossing) trong một khoảng thời gian, từ đó suy ra tần số. Tuy nhiên, một sự thật vật lý cơ bản là: tiếng nói có thanh (voiced speech) của con người hầu như không bao giờ là một sóng hình sin đơn âm. Khi dây thanh âm rung động, luồng khí đóng mở theo chu kỳ tạo ra dạng sóng có chứa rất nhiều hài âm (harmonics). Nếu hài âm bậc hai ($2f_0$) có biên độ mạnh và lệch pha so với sóng cơ bản, nó sẽ uốn dạng sóng tổng hợp cắt qua trục 0 nhiều lần trong cùng một chu kỳ lặp lại.

Để thấy rõ sự thất bại này bằng số liệu định lượng, ta xét một đoạn tín hiệu tổng hợp được tạo bởi kịch bản kiểm chứng `verify_v3/ch00_naive_methods.py`:

$$x(t) = \sin(2\pi \cdot 100t) - 0{,}75\sin(2\pi \cdot 200t) \tag{1}$$

Tần số cơ bản thực sự là $f_0 = 100{,}0$ Hz, tần số lấy mẫu $f_s = 16.000$ Hz. Một chu kỳ cơ bản kéo dài $T_0 = 1/100 = 10$ ms, tương ứng đúng $P = 160$ mẫu. Nếu tín hiệu là sóng sin thuần, một chu kỳ 160 mẫu sẽ cắt qua trục 0 đúng 2 lần. Nhưng khi kiểm tra trên đoạn tín hiệu dài 3 chu kỳ (480 mẫu), bộ đếm điểm qua không ghi nhận tổng cộng 12 lần đổi dấu, tức trung bình 4 điểm qua không trong mỗi chu kỳ. Tần số ước lượng bị thổi phồng thành:

$$f_{\text{est}} = \frac{12}{2 \times 3 \times 0{,}01\ \text{s}} = 200{,}0\ \text{Hz} \tag{2}$$

Thuật toán bị đánh lừa hoàn toàn bởi hài âm bậc hai, báo sai lệch cao độ gấp đôi (100% lỗi) ngay trên một tín hiệu hoàn toàn tuần hoàn và không hề có nhiễu bên ngoài.

Ví dụ tính tay thu nhỏ. Để nhìn thấy từng bước ở mức mẫu, xét dãy 16 mẫu rời rạc đại diện cho một chu kỳ rút gọn được sinh theo công thức $x_{16}[n] = \text{round}\left(100 \cdot \left(\sin\left(\frac{2\pi n}{16}\right) - 0{,}8\sin\left(\frac{4\pi n}{16}\right)\right)\right)$ (trích xuất trực tiếp từ kịch bản `verify_v3/ch00_naive_methods.py`):

n	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
$x[n]$	0	-18	-9	36	100	149	151	95	0	-95	-151	-149	-100	-36	9	18

Quan sát dãy mẫu: từ $n = 2$ (giá trị -9) sang $n = 3$ ($+36$), tín hiệu đổi dấu lần thứ nhất. Đến $n = 8$, tín hiệu bằng 0 và giảm xuống -95 ở $n = 9$, đổi dấu lần thứ hai. Từ $n = 13$ (-36) sang $n = 14$ ($+9$), tín hiệu lại đổi dấu lần thứ ba. Cùng với điểm 0 ban đầu ở $n = 0$, ta có 4 lần cắt qua 0 trong chỉ một chu kỳ cơ bản. Mạch đếm phần cứng nếu dựa vào hiện tượng này sẽ nhận diện nhầm chu kỳ thành 8 mẫu thay vì 16 mẫu.

Sự sụp đổ của phương pháp chọn đỉnh phổ FFT (FFT Peak Picking)

Cách tiếp cận thứ hai là chuyển tín hiệu sang miền tần số bằng phép biến đổi Fourier nhanh (FFT, Fast Fourier Transform) rồi tìm vị trí bin có biên độ cực đại. Để bám sát ngữ điệu biến đổi liên tục của tiếng nói, ta bắt buộc phải phân tích trên từng khung thời gian ngắn khoảng vài chục mili-giây (thường từ 20 ms đến 40 ms). Tại tần số lấy mẫu $f_s = 16.000$ Hz, một khung 40 ms tương ứng với 640 mẫu (hoặc 638 mẫu theo quy ước làm tròn đối xứng của Praat).

Độ phân giải tần số của phép phân tích FFT bị chặn cứng bởi chiều dài khung N:

$$\Delta f = \frac{f_s}{N} = \frac{16.000}{640} = 25{,}00 \text{ Hz} \quad (\text{với } N = 638, \Delta f = \frac{16.000}{638} \approx 25{,}08 \text{ Hz}) \tag{3}$$

Khoảng cách giữa hai vạch phổ kề nhau lên tới 25 Hz. Hãy xét một giọng nam trầm có tần số cơ bản thực là $f_0 = 112{,}5$ Hz. Tần số này rơi vào chính giữa bin 4 ($4 \times 25 = 100$ Hz) và bin 5 ($5 \times 25 = 125$ Hz). Bộ chọn đỉnh phổ dù lý tưởng nhất cũng chỉ có thể chỉ định bin 4 hoặc bin 5, gây ra sai số tuyệt đối ít nhất là $12{,}5$ Hz.

Trong âm nhạc học và ngữ âm học, tai người cảm nhận cao độ theo hàm logarit của tần số. Đơn vị đo độ chênh lệch cao độ chuẩn là **cent**, trong đó 1 quãng tám (tần số gấp đôi) tương ứng với 1.200 cent, và 1 nửa cung (semitone) tương ứng với 100 cent:

$$\Delta\text{cent} = 1.200 \cdot \left|\log_2\left(\frac{f_{\text{est}}}{f_0}\right)\right| \tag{4}$$

Áp dụng cho trường hợp chọn nhầm bin 100 Hz khi tần số thực là 112,5 Hz:

$$\Delta\text{cent} = 1.200 \cdot \left|\log_2\left(\frac{100{,}0}{112{,}5}\right)\right| \approx 203{,}91 \text{ cent} \tag{5}$$

(kết quả được kiểm chứng trong `verify_v3/ch00_naive_methods.log`). Sai số tương ứng 204 cent tương đương với hai nửa cung (một cung nguyên) trong âm nhạc. Một bộ ước lượng F_0 với sai số thô như vậy hoàn toàn không đáp ứng được yêu cầu trong các ứng dụng phân tích ngữ điệu hoặc đối sánh khoa học. Muốn giảm Δf xuống dưới 1 Hz, FFT cần cửa sổ dài hơn 1 giây, nhưng trong 1 giây đó người nói đã phát âm qua 3 đến 5 âm tiết khác nhau, tính dừng của tín hiệu bị phá vỡ hoàn toàn.

Bản chất của phép tự tương quan (Autocorrelation)

Để khắc phục triệt để hai nhược điểm trên, ta quay về nguyên lý đầu tiên: **một tín hiệu tuần hoàn luôn có hình dạng giống hệt chính nó khi bị dịch trễ đi đúng một chu kỳ cơ bản**. Phép đo mức độ tương đồng giữa tín hiệu $x[n]$ và bản sao dịch trễ của nó $x[n + \tau]$ ở độ trễ τ (lag) được định nghĩa qua hàm tự tương quan ngắn hạn:

$$R[\tau] = \sum_{n=0}^{N-1-\tau} x[n]x[n+\tau] \tag{6}$$

Ví dụ tính tay số nguyên thu nhỏ. Xét một tín hiệu rời rạc tuần hoàn lý tưởng với chu kỳ cơ bản $P = 3$ mẫu, độ dài khung $N = 9$ mẫu:

$$x = [3, -1, -2,\ 3, -1, -2,\ 3, -1, -2] \tag{7}$$

Ta tính trực tiếp giá trị tích vô hướng $R[\tau]$ theo từng bước cho các độ trễ từ $\tau = 0$ đến $\tau = 5$ (trích xuất từ kịch bản kiểm chứng `verify_v3/ch00_naive_methods.py`):

- Tại $\tau = 0$: lấy 9 cặp mẫu tự nhân với chính nó:

$$\begin{aligned} R[0] &= 3^2 + (-1)^2 + (-2)^2 + 3^2 + (-1)^2 + (-2)^2 + 3^2 + (-1)^2 + (-2)^2 \\ &= 9 + 1 + 4 + 9 + 1 + 4 + 9 + 1 + 4 = 42 \end{aligned}$$

- Tại $\tau = 1$: nhân 8 cặp mẫu kề nhau:

$$\begin{aligned} R[1] &= 3(-1) + (-1)(-2) + (-2)3 + 3(-1) + (-1)(-2) + (-2)3 + 3(-1) + (-1)(-2) \\ &= -3 + 2 - 6 - 3 + 2 - 6 - 3 + 2 = -15 \end{aligned}$$

- Tại $\tau = 2$: nhân 7 cặp mẫu cách nhau 2 vị trí:

$$\begin{aligned} R[2] &= 3(-2) + (-1)3 + (-2)(-1) + 3(-2) + (-1)3 + (-2)(-1) + 3(-2) \\ &= -6 - 3 + 2 - 6 - 3 + 2 - 6 = -20 \end{aligned}$$

- Tại $\tau = 3$: nhân 6 cặp mẫu cách nhau đúng một chu kỳ $P = 3$:

$$\begin{aligned} R[3] &= 3(3) + (-1)(-1) + (-2)(-2) + 3(3) + (-1)(-1) + (-2)(-2) \\ &= 9 + 1 + 4 + 9 + 1 + 4 = 28 \end{aligned}$$

- Tại $\tau = 4$: nhân 5 cặp mẫu, thu được $R[4] = -8$.
- Tại $\tau = 5$: nhân 4 cặp mẫu, thu được $R[5] = -13$.

Chỗ dễ vướng: Vì sao $R[3] = 28$ lại bé hơn $R[0] = 42$ dù tín hiệu tuần hoàn lý tưởng? Đây là bẫy tư duy kinh điển mà người thiết kế phần cứng hay gặp. Ở độ trễ $\tau = 3$, số cặp mẫu còn lại để nhân với nhau chỉ là $N - \tau = 9 - 3 = 6$ cặp, trong khi ở $\tau = 0$ ta cộng dồn tới 9 cặp. Nếu ta tính giá trị trung bình không chệch bằng cách chia tổng tích lũy cho số cặp mẫu tham gia:

$$\frac{R[0]}{9-0} = \frac{42}{9} \approx 4{,}67 \tag{8}$$

$$\frac{R[3]}{9-3} = \frac{28}{6} \approx 4{,}67 \tag{9}$$

Hai con số hoàn toàn trùng khớp! Đỉnh tương quan tại $\tau = 3$ đã chỉ ra chính xác chu kỳ cơ bản của tín hiệu là 3 mẫu mà không bị nhiễu hài âm đánh lừa như phương pháp đếm điểm qua 0, cũng không bị lượng tử hóa thô thiển như phương pháp chọn đỉnh FFT.

Tóm lại: Đếm điểm qua không thất bại vì hài âm bậc hai tạo thêm điểm cắt giả, lấy đỉnh FFT thất bại vì độ phân giải bin thô tới 204 cent ở 112,5 Hz (khi làm tròn về bin 100 Hz); phương pháp tự tương quan đo mức độ tương đồng theo miền thời gian, mở ra con đường xác định chu kỳ cơ bản chính xác và bền vững trước cấu trúc hài âm phức tạp của tiếng nói.

0.2. Ý tưởng của Boersma: Chuẩn hóa bằng tự tương quan của cửa sổ

Khi áp dụng phương pháp tự tương quan lên các đoạn tiếng nói thực tế, một bài toán kỹ thuật mới lập tức nảy sinh: ta không thể tính toán trên toàn bộ tệp âm thanh dài hàng phút mà bắt buộc phải cắt tín hiệu thành từng khung ngắn hữu hạn (khoảng 40 ms) để theo dõi sự biến đổi cao độ theo thời gian. Việc cắt khung bằng một hàm chữ nhật đột ngột (rectangular window) sẽ tạo ra các điểm gián đoạn sắc nhọn ở hai biên khung, dẫn đến hiện tượng rò rỉ năng lượng phổ (spectral leakage) nghiêm trọng. Để khắc phục hiện tượng này, giải pháp kinh điển là nhân khung tín hiệu với một hàm cửa sổ êm dịu, tiêu biểu là cửa sổ Hanning (hoặc Hamming), có biên độ vuốt dần về 0 ở hai đầu mút.

Tuy nhiên, việc nhân cửa sổ lại kéo theo một hậu quả tiêu cực khác: các mẫu âm thanh nằm xa tâm khung bị suy giảm biên độ mạnh mẽ. Khi ta tính hàm tự tương quan trên tín hiệu đã nhân cửa sổ $x_w[n] = x[n] \cdot w[n]$:

$$R_{xw}[\tau] = \sum_{n=0}^{N-1-\tau} x_w[n] x_w[n+\tau] = \sum_{n=0}^{N-1-\tau} x[n] w[n] x[n+\tau] w[n+\tau] \tag{10}$$

Độ trễ τ càng lớn thì khoảng chồng lấn giữa $w[n]$ và $w[n+\tau]$ càng thu hẹp, và tích $w[n]w[n+\tau]$ càng bị đè bẹp về gần 0. Đồ thị $R_{xw}[\tau]$ xuất hiện một sườn dốc suy thoái nhân tạo theo τ. Hệ quả là đỉnh tương quan ứng với chu kỳ cơ bản thực sự T_0 của một giọng nam trầm (nằm ở các độ trễ lớn, ví dụ $\tau \approx 160$ mẫu tại 100 Hz) bị lùn đi nghiêm trọng. Nếu dùng hàm tương quan chuẩn hóa thông thường $R_{xw}[\tau]/R_{xw}[0]$, giá trị đỉnh này thường xuyên rơi xuống dưới ngưỡng hữu thanh (voicing threshold mặc định của Praat là 0,45) khiến khung âm thanh bị nhận diện nhầm thành vô thanh, hoặc bị lép vế trước một đỉnh phụ ở độ trễ nhỏ hơn ($\tau = T_0/2$), gây ra lỗi kinh điển là **nhân đôi quãng tám (octave doubling)**.

Cơ chế chia cho tự tương quan cửa sổ $R_w[\tau]$

Đóng góp đột phá của Paul Boersma (1993) nằm ở nhận định giải tích: hàm tự tương quan của tín hiệu đã nhân cửa sổ xấp xỉ bằng tích giữa hàm tự tương quan thực sự của tín hiệu $R_x[\tau]$ với hàm tự tương quan của chính hàm cửa sổ $R_w[\tau]$:

$$R_{xw}[\tau] \approx R_x[\tau] \cdot R_w[\tau] \tag{11}$$

Trong đó hàm tự tương quan của cửa sổ được định nghĩa hoàn toàn tương tự:

$$R_w[\tau] = \sum_{n=0}^{N-1-\tau} w[n]w[n+\tau] \tag{12}$$

Từ mối liên hệ nhân này, Boersma đề xuất thuật toán chuẩn hóa: muốn tìm lại hàm tự tương quan chuẩn hóa không thiên vị $r[\tau]$ của bản thân tín hiệu gốc (với giá trị lý tưởng $r[0] = 1$ và $r[T_0] = 1$ đối với tín hiệu tuần hoàn lý tưởng), ta chỉ cần lấy $R_{xw}[\tau]$ chia cho $R_w[\tau]$, rồi chuẩn hóa tỉ lệ theo năng lượng tại $\tau = 0$:

$$r[\tau] = \frac{R_{xw}[\tau]/R_w[\tau]}{R_{xw}[0]/R_w[0]} = \frac{R_{xw}[\tau]}{R_w[\tau]} \cdot \frac{R_w[0]}{R_{xw}[0]} \tag{13}$$

Phép chia này đóng vai trò như một bộ bù độ dốc: vì $R_w[\tau]$ giảm dần về 0 khi τ tăng theo đúng quy luật hình học của cửa sổ, việc chia cho $R_w[\tau]$ sẽ nâng bổng các đỉnh ở độ trễ xa lên đúng biên độ thực sự của chúng, triệt tiêu hoàn toàn sự thiên lệch về phía các độ trễ nhỏ.

Vì sao phép xấp xỉ $R_{xw}[\tau] \approx R_x[\tau] \cdot R_w[\tau]$ lại đúng?

Để trả lời thuyết phục người kỹ sư phần cứng, ta xét bản chất vật lý của hai chuyển động: dao động nhanh của dây thanh âm và sự biến thiên chậm của đường bao cửa sổ.

Trong thuật toán Praat, chiều dài khung phân tích được ấn định bằng đúng 3 chu kỳ của tần số thấp nhất (pitch floor $f_{\text{min}} = 75$ Hz):

$$T_{\text{window}} = \frac{3}{f_{\text{min}}} = \frac{3}{75} = 0{,}04 \text{ s} = 40 \text{ ms} \quad (N = 638 \text{ mẫu tại } f_s = 16 \text{ kHz}) \tag{14}$$

Trong khi đó, tần số cơ bản của người nói dao động từ 75 Hz đến 600 Hz, tương ứng với chu kỳ cơ bản T_0 từ 26 mẫu (600 Hz) đến 213 mẫu (75 Hz). Điều này có nghĩa là bên trong một khung 638 mẫu, tín hiệu tiếng nói thực hiện từ 3 đến hơn 24 chu kỳ dao động tuần hoàn nhanh.

Đường bao cửa sổ Hanning $w[n]$ là một hàm cosin trơn dốc êm dịu trải dài trên toàn bộ 638 mẫu. Trong phạm vi một chu kỳ đơn lẻ T_0, giá trị của $w[n]$ và $w[n+\tau]$ gần như là một hằng số cục bộ. Khi ta lấy tổng tích lũy $\sum x[n]x[n+\tau] \cdot (w[n]w[n+\tau])$, đại lượng dao động nhanh $x[n]x[n+\tau]$ tạo ra giá trị kỳ vọng trung bình tuần hoàn $R_x[\tau]$, trong khi đại lượng biến thiên chậm $w[n]w[n+\tau]$ có thể đưa ra ngoài dấu tích phân cục bộ dưới dạng trọng số bao $R_w[\tau]$. Phép nhân chập giữa phổ tín hiệu hẹp và phổ cửa sổ rộng trong miền tần số tương ứng với phép nhân đường bao trực tiếp trong miền thời gian.

Ví dụ tính tay thu nhỏ: Sóng sin thuần qua cửa sổ Hanning 8 mẫu

Để minh chứng tường minh bằng số học cụ thể, ta xét một thí nghiệm mô phỏng thu nhỏ được thực thi bởi kịch bản `verify_v3/ch00_norm_sine.py`. Ta sử dụng một sóng sin tuần hoàn lý tưởng có chu kỳ $P = 4$ mẫu (tương ứng tần số $f_0 = f_s/4 = 4.000$ Hz), chiều dài phân tích $N = 8$ mẫu:

$$x[n] = \sin\left(\frac{2\pi n}{4}\right) = [0{,}000,\ 1{,}000,\ 0{,}000,\ -1{,}000,\ 0{,}000,\ 1{,}000,\ 0{,}000,\ -1{,}000] \tag{15}$$

Hàm cửa sổ Hanning 8 mẫu của Praat tuân thủ công thức $N+1$ (được định nghĩa trong mã nguồn Praat `Sound_to_Pitch.cpp` dòng 422–425):

$$w[i] = 0{,}5 - 0{,}5\cos\left(\frac{2\pi i}{N+1}\right), \quad i = 1, \ldots, N \quad (i = n+1) \tag{16}$$

Với $N = 8$, các mẫu cửa sổ tính ra là:

$$w = [0{,}116978,\ 0{,}413176,\ 0{,}750000,\ 0{,}969846,\ 0{,}969846,\ 0{,}750000,\ 0{,}413176,\ 0{,}116978]$$

Tín hiệu sau khi nhân cửa sổ $x_w[n] = x[n] \cdot w[n]$ là:

$$x_w = [0{,}0000,\ 0{,}4132,\ 0{,}0000,\ -0{,}9698,\ -0{,}0000,\ 0{,}7500,\ 0{,}0000,\ -0{,}1170]$$

Bảng 1 liệt kê kết quả tính tay chi tiết hàm tự tương quan của cửa sổ $R_w[\tau]$, hàm tự tương quan của tín hiệu nhân cửa sổ $R_{xw}[\tau]$, hệ số tương quan chưa chuẩn hóa $r_{\text{unnorm}}[\tau] = R_{xw}[\tau]/R_{xw}[0]$, và hệ số chuẩn hóa Boersma $r_{\text{Boersma}}[\tau]$ cho các độ trễ từ $\tau = 0$ đến $\tau = 4$ (trích xuất nguyên văn từ tệp nhật ký `verify_v3/ch00_norm_sine.log`).

Bảng 1: Đối chiếu giá trị tự tương quan trước và sau khi chuẩn hóa cửa sổ Boersma trên sóng sin thuần ($N = 8, P = 4$)

Độ trễ τ	$R_w[\tau]$	$R_{xw}[\tau]$	$r_{\text{unnorm}}[\tau]$	$r_{\text{Boersma}}[\tau]$	Đánh giá vật lý
0	3,375000	1,687500	1,000000	1,000000	Gốc chuẩn hóa năng lượng
1	3,111800	0,000000	0,000000	0,000000	Trực giao ở góc pha $90°$
2	2,431670	$-1{,}215835$	$-0{,}720495$	$-1{,}000000$	Ngược pha hoàn toàn ($180°$)
3	1,590835	0,000000	0,000000	0,000000	Trực giao ở góc pha $270°$
4	**0,846665**	**0,423332**	**0,250864**	**1,000000**	**Chu kỳ cơ bản** $P = 4$

Đọc bảng và kết luận định lượng. Hãy nhìn thẳng vào dòng $\tau = 4$, chính là chu kỳ lặp lại thực tế của sóng sin:

- Nếu **không chuẩn hóa cửa sổ**, hệ số tương quan chỉ đạt $r_{\text{unnorm}}[4] = 0{,}250864 \approx 0{,}251$. Tín hiệu tuần hoàn lý tưởng nhưng giá trị đỉnh tương quan bị sụt giảm tới gần 75% năng lượng! Với ngưỡng voicing mặc định của Praat là 0,45, thuật toán sẽ kết luận ngay khung này là *vô thanh* (unvoiced) do $0{,}251 < 0{,}45$.
- Ngược lại, khi áp dụng **chuẩn hóa Boersma**: ta lấy $R_{xw}[4]/R_w[4] = 0{,}423332/0{,}846665 = 0{,}500000$. Chia cho tỉ số tại gốc $R_{xw}[0]/R_w[0] = 1{,}687500/3{,}375000 = 0{,}500000$, ta thu được kết quả chính xác:

$$r_{\text{Boersma}}[4] = \frac{0{,}500000}{0{,}500000} = 1{,}000000 \tag{17}$$

Giá trị 1,000000 phản ánh đúng tính tuần hoàn của sóng sin, không hề bị ảnh hưởng bởi việc cửa sổ bị thót hẹp ở hai đầu.

Tóm lại: Cửa sổ cắt khung làm suy hao giả tạo năng lượng ở các độ trễ lớn khiến đỉnh cao độ thật bị lùn đi; ý tưởng cốt lõi của Boersma là chia hàm tự tương quan của tín hiệu cho hàm tự tương quan của cửa sổ $R_{xw}[\tau]/R_w[\tau]$, khôi phục trọn vẹn độ mạnh của chu kỳ cơ bản và triệt tiêu định kiến thiên lệch về các độ trễ nhỏ.

0.3. Tám bước của Praat đúng như mã nguồn và các lựa chọn thiết kế phần cứng

Mọi so sánh định lượng giữa bộ đo trên FPGA và phần mềm Praat chỉ có giá trị khoa học khi và chỉ khi ta hiểu chính xác từng dòng mã nguồn của Praat thực sự làm gì. Để tránh các sai lệch thường gặp về kích thước cửa sổ, số lượng tham số hay phạm vi tính toán, tài liệu này đối chiếu trực tiếp với mã nguồn chuẩn của bản phát hành **Praat 7.0.03** (ghim làm mốc tham chiếu xuyên suốt), cụ thể là các tệp `fon/Sound_to_Pitch.cpp`, `fon/Pitch.cpp`, `fon/Sampled.cpp` và `fon/praat_Sound.cpp`.

Tám bước thuật toán theo đúng thứ tự thực thi trong mã nguồn Praat

Thuật toán ước lượng cao độ bằng tự tương quan thô của Praat vận hành tuần tự qua tám bước sau:

Bước 1: Tính đỉnh biên độ toàn cục (`globalPeak`). Trong tệp `Sound_to_Pitch.cpp` (dòng 375–385), Praat quét qua toàn bộ tệp âm thanh đầu vào, tính giá trị trung bình DC của cả tệp, rồi xác định đỉnh biên độ cực đại:

$$\text{globalPeak} = \max_t |x(t) - \bar{x}_{\text{file}}| \tag{18}$$

Giá trị này phản ánh mức biên độ đỉnh tuyệt đối của người nói, được giữ nguyên làm mốc quy chiếu xuyên suốt để thiết lập ngưỡng yên lặng động cho từng khung ở Bước 8.

Bước 2: Phân bố lưới khung thời gian (`Sampled_shortTermAnalysis`). Trong tệp `Sampled.cpp` (hàm `Sampled_getWindowSamples` từ dòng 53, cùng hàm nội tuyến `Sampled_xToLowIndex` trong `Sampled.h` dòng 34; dòng 71–82 định nghĩa `Sampled_shortTermAnalysis`), Praat không đặt khung đầu tiên tại mẫu 0 mà căn các khung đối xứng quanh tâm của tệp âm thanh. Với độ dài tệp T, số khung phân tích được tính bởi công thức làm tròn sàn $\lfloor (T - 0{,}04)/0{,}01 \rfloor + 1$. Vị trí tâm của khung đầu tiên t_1 phụ thuộc vào độ dài tổng thể của tệp. Vì vậy, khung thứ nhất không bắt đầu ở mẫu 0 mà lệch đi một số mẫu nhất định; lõi phần cứng FPGA bắt buộc phải nhận tọa độ t_1 và chỉ số mẫu bắt đầu do bộ vi xử lý HPS tính toán để đảm bảo hai bên phân tích trên đúng từng mẫu âm thanh.

Bước 3: Trừ trung bình cục bộ trên 426 mẫu. Trong tệp `Sound_to_Pitch.cpp` (dòng 56–66), Praat tính giá trị trung bình cục bộ của tín hiệu trên đoạn dài đúng hai chu kỳ dài nhất về hai phía tâm khung ("one longest period to both sides"). Tại $f_{\text{min}} = 75$ Hz, chu kỳ dài nhất là $T_{0,\text{max}} = 16.000/75 = 213{,}33$ mẫu, làm tròn thành 213 mẫu. Do đó, trung bình cục bộ được tính trên đúng $2 \times 213 = 426$ mẫu quanh tâm khung ($106 \ldots 531$), hoàn toàn không phải trên toàn bộ 638 mẫu. Sau đó, tại dòng 68–84, Praat sao chép khung và trừ trung bình cục bộ này trước khi nhân với cửa sổ Hanning nhằm triệt tiêu triệt để thành phần một chiều (DC offset) cục bộ, ngăn ngừa sai lệch hàm tự tương quan.

Bước 4: Nhân cửa sổ Hanning $N + 1$ với $N = 638$ mẫu. Trong tệp `Sound_to_Pitch.cpp` (dòng 335–339 và dòng 422–425), kích thước cửa sổ phân tích được làm tròn đối xứng: từ 640 mẫu lý thuyết, Praat lấy nửa cửa sổ half $= 640/2 - 1 = 319$ mẫu, suy ra chiều dài cửa sổ thực tế là $N = 2 \times 319 = 638$ mẫu. Tín hiệu sau khi trừ trung bình cục bộ được nhân với hàm cửa sổ Hanning dạng $N + 1$:

$$w[i] = 0{,}5 - 0{,}5 \cos\left(\frac{2\pi i}{639}\right), \quad i = 1, \ldots, 638 \tag{19}$$

Bước 5: Tính đỉnh biên độ cục bộ (`localPeak`). Trong tệp `Sound_to_Pitch.cpp` (dòng 87–102), Praat tìm giá trị biên độ đỉnh tuyệt đối lớn nhất của tín hiệu đã nhân cửa sổ bên trong khoảng cách nửa chu kỳ dài nhất ± 107 mẫu quanh tâm khung ($j \in [212, 425]$):

$$\text{localPeak} = \max_{j \in [212,425]} |x_w[j]| \tag{20}$$

Tỉ số cường độ tương đối intensity được xác lập qua biểu thức (dòng 102):

$$\text{intensity} = \begin{cases} 1{,}0, & \text{nếu localPeak} > \text{globalPeak}, \\ \dfrac{\text{localPeak}}{\text{globalPeak}}, & \text{ngược lại}, \end{cases} \tag{21}$$

phục vụ trực tiếp cho việc xác định ngưỡng chuyển đổi có thanh/vô thanh.

Bước 6: Tự tương quan và chuẩn hóa Boersma bằng FFT 1024 điểm.

Trong tệp `Sound_to_Pitch.cpp` (dòng 147–157), Praat tính hàm tự tương quan khung $R_{xw}[\tau]$ bằng phép biến đổi FFT 1024 điểm qua hàm `NUMfft_forward` (dòng 150, với độ dài nhân chập $638 \times 1{,}5 = 957$ mẫu, chọn lũy thừa 2 gần nhất là 1024).

Đồng thời, tại dòng 430–445, Praat biến đổi FFT và tính tự tương quan của chính cửa sổ $R_w[\tau]$ (mảng `windowR`), chuẩn hóa mảng này và xác định độ trễ tối đa `brent_ixmax` $= 319$ mẫu. Sau đó, tại dòng 163–165, kết quả được chuẩn hóa và chia theo công thức Boersma:

$$r[i] = \frac{ac[i+1]}{ac[1] \cdot \texttt{windowR}[i+1]}$$

Bước 7: Tìm kiếm ứng viên cao độ và tinh chỉnh sinc70 + Brent. Trong tệp `Sound_to_Pitch.cpp` (dòng 185–200), Praat quét tìm các đỉnh cực đại cục bộ thỏa mãn đồng thời: $r[i] > 0{,}5 \times$ voicingThreshold, $r[i] > r[i-1]$ và $r[i] \geq r[i+1]$ với $i \in [2, \text{maximumLag} - 1]$. Với $N = 638$, maximumLag $= \lfloor 638/3 \rfloor + 2 = 214$, nên Praat quét qua $i = 2 \ldots 213$. Các độ trễ $i \in [2, 26]$ vẫn có thể trở thành ứng viên cực đại, nhưng do chúng tương ứng với tần số $f \geq 600$ Hz (vượt quá trần `pitch ceiling` 600 Hz), bộ dò đường Viterbi coi chúng là vô thanh (`Pitch.cpp` dòng 558, hàm `Pitch_util_frequencyIsVoiced`). Trong khi đó, mạch số FPGA của ta quét trực tiếp trên dải $[27, 213]$ (với độ trễ 26 và 214 đóng vai trò làm hàng xóm biên cho nội suy parabol 3 điểm; xem `golden_v2/README` độ lệch 6). Vị trí đỉnh ban đầu được ước lượng bằng nội suy parabol (dòng 195–200), nhưng sau đó **luôn luôn được tinh chỉnh chính xác bằng phép nội suy hàm sinc 70 điểm kết hợp thuật toán tìm kiếm Brent** (dòng 254–270, hàm `NUMimproveMaximum` ở dòng 258), bất kể chế độ `Very accurate` bật hay tắt. Độ mạnh (`strength`) của đỉnh nếu vượt quá 1 sẽ được phản xạ thành $1/\text{strength}$. Praat lưu lại tối đa 15 ứng viên có độ mạnh lớn nhất trong mỗi khung.

Bước 8: Thuật toán quy hoạch động Viterbi (Path Finder). Trong tệp `Pitch.cpp` (dòng 550–565), một bộ tìm đường tối ưu toàn cục (Viterbi) xâu chuỗi các ứng viên qua tất cả các khung trong tệp âm thanh. Ứng viên vô thanh có độ mạnh phụ thuộc vào cường độ biên độ:

$$\text{strength}_{\text{unvoiced}} = 0{,}45 + \max\left(0,\ 2 - \frac{\text{intensity}}{0{,}03/1{,}45}\right) \tag{22}$$

Ứng viên hữu thanh bị trừ đi một lượng chi phí quãng tám:

$$\text{octave_cost} = 0{,}01 \cdot \log_2\left(\frac{\text{Pitch ceiling}}{f}\right) \tag{23}$$

Đặc biệt, trong tệp `Pitch.cpp` (dòng 543–545), Praat nhân hệ số tỉ lệ thời gian timeStepCorrection $= 0{,}01/\Delta t$ vào hai chi phí chuyển tiếp: chi phí nhảy quãng tám (`octave-jump cost` $= 0{,}35 \cdot (0{,}01/\Delta t)$) và chi phí chuyển đổi hữu thanh/vô thanh (`voiced/unvoiced cost` $= 0{,}14 \cdot (0{,}01/\Delta t)$). Với bước nhảy mặc định $\Delta t = 10$ ms $= 0{,}01$ s, hệ số tỉ lệ này đúng bằng $1{,}0$; đối với các bước nhảy tổng quát khác, công thức này bảo đảm chi phí chuyển tiếp luôn tương thích theo thang thời gian vật lý.

Mười tham số chuẩn của Praat và hành vi ở $f_s = 16$ kHz

Lệnh chuẩn trong giao diện Praat là `Sound: To Pitch (raw autocorrelation)...` tiếp nhận đúng **10 tham số** cấu hình:

- **Time step** (mặc định 0,0 s, tự động): Praat tự tính $\Delta t = 0{,}75/f_{\text{min}} = 0{,}75/75 = 10$ ms, tương ứng bước nhảy khung (`hop`) đúng 160 mẫu tại 16 kHz.
- **Pitch floor** (mặc định 75 Hz): Đặt giới hạn tần số thấp nhất, quyết định cửa sổ phân tích dài 3 chu kỳ ($3/75 = 40$ ms $= 638$ mẫu) và độ trễ tối đa quét là $\tau_{\text{max}} = 214$ mẫu.
- **Pitch ceiling** (mặc định 600 Hz): Đặt giới hạn tần số cao nhất, quyết định độ trễ tối thiểu quét là $\tau_{\text{min}} = \lfloor 16000/600 \rfloor = 26$ mẫu; mọi ứng viên có tần số vượt 600 Hz bị coi là vô thanh.
- **Max. number of candidates** (mặc định 15): Số lượng ứng viên cao độ tối đa được giữ lại ở mỗi khung cho bộ giải thuật Viterbi; mã nguồn tự động ép giá trị này tối thiểu bằng $\lfloor 600/75 \rfloor = 8$.
- **Very accurate** (mặc định `off`): Khi tắt, thuật toán dùng cửa sổ Hanning 3 chu kỳ và nội suy sinc 70 điểm; nếu bật, Praat chuyển sang cửa sổ Gaussian 6 chu kỳ và nội suy sinc 700 điểm.
- **Silence threshold** (mặc định 0,03): Ngưỡng tỉ số biên độ đỉnh tuyệt đối cục bộ so với đỉnh toàn cục (tỉ số localPeak/globalPeak, không phải đo bằng năng lượng RMS); khi intensity giảm xuống dưới ngưỡng, Praat kích hoạt vùng dốc phi tuyến nhằm nâng cao độ mạnh của ứng viên vô thanh qua công thức $0{,}45 + \max(0,\ 2 - \text{intensity}/(0{,}03/1{,}45))$ (`Pitch.cpp` dòng 553–555), giúp ứng viên vô thanh thắng thế trong các khung thì thầm hoặc yên lặng.
- **Voicing threshold** (mặc định 0,45): Ngưỡng sức mạnh tương quan để ứng viên có thanh được chấp nhận cạnh tranh với ứng viên vô thanh.
- **Octave cost** (mặc định 0,01): Trọng số phạt theo logarit tần số, ưu tiên nhẹ cho các tần số cao hơn để chống lại xu hướng rơi vào bẫy chia đôi quãng tám.

- **Octave-jump cost** (mặc định 0,35): Hệ số phạt khi đường cao độ nhảy vọt đột ngột quá một quãng tám giữa hai khung liên tiếp trong thuật toán Viterbi (nhân tỉ lệ $0{,}01/\Delta t$).
- **Voiced/unvoiced cost** (mặc định 0,14): Hệ số phạt khi có sự chuyển trạng thái giữa có thanh và vô thanh giữa hai khung kề nhau trong thuật toán Viterbi (nhân tỉ lệ $0{,}01/\Delta t$).

Lưu ý về bước lọc mềm mép Nyquist (Soft Upsampling): Trong bài báo gốc của Paul Boersma năm 1993, tác giả có đề xuất một bước tiền xử lý gọi là "soft upsampling": làm mềm phổ biên độ dốc tuyến tính về 0 từ 95% đến 100% tần số Nyquist (tức từ 7,6 kHz đến 8,0 kHz ở $f_s = 16$ kHz) bằng biến đổi FFT nhằm giảm hiện tượng gợn sóng bậc cao trong hàm tương quan. Tuy nhiên, khi kiểm tra kỹ lưỡng toàn bộ mã nguồn của Praat 6.4.12 và Praat 7.0.03, **bước xử lý này hoàn toàn không tồn tại trong đường thực thi `raw autocorrelation`**. Tài liệu hướng dẫn sử dụng hiện hành của Praat cũng đã loại bỏ bước này. Do đó, trong thiết kế phần cứng ở Chương 3, ta bố trí một khối lọc FIR 31-tap tùy chọn nhưng đặt công tắc mặc định là **bypass** khi tiến hành đo đạc đối sánh với Praat để tránh tự tạo ra sai lệch không đáng có.

Bảng đối chiếu thiết kế: Praat phần mềm so với FPGA phần cứng

Bảng 2 tổng hợp toàn diện các quyết định kiến trúc: khâu nào ta bám sát Praat nguyên bản, khâu nào ta chủ động thay thế bằng giải thuật thân thiện với phần cứng, lý do kỹ thuật và tác động định lượng đến phép so sánh.

Bảng 2: Đối chiếu toàn diện giữa thuật toán Praat 7.0.03 và kiến trúc phần cứng FPGA trên DE10-Standard

Khâu xử lý	**Praat 7.0.03 (SW)**	**Thiết kế FPGA (HW)**	**Lý do kỹ thuật phần cứng**	**Ảnh hưởng tới phép so sánh**
1. Cửa sổ và lưới thời gian	$N = 638$ mẫu, hop 160, căn đối xứng cả tệp	Bám đúng $N = 638$, hop 160, nhận lưới thời gian từ HPS	Cùng lưới là điều kiện tiên quyết bắt buộc để so sánh từng khung	Sai lệch thời gian bằng 0; khung phân tích trùng khít hoàn toàn.
2. Trung bình cục bộ	Lấy trên 426 mẫu ($106 \dots 531$), số thực double	426 mẫu, nhân hằng số nghịch đảo $M = 10.082.083$, dịch 32 bit	Tránh bộ chia số nguyên phức tạp, vừa vặn trong bộ nhân cứng DSP 25×24	Trừ trung bình khớp bit chính xác (bit-exact) với mô hình số nguyên.
3. Tính tự tương quan	FFT 1024 điểm, số thực double 64-bit	Nhân-cộng MAC trực tiếp, toán hạng 17-bit, cần 43 bit, dùng 48 bit	1 khối DSP 18×18 chạy nối tiếp 98.543 nhịp, tiết kiệm hàng nghìn ALM so với lõi FFT	Khớp về mặt toán học; mô hình số nguyên twin vs R2 đạt sai số cent mịn trung bình $0{,}21$ cent ($p_{95} = 0{,}48$ cent) trên tín hiệu sạch, và $3{,}07$ cent ($p_{95} = 9{,}78$ cent) trên tín hiệu ồn 5 dB.
4. Chi phí quãng tám (`octave cost`)	Trừ $0{,}01 \log_2(\text{ceiling}/f)$ vào độ mạnh ứng viên	Chuẩn hóa $R[\tau]$ về 27 bit, trừ số hạng tra bảng $K[\tau]$	Thay thế phép chia và hàm logarit phức tạp bằng phép trừ số nguyên	Khớp hoàn toàn với công thức Praat, không làm méo dạng đỉnh parabol.
5. Tinh chỉnh đỉnh cực đại	Nội suy Parabol sơ bộ rồi tinh chỉnh sinc 70 điểm + Brent	Nội suy Parabol 3 điểm, gộp hai phép chia thành một phép chia duy nhất	Phép nội suy sinc70 lặp Brent quá cồng kềnh cho FPGA; parabol chỉ tốn 1 bộ chia	Lệch cực đại $1{,}08$ cent trên tín hiệu sạch; lệch cực đại $32{,}91$ cent trên tín hiệu ồn 5 dB.
6. Xếp hạng ứng viên	Quét $i \in [2, 213]$ (lags $2 \dots 26$ thành vô thanh), lưu 15 ứng viên	Quét trực tiếp dải $[27, 213]$, chỉ lưu 1 ứng viên có điểm số S_c cao nhất trên lưới	Tiết kiệm thanh ghi và logic quản lý bảng ứng viên phức tạp	Ở tín hiệu ồn có hai đỉnh gần hòa, FPGA có thể chọn đỉnh khác Praat (3/207 khung).
7. Lựa chọn cao độ toàn cục	Quy hoạch động Viterbi (Path Finder) duyệt toàn bộ tệp âm thanh	Bộ lọc trung vị 3 điểm (median-3) xử lý trôi (streaming)	Viterbi cần bộ nhớ lưu trữ toàn bộ tệp và giải thuật đệ quy quay lui; median-3 chỉ trễ 1 khung	Median-3 sửa được các đỉnh gai đơn lẻ nhưng không thay thế được Viterbi; phần chênh lệch đo bằng $R_1 - R_2$.

Đọc bảng và định vị mốc tham chiếu. Bảng đối chiếu trên cho thấy một nguyên tắc thiết kế mạch lạc: các khâu biến đổi tín hiệu vật lý cơ bản (lưới khung, độ dài cửa sổ 638 mẫu, trung bình 426 mẫu, chuẩn hóa cửa sổ và chi phí quãng tám dạng trừ) được hiện thực bám sát Praat để loại trừ mọi sai số tiên nghiệm.

Ba sự khác biệt có chủ ý nằm ở: (1) nội suy parabol thay cho sinc70 + Brent, (2) chọn đỉnh tốt nhất trên lưới thay vì 15 ứng viên, và (3) bộ lọc trung vị median-3 thay cho Viterbi. Chính vì FPGA không chứa bộ giải thuật Viterbi, trong toàn bộ tài liệu này ta định nghĩa hai mốc tham chiếu Praat chuẩn:

- **Praat** R_1: Chạy Praat với toàn bộ 10 tham số mặc định (gồm cả thuật toán Viterbi với `octave-jump cost` = 0,35 và `V/UV cost` = 0,14).
- **Praat** R_2: Chạy Praat với cấu hình tương tự nhưng đặt `octave-jump cost` = 0 và `V/UV cost` = 0. Khi triệt tiêu hai chi phí chuyển tiếp này, thuật toán Viterbi tự động suy biến thành phép chọn cực đại độc lập trên từng khung (argmax cục bộ).

Nhờ định nghĩa này, phép so sánh giữa FPGA (trước bộ lọc median) và Praat R_2 trở thành một phép so sánh **ngang hàng (apples-to-apples)**: cả hai đều chọn ứng viên tốt nhất trên từng khung độc lập. Hiệu số giữa Praat

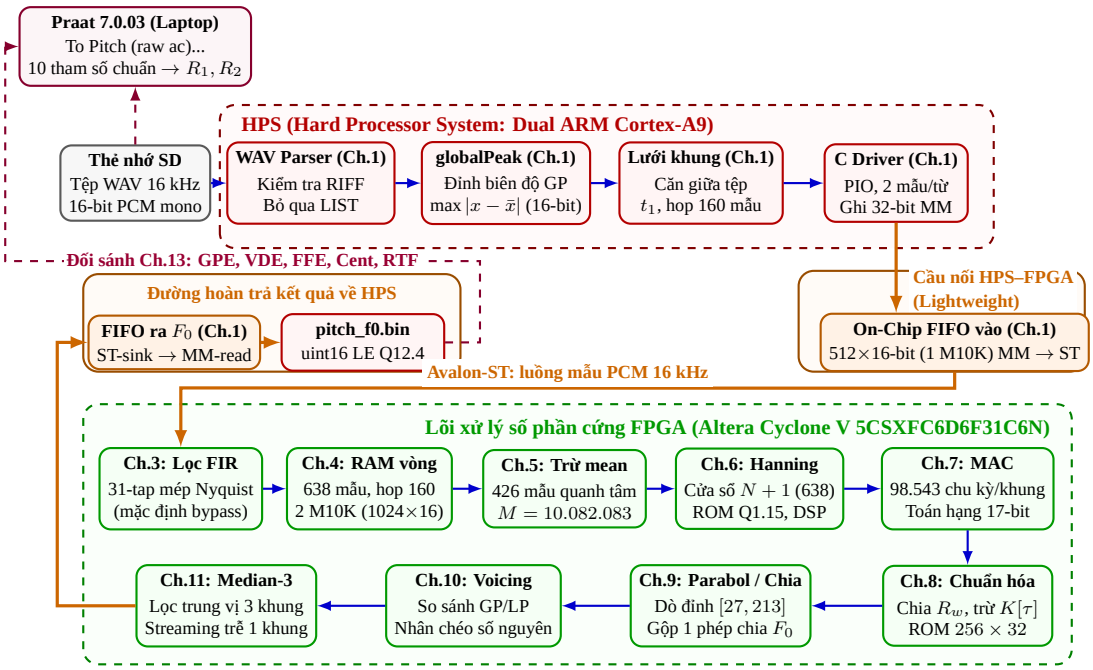
R_1 và Praat R_2 sẽ bóc tách và định lượng chính xác phần đóng góp riêng biệt của thuật toán Viterbi, giúp anh Tú trả lời trọn vẹn câu hỏi: phần chênh lệch nào do phần cứng số nguyên gây ra, và phần nào do sự vắng mặt của thuật toán quy hoạch động toàn cục.

Tóm lại: Praat vận hành qua 8 bước chặt chẽ với 10 tham số chuẩn; trên FPGA ta bám sát Praat ở kích thước cửa sổ 638 mẫu, trung bình 426 mẫu và chuẩn hóa cửa sổ, đồng thời thay thế sinc70+Brent bằng parabol và Viterbi bằng median-3, sử dụng mốc Praat R_2 làm chuẩn đối sánh ngang hàng trực tiếp.

0.4. Kiến trúc hệ thống và bản đồ các chương

Để hiện thực hóa thuật toán ước lượng cao độ kiểu Praat trên hệ thống phần cứng dị thể SoC (System-on-Chip) Altera Cyclone V (DE10-Standard), toàn bộ quy trình xử lý được phân bổ khoa học giữa hệ thống vi xử lý ARM Cortex-A9 (HPS) và vùng logic khả trình FPGA.

Hình 1 phác họa toàn cảnh kiến trúc luồng dữ liệu hai chiều: đi từ tệp âm thanh WAV trên thẻ nhớ SD, qua phần mềm HPS, đổ vào bộ đệm FIFO thông qua cầu nối bus Lightweight, chạy qua chuỗi xử lý số tín hiệu nối tiếp trong FPGA, rồi quay trở lại HPS để ghi tệp nhị phân kết quả, song song với nhánh đối chứng Praat 7.0.03 chạy trên máy tính cá nhân.



Hình 1: Kiến trúc tổng thể hệ thống ước lượng cao độ F_0: luồng dữ liệu từ tệp WAV qua HPS, cầu Lightweight, lõi xử lý phần cứng FPGA, đường hoàn trả kết quả và nhánh đối sánh với Praat trên máy tính.

Đọc hình. Hãy quan sát kỹ từng phân vùng chức năng trên sơ đồ kiến trúc:

1. **Phân vùng HPS (màu đỏ):** Bắt đầu từ tệp âm thanh WAV chuẩn nằm trên thẻ nhớ SD. Khối `WAV Parser` đọc và bóc tách cấu trúc nhị phân RIFF/WAVE, kiểm tra tần số lấy mẫu 16 kHz, định dạng 16-bit PCM mono và bỏ qua các khối mở rộng (như khối `LIST`). Khối tiếp theo tính toán đỉnh biên độ toàn cục globalPeak (GP) của cả tệp. Khối `Lưới khung` tính số khung, mốc thời gian t_1 và chỉ số mẫu bắt đầu của từng khung theo công thức đối xứng của Praat. Cuối cùng, khối điều khiển `C Driver` thực hiện ánh xạ bộ nhớ (`mmap`) thanh ghi cầu nối và phát các lệnh ghi mẫu PCM theo cơ chế PIO (Programmed I/O).

2. **Vùng cầu nối và hàng đợi (màu cam):** Cầu nối `Lightweight HPS-to-FPGA` (32-bit Avalon-MM) tiếp nhận các giao dịch ghi từ vi xử lý. Dữ liệu PCM được nạp vào khối bộ nhớ đệm `On-Chip FIFO` (dung lượng 512 từ 16-bit, chiếm đúng 1 khối RAM M10K) có chức năng đệm mẫu và chuyển đổi giao thức từ bus nhớ Avalon-MM sang giao tiếp luồng Avalon-ST (`valid/ready`).

3. **Phân vùng lõi xử lý phần cứng FPGA (màu xanh lá):** Các mẫu âm thanh lần lượt đi qua chuỗi khối chuyên biệt:

   - Khối lọc `FIR 31-tap` làm mềm mép phổ Nyquist (có công tắc bypass khi đo đối chiếu Praat);

- Bộ đệm `RAM vòng` 1024 mẫu (sử dụng 2 khối M10K) tích lũy đủ 638 mẫu cho mỗi khung và dịch chuyển mốc đọc `hop` = 160 mẫu;
- Khối `Trừ trung bình` cục bộ trên 426 mẫu quanh tâm khung, sử dụng phép nhân hằng số nghịch đảo $M = 10.082.083$ trên bộ nhân DSP cứng 25×24;
- Khối `Cửa sổ Hanning` nhân khung tín hiệu với mảng trọng số 638 mẫu lưu trong bộ nhớ ROM Q1.15;
- Khối tích lũy `MAC tự tương quan` sử dụng 1 khối DSP 18×18 chạy liên tục 98.543 chu kỳ để tính năng lượng $R[0]$ và 189 giá trị $R[\tau]$ ($\tau = 26 \dots 214$);
- Khối `Chuẩn hóa và Octave` tra bảng ROM 256×32 để nhân hệ số bù $C[\tau]$ và trừ chi phí quãng tám $K[\tau]$;
- Khối `Parabol và Bộ chia` dò tìm đỉnh cục bộ lớn nhất trên dải $[27, 213]$ và thực thi một phép chia số nguyên duy nhất để tính F_0 định dạng Q12.4;
- Khối `Quyết định Voicing` so sánh độ mạnh đỉnh với năng lượng và biên độ GP, LP qua bất đẳng thức nhân chéo số nguyên;
- Khối `Bộ lọc Median-3` nhận chuỗi kết quả theo cơ chế dòng (streaming), loại bỏ các gai cao độ đơn lẻ với độ trễ cố định 1 khung.

4. **Đường trả kết quả và nhánh đối chứng (màu tím):** Giá trị F_0 tính được cùng trạng thái có thanh/vô thanh được đẩy vào FIFO ngõ ra F0. Bộ điều khiển HPS đọc từ FIFO này qua cầu nối và ghi tuần tự vào tệp nhị phân `pitch_f0.bin` kèm tệp mô tả meta. Song song trên máy tính, phần mềm Praat 7.0.03 xử lý cùng tệp WAV với 10 tham số chuẩn, xuất ra hai đường tham chiếu R_1 và R_2 để phục vụ phân tích sai số toàn diện ở Chương 13.

Phân tích phương án truyền dữ liệu HPS–FPGA và kích thước bộ đệm FIFO

Để đưa dữ liệu từ vi xử lý sang FPGA một cách tối ưu, ta tiến hành phân tích so sánh giữa các phương án phần cứng:

1. Cơ chế truyền dữ liệu: PIO qua Lightweight Bridge so với mSGDMA qua Full Bridge.

- *Phương án A (Cơ chế PIO qua Lightweight HPS-to-FPGA):* Bộ điều khiển C trên HPS sử dụng ánh xạ bộ nhớ (`mmap`) trực tiếp vào không gian địa chỉ 2 MB của cầu nối Lightweight 32-bit Avalon-MM và ghi từng từ vào FIFO ngõ vào.
- *Phương án B (Cơ chế DMA qua HPS-to-FPGA toàn phần):* Sử dụng bộ điều khiển DMA phân tán `altera_avalon_msgdma` kết nối qua cầu nối HPS-to-FPGA AXI 64/128-bit.
- *Đánh đổi định lượng:* Âm thanh chuẩn 16 kHz, 16-bit PCM mono có lưu lượng dữ liệu vật lý rất thấp:

$$B = 16.000 \text{ mẫu/s} \times 2 \text{ byte/mẫu} = 32.000 \text{ byte/s} = 32 \text{ KB/s} \tag{24}$$

Khi ghép 2 mẫu 16-bit thành một từ 32-bit (chẵn khung), HPS chỉ cần phát 8.000 thao tác ghi/giây. Trên xung nhịp bus 50 MHz, một chu kỳ ghi bus tốn 20 ns. Ngay cả khi ước lượng với con số cực kỳ bi quan là $1\ \mu$s cho mỗi thao tác ghi trong không gian người dùng của Linux kernel, tổng thời gian CPU tiêu tốn cho việc truyền dữ liệu trong mỗi 1 giây âm thanh chỉ là $8.000 \times 1\ \mu\text{s} = 8$ ms, tương ứng hệ số thời gian thực truyền dữ liệu:

$$\text{RTF}_{\text{transfer}} = \frac{8 \text{ ms}}{1.000 \text{ ms}} = 0{,}008 \tag{25}$$

(thời gian ghi thực tế trên Linux cần được đo đạc chính xác trên bo mạch ở Chương 13 – phải đo). Hệ số $0{,}008$ chỉ chiếm chưa đầy 1% thời gian của hệ thống. Ngược lại, cấu hình mSGDMA đòi hỏi phân bổ bộ nhớ vật lý liên tục, khởi tạo bảng mô tả descriptor, quản lý ngắt hoàn tất và tiêu tốn hàng nghìn logic ALM trên Cyclone V. Với băng thông 32 KB/s, mSGDMA là sự phức tạp hóa không cần thiết.

2. Kích thước On-Chip FIFO: 512×16 bit so với 1024×16 bit.

- Một khối RAM nhúng M10K trên Cyclone V có dung lượng 10.240 bit. Ở độ rộng 16-bit, cấu hình sâu nhất của 1 khối M10K là 512 từ $\times$ 16 bit $= 8.192$ bit. FIFO 512×16 chiếm vừa vặn đúng **1 khối M10K duy nhất**.

- Dung lượng 512 mẫu chứa được $512/160 = 3{,}2$ bước nhảy hop (tương đương 32 ms âm thanh), hoàn toàn đủ để hấp thụ các dao động trễ lập lịch (scheduling jitter) của hệ điều hành Linux HPS (thường dưới $2 \ldots 5$ ms).
- Nếu nâng lên 1024×16, thiết kế bắt buộc phải tiêu tốn **2 khối M10K**. Do HPS phát dữ liệu đều đặn mỗi 10 ms và FPGA xử lý một khung chỉ mất khoảng 2 ms, bộ đệm 512 mẫu hầu như luôn ở mức rỗng; việc tăng lên 1024 mẫu không đem lại lợi ích thiết thực mà gây lãng phí tài nguyên RAM nhúng.

Lựa chọn thiết kế: Sử dụng cơ chế **PIO qua cầu nối Lightweight HPS-to-FPGA** kết hợp **On-Chip FIFO** 512×16 **(chiếm đúng 1 M10K)**. Đây là lựa chọn tối giản, vững chắc và tiết kiệm tài nguyên nhất (phù hợp với thiết kế chi tiết ở Chương 1, Mục 1.3).

Bản đồ phân công nội dung chi tiết theo từng chương

Mỗi khối chức năng trong kiến trúc tổng thể trên được nghiên cứu, thiết kế và kiểm chứng chi tiết trong từng chương chuyên biệt của tài liệu này:

- **Chương 1 (WAV trên HPS và Cầu giao tiếp):** Phân tích định dạng WAV nhị phân, giải thuật tính GP và lưới khung Praat bằng C; phân tích đánh đổi lưu lượng và thiết kế On-Chip FIFO (IP `altera_avalon_fifo` và HDL tự viết).
- **Chương 2 (Giao thức Avalon-ST và Mã hóa FSM):** Bản chất giao tiếp bắt tay `valid/ready`, quy tắc `readyLatency` 0 và 1, ngân sách chu kỳ chế độ tệp (file-driven), và so sánh tài nguyên các kiểu mã hóa máy trạng thái FSM (One-Hot, Binary, Gray).
- **Chương 3 (Lọc mép Nyquist FIR):** Hiện thực bộ lọc FIR 31-tap đối xứng bằng FIR Compiler II và HDL gấp tap qua pre-adder; phân tích đáp ứng tần số và cơ chế bypass khi so với Praat.
- **Chương 4 (Chia khung và RAM vòng):** Kỹ thuật đệm trượt 638 mẫu trên RAM vòng 1024×16 (2 khối M10K) với chế độ đọc trước ghi (read-during-write) và quản lý con trỏ cơ sở `rd_base`.
- **Chương 5 (Trừ trung bình cục bộ và DC-Blocker):** Thuật toán trừ trung bình 426 mẫu bằng phép nhân nghịch đảo khớp bit $M = 10.082.083$, $k = 32$; bộ lọc IIR DC-blocker tùy chọn $\alpha = 1 - 2^{-8}$.
- **Chương 6 (Cửa sổ Hanning $N + 1$):** Thiết kế bộ nhớ ROM chứa 638 hệ số Hanning định dạng Q1.15, tối ưu hóa một nửa ROM đối xứng và bộ nhân có chốt thanh ghi.
- **Chương 7 (Tự tương quan bằng MAC):** Trục tính toán nặng nhất của hệ thống: dẫn xuất 98.540 phép nhân-cộng 17-bit, cấu hình khối DSP cứng 18×18 kết hợp cờ `sload` để chuyển độ trễ với 0 nhịp nghỉ.
- **Chương 8 (Chuẩn hóa cửa sổ và Chi phí quãng tám):** Bộ nhớ ROM 256×32 chứa hệ số $C[\tau]$ và $K[\tau]$, kỹ thuật chuẩn hóa $R[\tau]$ về 27 bit để phép nhân vừa vặn trong 1 khối DSP.
- **Chương 9 (Dò đỉnh Parabol và Bộ chia gộp):** Điều kiện tìm đỉnh cục bộ trên dải $[27, 213]$, công thức nội suy parabol đúng dấu, và kiến trúc bộ chia số nguyên LPM_DIVIDE gộp cả nội suy lẫn đổi ra Hz thành một phép chia duy nhất.
- **Chương 10 (Quyết định Voicing):** Bất đẳng thức nhân chéo số nguyên kiểm tra năng lượng và biên độ GP, LP, thay thế hoàn toàn phép chia số thực của Praat.
- **Chương 11 (Bộ lọc trung vị Median-3):** Mạch so sánh lọc trung vị 3 khung nối tiếp, đánh giá khả năng sửa lỗi nhảy quãng tám và tác động tới sai số phân loại vô thanh.
- **Chương 12 (Tích hợp hệ thống và FSM đỉnh):** Kết nối liên hoàn toàn bộ các khối phần cứng, lập bảng phân bổ chu kỳ từng khung, bộ đếm chu kỳ hiệu năng và đóng gói tệp `pitch_f0.bin`.
- **Chương 13 (Kiểm chứng và Đo đạc đối sánh):** Thực hiện đo đạc 3 chiều: sai số chất lượng (GPE, VDE, FFE, cent) so với Praat R_1, R_2; đo thời gian ba phía (chu kỳ FPGA, HPS end-to-end, Praat trên máy tính); bảng so sánh tài nguyên IP và HDL từ báo cáo Fitter.
- **Chương 14 (Bảng kiểm tra lỗi Checklist):** Bảng rà soát 25 tiêu chí kỹ thuật: độ rộng bit, tràn số, độ trễ nhịp, tham số IP và bảo đảm tính tái lập của mã nguồn.
- **Phụ lục A (Bộ chuyển đổi tần số lấy mẫu Resampling):** Thiết kế khối lọc đa pha chuyển đổi tần số âm thanh từ 48 kHz và 44,1 kHz về chuẩn 16 kHz.

Tóm lại: Hệ thống phân chia rành mạch giữa HPS (quản lý tệp, căn lưới thời gian, giao tiếp bus) và FPGA (xử lý số tín hiệu nối tiếp theo dòng); từng khối phần cứng được ánh xạ tương ứng vào từng chương độc lập với kiểm chứng song song hai đường IP và HDL tự viết.

0.5. Các phép đo đánh giá: GPE, VDE, FFE và sai số cent

Để so sánh đường cao độ do mạch số FPGA ước lượng với đường cao độ tham chiếu chuẩn của phần mềm Praat (hoặc cao độ thực *ground-truth* của tệp thử nghiệm), ta không thể dùng các độ đo sai số trung bình bình phương (MSE) thông thường. Tiếng nói gồm các đoạn có thanh (voiced, dây thanh rung) xen kẽ các đoạn vô thanh (unvoiced, âm ma sát hoặc khoảng lặng). Khi một khung bị đoán sai trạng thái thanh tính, giá trị tần số hoàn toàn không xác định, khiến phép trừ số học trực tiếp mất ý nghĩa.

Dẫn xuất các tình huống xảy ra trên một khung phân tích

Xét một khung phân tích bất kỳ tại mốc thời gian t. Gọi F_{ref} là tần số cơ bản của Praat (hoặc ground-truth) và F_{test} là tần số do FPGA tính toán (quy ước quy đổi giá trị 0 đại diện cho trạng thái vô thanh). So sánh giữa hai giá trị này chỉ có thể rơi vào đúng **bốn tình huống vật lý** sau:

- **Tình huống 1: Cả hai bên cùng hữu thanh và gần nhau ($F_{\text{ref}} > 0, F_{\text{test}} > 0$, sai số tương đối $\leq 20\%$).** Đây là các khung ước lượng thành công về mặt cao độ cơ bản. Phần chênh lệch nhỏ giữa hai bên phản ánh sai số làm tròn số học, sự khác biệt giữa thuật toán nội suy đỉnh parabol và nội suy hàm sinc. Tình huống này được đưa vào tập đo **sai số cent mịn (fine-cent error)**.
- **Tình huống 2: Cả hai bên cùng hữu thanh nhưng lệch nhau rất xa ($F_{\text{ref}} > 0, F_{\text{test}} > 0$, sai số tương đối $> 20\%$).** Đây là các lỗi thô nghiêm trọng về mặt giải thuật, thường do mạch số chọn nhầm hài âm bậc hai (nhân đôi quãng tám: $F_{\text{test}} \approx 2F_{\text{ref}}$) hoặc chọn nhầm độ trễ gấp đôi (chia đôi quãng tám: $F_{\text{test}} \approx 0{,}5F_{\text{ref}}$). Hiện tượng này được định nghĩa là **Lỗi cao độ thô (Gross Pitch Error – GPE)**.
- **Tình huống 3: Quyết định thanh tính không đồng nhất (một bên có thanh, bên kia vô thanh).** Hoặc Praat phát hiện có thanh mà FPGA bỏ sót ($F_{\text{ref}} > 0, F_{\text{test}} = 0$), hoặc ngược lại FPGA nhận diện nhầm tiếng ồn thành có thanh ($F_{\text{ref}} = 0, F_{\text{test}} > 0$). Hiện tượng này được định nghĩa là **Lỗi quyết định thanh tính (Voicing Decision Error – VDE)**.
- **Tình huống 4: Cả hai bên cùng vô thanh ($F_{\text{ref}} = 0, F_{\text{test}} = 0$).** Hai bên hoàn toàn đồng thuận về khoảng lặng hoặc âm vô thanh; khung phân tích hoàn toàn chính xác và không có lỗi nào phát sinh.

Từ bốn tình huống trên, cộng đồng xử lý tiếng nói quốc tế chuẩn hóa bốn chỉ số định lượng:

1. Tỉ lệ lỗi quyết định thanh tính (VDE – Voicing Decision Error): Là tỉ số giữa số khung có quyết định thanh tính khác nhau (N_{VDE}, tức Tình huống 3) trên tổng số khung phân tích N:

$$\text{VDE} = \frac{N_{\text{VDE}}}{N} = \frac{\sum_{i=1}^{N} \left[(F_{\text{ref},i} > 0) \oplus (F_{\text{test},i} > 0) \right]}{N} \tag{26}$$

2. Tỉ lệ lỗi cao độ thô (GPE – Gross Pitch Error): Chỉ tính riêng trên tập hợp các khung mà *cả hai bên đều nhận định là có thanh* (N_{both} khung). GPE là tỉ lệ số khung bị lệch tần số quá 20% (N_{GPE}, tức Tình huống 2) trên N_{both}:

$$\text{GPE} = \frac{N_{\text{GPE}}}{N_{\text{both}}} = \frac{\sum_{i \in \text{both}} \mathbb{I}\left(\left| \frac{F_{\text{test},i}}{F_{\text{ref},i}} - 1 \right| > 0{,}20 \right)}{N_{\text{both}}} \tag{27}$$

3. Tỉ lệ lỗi tổng hợp toàn khung (FFE – F_0 Frame Error): Phản ánh bức tranh toàn cảnh về độ chính xác của hệ thống, bằng tổng số khung bị lỗi thanh tính cộng số khung bị lỗi cao độ thô, chia cho tổng số khung N:

$$\text{FFE} = \frac{N_{\text{VDE}} + N_{\text{GPE}}}{N} \tag{28}$$

4. Thống kê sai số cent mịn (Fine-Cent Statistics): Được tính riêng biệt trên các khung ở Tình huống 1 (cùng có thanh và không phạm lỗi thô GPE). Độ lệch cao độ tính bằng cent:

$$C_i = 1.200 \cdot \log_2 \left(\frac{F_{\text{test},i}}{F_{\text{ref},i}} \right) \tag{29}$$

Ta báo cáo bốn đại lượng thống kê: trung bình độ lệch tuyệt đối (Mean $|C|$), độ lệch chuẩn (Std C), phân vị 95 (P95 $|C|$) và sai số mịn cực đại (max $|C|$).

Ý nghĩa của ngưỡng 20% **và vì sao phải loại bỏ lỗi thô khi đo cent?** Một ngưỡng lệch tương đối 20% về mặt tần số tương ứng với bao nhiêu trong cảm thụ âm nhạc? Ta quy đổi trực tiếp ra cent:

$$1.200 \cdot \log_2(1{,}20) \approx 315{,}64 \text{ cent} \approx 3{,}16 \text{ nửa cung (semitones)} \tag{30}$$

Khoảng cách hơn 3 nửa cung vượt xa ngưỡng nghe nhận biết cao độ (JND – Just Noticeable Difference) của con người (khoảng $5 \ldots 10$ cent đối với âm thanh đơn âm hoặc người nghe được huấn luyện, và $10 \ldots 25$ cent đối với tiếng nói tự nhiên trong ngữ cảnh phức tạp). Khi một khung bị lỗi nhảy quãng tám (nhân đôi tần số: $F_{\text{test}} = 2F_{\text{ref}}$), sai số cent lên tới đúng 1.200 cent! Nếu gộp các khung lỗi quãng tám này vào thống kê cent mịn, chỉ cần một khung lỗi 1.200 cent duy nhất cũng đủ làm biến dạng giá trị trung bình từ $0{,}2$ cent tăng lên hàng chục cent, che lấp độ chính xác vi mô của bộ nội suy đỉnh parabol. Do đó, việc tách rời GPE (đánh giá lỗi giải thuật) và cent mịn (đánh giá độ tinh xảo số học) là nguyên tắc bắt buộc cần tuân thủ.

Ví dụ tính tay thu nhỏ trên 10 khung mô phỏng

Để người đọc tự tay kiểm tra từng con số, xét kịch bản mẫu gồm 10 khung phân tích đại diện đầy đủ cho 4 tình huống vật lý (được sinh và tính toán bởi kịch bản `verify_v3/ch00_metrics.py`):

Bảng 3: Bảng tính tay chi tiết các chỉ số chất lượng trên 10 khung mô phỏng sư phạm

Khung	F_{ref} (Hz)	F_{test} (Hz)	Phân loại tình huống	Sai số tương đối	Sai số cent	Trạng thái lỗi
0	0,0	0,0	Cả hai vô thanh (TH 4)	—	—	Không lỗi
1	120,0	120,0	Cùng có thanh (TH 1)	$0{,}00\%$	$+0{,}00$	Khớp hoàn toàn
2	150,0	152,0	Cùng có thanh (TH 1)	$1{,}33\%$	$+22{,}93$	Sai số mịn
3	200,0	196,0	Cùng có thanh (TH 1)	$2{,}00\%$	$-34{,}98$	Sai số mịn
4	220,0	224,0	Cùng có thanh (TH 1)	$1{,}82\%$	$+31{,}19$	Sai số mịn
5	0,0	130,0	Lệch thanh tính (TH 3)	—	—	**Lỗi VDE**
6	140,0	0,0	Lệch thanh tính (TH 3)	—	—	**Lỗi VDE**
7	100,0	200,0	Nhân đôi quãng tám (TH 2)	$100{,}00\%$	$+1.200{,}00$	**Lỗi GPE**
8	300,0	150,0	Chia đôi quãng tám (TH 2)	$50{,}00\%$	$-1.200{,}00$	**Lỗi GPE**
9	0,0	0,0	Cả hai vô thanh (TH 4)	—	—	Không lỗi

Dẫn xuất từng chỉ số từ Bảng 3:

- Tổng số khung: $N = 10$.
- Số khung cùng có thanh: $N_{\text{both}} = 6$ (Khung 1, 2, 3, 4, 7, 8).
- Lỗi thanh tính (VDE): xuất hiện ở Khung 5 và Khung 6 ($N_{\text{VDE}} = 2$). Tỉ lệ:

$$\text{VDE} = \frac{2}{10} = 20{,}0\% \tag{31}$$

- Lỗi cao độ thô (GPE): trong 6 khung cùng có thanh, Khung 7 bị nhân đôi cao độ ($100\% > 20\%$) và Khung 8 bị chia đôi ($50\% > 20\%$), nên $N_{\text{GPE}} = 2$. Tỉ lệ:

$$\text{GPE} = \frac{2}{6} \approx 33{,}3\% \tag{32}$$

- Lỗi tổng hợp toàn khung (FFE):

$$\text{FFE} = \frac{N_{\text{VDE}} + N_{\text{GPE}}}{N} = \frac{2+2}{10} = 40{,}0\% \tag{33}$$

- Thống kê sai số cent mịn trên 4 khung không lỗi thô (Khung 1, 2, 3, 4 với độ lệch tuyệt đối là $0{,}00$, $22{,}93$, $34{,}98$, $31{,}19$ cent):

$$\text{Trung bình tuyệt đối} = \frac{0{,}00 + 22{,}93 + 34{,}98 + 31{,}19}{4} = 22{,}28 \text{ cent} \tag{34}$$

$$\text{Độ lệch chuẩn (Std)} = 25{,}64 \text{ cent} \tag{35}$$

$$\text{Phân vị 95 (P95)} = 34{,}41 \text{ cent} \tag{36}$$

$$\text{Sai số mịn cực đại} = 34{,}98 \text{ cent} \tag{37}$$

Để khẳng định tính đúng đắn của các hàm đo đạc, kịch bản `verify_v3/ch00_metrics.py` đã nạp trực tiếp toàn bộ 16 tệp dữ liệu `.tsv` kết quả trong `golden_v2/out/` và chạy hàm `compute_metrics()`, xác nhận số lượng khung VDE, GPE, FFE và giá trị sai số cent mịn cực đại trùng khớp với từng con số trong `golden_v2/out/compare_summary.json`. Đồng thời, sự khác biệt có chủ ý về trung bình và phân vị 95 (do `compare.py` tính trên toàn bộ các khung cùng có thanh bao gồm cả lỗi thô, còn `ch00_metrics.py` chỉ tính trên các khung cent mịn) đã được ghi nhận và giải thích minh bạch trong mã nguồn.

Tóm lại: VDE đo sự sai lệch phân loại hữu thanh/vô thanh trên toàn tệp, GPE đo lỗi nhảy quãng tám trên các khung cùng hữu thanh, FFE gộp cả hai loại lỗi trên tổng số khung, còn sai số cent mịn đánh giá độ chính xác toán học của bộ nội suy đỉnh sau khi đã bóc tách triệt để các lỗi thô.

0.6. Ngân sách chu kỳ, hệ số thời gian thực RTF, và mạch phát cặp mẫu phần cứng

Khi chuyển dịch một thuật toán từ môi trường máy tính sang chip bán dẫn FPGA, bài toán thiết kế trọng tâm đầu tiên của người kỹ sư là: mạch số có đủ nhanh để xử lý tín hiệu theo thời gian thực (real-time) hay không, và ta cần bao nhiêu tài nguyên phần cứng (bộ nhân DSP, khối nhớ RAM M10K) để đáp ứng thời hạn đó?

Dẫn xuất ngân sách chu kỳ từ nguyên lý đầu tiên

Hệ thống phần cứng trên bo mạch DE10-Standard vận hành với xung nhịp đồng hồ hệ thống $f_{\text{clk}} = 50$ MHz (chu kỳ xung nhịp $T_{\text{clk}} = 20$ ns). Tín hiệu âm thanh chuẩn được lấy mẫu ở tần số $f_s = 16.000$ Hz (chu kỳ lấy mẫu $T_s = 62{,}5\ \mu$s).

Số chu kỳ xung nhịp đồng hồ FPGA có sẵn giữa hai mẫu âm thanh liên tiếp là:

$$N_{\text{cycles/sample}} = \frac{f_{\text{clk}}}{f_s} = \frac{50.000.000}{16.000} = 3.125 \text{ chu kỳ/mẫu} \tag{38}$$

Cứ sau một khoảng thời gian bước nhảy khung $\Delta t = 10$ ms (`hop` = 160 mẫu), hệ thống phải hoàn tất việc tính toán một giá trị F_0 mới. Ngân sách chu kỳ tối đa cho mỗi khung để đạt ngưỡng thời gian thực là:

$$N_{\text{budget/hop}} = 160 \text{ mẫu} \times 3.125 \text{ chu kỳ/mẫu} = 500.000 \text{ chu kỳ/khung} \tag{39}$$

Con số 500.000 chu kỳ ở 50 MHz chính là trần thời gian thực. Bất kỳ thiết kế nào hoàn thành một khung dưới 500.000 chu kỳ đều chạy nhanh hơn tốc độ phát âm thanh thực tế.

Dẫn xuất số phép nhân-cộng MAC và số chu kỳ của khối tự tương quan

Khối tiêu tốn chu kỳ tính toán nặng nề nhất trong toàn bộ hệ thống là khối tự tương quan. Ta xét một khung có độ dài $N = 638$ mẫu, rà soát trên dải độ trễ từ $\tau_{\min} = 26$ đến $\tau_{\max} = 214$ (tương ứng với $214 - 26 + 1 = 189$ độ trễ).

Tại một độ trễ τ bất kỳ, số cặp mẫu cần nhân và tích lũy là $N - \tau = 638 - \tau$. Tổng số phép nhân-cộng (MAC) cho toàn bộ 189 độ trễ được tính chính xác bằng công thức tổng cấp số cộng:

$$\sum_{\tau=26}^{214} (638 - \tau) = \sum_{\tau=26}^{214} 638 - \sum_{\tau=26}^{214} \tau \tag{40}$$

$$= 189 \times 638 - \frac{189 \times (26 + 214)}{2} \tag{41}$$

$$= 120.582 - \frac{189 \times 240}{2} = 120.582 - 189 \times 120 \tag{42}$$

$$= 120.582 - 22.680 = 97.902 \text{ phép MAC} \tag{43}$$

Để tính năng lượng khung chuẩn hóa tại gốc $\tau = 0$ ($R[0]$), ta cần thêm $N = 638$ phép nhân-cộng nữa:

$$N_{\text{pairs}} = 97.902 + 638 = 98.540 \text{ cặp mẫu} \tag{44}$$

Khối nhân-cộng phần cứng trong Chương 7 được thiết kế với đường ống 2 tầng thanh ghi (thanh ghi ngõ vào toán hạng trong khối DSP cứng và thanh ghi tổng tích lũy ngõ ra), cộng với 1 nhịp đọc từ bộ nhớ RAM M10K hai cổng, tạo ra độ trễ đường ống (pipeline latency) tổng cộng là 3 chu kỳ xung nhịp.

Nhờ cơ chế phát cặp mẫu liên tục không nhịp nghỉ (sẽ phân tích ở phần HDL bên dưới), tổng số chu kỳ xung nhịp từ lúc phát cặp địa chỉ đầu tiên đến khi giá trị $R[214]$ xuất hiện hợp lệ ở đầu ra là:

$$N_{\text{cycles, MAC}} = 98.540 \text{ nhịp phát} + 3 \text{ nhịp trễ} = 98.543 \text{ chu kỳ/khung} \tag{45}$$

Con số 98.543 chu kỳ này đã được đo đạc chính xác bằng bộ mô phỏng iverilog trên mã nguồn `golden_v2/hdl_check/tb_mac638.v`.

Phân tích phương án kiến trúc khối MAC: Tuần tự 1 DSP so với Song song P DSP

Trước khi ấn định thiết kế phần cứng cho khối tự tương quan, ta phân tích đánh đổi giữa hai phương án kiến trúc:

- *Phương án 1 (Kiến trúc tuần tự 1 DSP, $P = 1$):* Dùng duy nhất 1 bộ nhân-cộng MAC 18×18 bit vận hành nối tiếp qua tất cả các cặp mẫu của 190 độ trễ ($R[0]$ và 189 độ trễ τ).
- *Phương án 2 (Kiến trúc song song P DSP, $P \geq 2$):* Bố trí P bộ nhân-cộng MAC hoạt động song song để chia sẻ việc tính toán các độ trễ khác nhau.
- *Đánh đổi định lượng:*
  - **Số chu kỳ:** Số chu kỳ tính toán của khối MAC tỉ lệ nghịch với P:

$$N_{\text{cycles, MAC}} \approx \frac{98.540}{P} + 3 \text{ chu kỳ} \tag{46}$$

    Với $P = 2$, số chu kỳ giảm còn khoảng 49.273 chu kỳ; với $P = 4$, giảm còn khoảng 24.638 chu kỳ.
  - **Tài nguyên DSP:** Số khối DSP tiêu tốn tăng tuyến tính bằng đúng P khối nhân.
  - **Nghẽn cổ chai cổng đọc RAM M10K:** Đây là rào cản vật lý then chốt. Mỗi bộ nhân MAC ở mỗi chu kỳ cần đọc đồng thời 2 mẫu âm thanh $x[n]$ và $x[n+\tau]$. Một khối RAM nhúng M10K chỉ hỗ trợ tối đa 2 cổng đọc độc lập (True Dual-Port RAM). Nếu sử dụng P bộ MAC song song, hệ thống đòi hỏi $2P$ cổng đọc đồng thời tại mỗi nhịp xung nhịp. Để giải quyết, ta buộc phải nhân bản bộ nhớ RAM vòng thành P bản sao giống hệt nhau (tiêu tốn $2P$ khối M10K thay vì 2 khối M10K), hoặc phải thiết kế cơ chế phân vùng ngân hàng nhớ (multi-banking) kèm logic phân xử chuyển mạch vô cùng phức tạp để chống xung đột truy xuất.
  - **Khả năng đáp ứng thời gian thực:** Ở tần số xung nhịp chuẩn 50 MHz, ngân sách cho mỗi bước nhảy khung 10 ms là 500.000 chu kỳ. Kiến trúc tuần tự với $P = 1$ chỉ tiêu tốn 98.543 chu kỳ, tương ứng hệ số $\text{RTF}_{\text{MAC}} \approx 0{,}197$ (nhanh gấp 5 lần thời gian thực).
- *Lựa chọn thiết kế:* Sử dụng **kiến trúc tuần tự 1 khối DSP 18×18 ($P = 1$)**. Vì $P = 1$ đã thừa sức đáp ứng thời gian thực với biên an toàn gấp 5 lần, việc tăng P chỉ gây lãng phí bộ nhân DSP và khối nhớ M10K quý giá trên FPGA mà không đem lại bất kỳ giá trị thực tiễn nào.

Khái niệm Hệ số thời gian thực (RTF) và Độ trễ so với Thông lượng

Hệ số thời gian thực (**RTF – Real-Time Factor**) được định nghĩa là tỉ số giữa thời gian xử lý của phần cứng chia cho thời lượng thực tế của đoạn âm thanh:

$$\text{RTF} = \frac{T_{\text{xử lý}}}{T_{\text{âm thanh}}} = \frac{\text{Số chu kỳ FPGA}}{N_{\text{budget/hop}}} \tag{47}$$

Riêng đối với khối tự tương quan MAC, hệ số RTF đạt được là:

$$\text{RTF}_{\text{MAC}} = \frac{98.543}{500.000} \approx 0{,}197086 \approx 0{,}197 \tag{48}$$

Tổng số chu kỳ của toàn bộ lõi FPGA (bao gồm cả trừ trung bình 426 mẫu, nhân cửa sổ Hanning 638 mẫu, chuẩn hóa cửa sổ, nội suy parabol và lọc median-3) ước lượng chỉ mất khoảng 99.700 chu kỳ/khung (sẽ phân tích chi tiết ở Chương 12):

$$\text{RTF}_{\text{core}} = \frac{99.700}{500.000} \approx 0{,}1994 \approx 0{,}199 \tag{49}$$

Hệ số RTF $\approx 0{,}20$ có nghĩa là: mạch số FPGA chỉ mất 2 mili-giây để tính toán xong một khung âm thanh dài 10 mili-giây! Phần cứng chạy nhanh gấp 5 lần tốc độ phát âm thanh thời gian thực.

Chỗ dễ vướng: Độ trễ đường ống (Latency) có làm chậm 98.540 phép tính không? Câu trả lời là hoàn toàn không. Người học cần phân biệt rành mạch giữa *độ trễ (latency)* và *thông lượng (throughput)*. Hãy liên tưởng đến một băng chuyền nhà máy: món hàng đầu tiên mất 3 nhịp di chuyển mới đến điểm cuối của băng chuyền (latency = 3 chu kỳ). Nhưng kể từ thời điểm đó trở đi, cứ mỗi nhịp xung nhịp đồng hồ lại có một món hàng hoàn thiện lăn ra khỏi băng chuyền (throughput = 1 kết quả/nhịp). Vì 98.540 cặp mẫu được phát liên tiếp không nghỉ, ta thu được 98.540 kết quả chỉ sau $98.540 + 3 = 98.543$ chu kỳ. Khoảng trễ 3 chu kỳ trên tổng số gần một trăm nghìn chu kỳ chỉ chiếm tỉ lệ vỏn vẹn $3/98.543 \approx 0{,}003\%$, hoàn toàn không đáng kể.

Ba phép đo thời gian và định nghĩa RTF trong hệ thống

Trong báo cáo kiểm thử toàn tuyến (Chương 13), ta đo đạc và so sánh ba mốc thời gian độc lập:

1. **Thời gian lõi FPGA (Core Cycle Count):** Đo bằng thanh ghi đếm chu kỳ phần cứng (`cycle counter`) tích hợp trực tiếp trên FPGA, bấm giờ từ lúc cờ `start` bật đến khi cờ `done` tích cực; quy đổi $\text{RTF}_{\text{FPGA}} = \text{Số chu kỳ}/500.000$.
2. **Thời gian toàn trình HPS (End-to-End HPS Time):** Đo bằng hàm chuẩn POSIX `clock_gettime` (với nguồn xung `CLOCK_MONOTONIC`) trên hệ điều hành Linux của HPS, bao gồm toàn bộ thời gian: đọc tệp WAV từ thẻ SD, bóc tách cấu trúc, tính GP, ghi mẫu qua cầu nối bus, đợi FPGA xử lý, đọc kết quả F0 từ FIFO và ghi tệp nhị phân xuống đĩa.
3. **Thời gian Praat trên máy tính (Laptop Praat Time):** Đo trực tiếp bên trong kịch bản Praat (tệp `praat_pitch.praat`) bằng các lệnh bấm giờ nội bộ, nhằm loại trừ thời gian khởi tạo giao diện GUI và nạp thư viện hệ điều hành. Do Praat mặc định kích hoạt tính toán đa luồng (cơ chế `MelderThread`), số luồng CPU của máy tính xách tay phải được ghi nhận rõ ràng trong báo cáo đo đạc.

Mạch phần cứng Chương 0: Bộ phát cặp mẫu tự tương quan (`ch00_pair_counter`)

Để gắn kết chặt chẽ ngân sách chu kỳ lý thuyết với phần cứng thực tế, ta thiết kế một khối phần cứng tổng hợp được: `rtl/v3/ch00_pair_counter.v`. Khối này có nhiệm vụ phát tuần tự chuỗi địa chỉ $(n, n+\tau)$ để truy xuất bộ nhớ RAM hai cổng, đồng thời sinh ra các cờ điều khiển `sload` (báo nạp tích mới) và `last` (báo kết thúc lag) đúng nhịp cho khối MAC ở Chương 7.

Vì sao không có Intel IP cho khối này? Trong thư viện IP Catalog của Intel Quartus Prime, Intel cung cấp các bộ đếm chuẩn (LPM_COUNTER) hoặc bộ tạo địa chỉ DMA tuyến tính. Tuy nhiên, quy luật duyệt chỉ số tự tương quan $(n, n+\tau)$ trượt hai chiều với giới hạn $n \in [0, N-1-\tau]$ biến thiên theo từng độ trễ τ là một hàm chỉ số phi tuyến tính đặc thù. Không có bất kỳ khối IP nào hỗ trợ sẵn cấu trúc này; việc tự viết mã HDL là lựa chọn tối ưu, linh hoạt và tiết kiệm tài nguyên nhất.

Listing 1: Trích đoạn máy trạng thái và bộ phát cặp mẫu trong `rtl/v3/ch00_pair_counter.v`

```
module ch00_pair_counter #(
    parameter integer N          = 638,
    parameter integer TAU_MIN    = 26,
    parameter integer TAU_MAX    = 214,
    parameter integer ADDR_WIDTH = 10
)(
    input  wire                  clk, rst_n, start,
    input  wire [ADDR_WIDTH-1:0] cfg_n, cfg_tau_min, cfg_tau_max,
    output reg                   busy, done, valid, sload, last,
    output reg  [ADDR_WIDTH-1:0] addr_a, addr_b, tau_out,
    output reg  [31:0]           total_cycles, pair_count
);
    localparam [1:0] STATE_IDLE = 2'b00, STATE_RUN = 2'b01, STATE_DONE = 2'b10;
    reg [1:0]           state;
    reg [ADDR_WIDTH-1:0] n_cnt, tau_cnt, act_n, act_tau_min, act_tau_max;

    wire [ADDR_WIDTH-1:0] max_n = (tau_cnt == 0) ?
                                  (act_n - 1'b1) : (act_n - 1'b1 - tau_cnt);

    always @(posedge clk or negedge rst_n) begin
```

```
        if (!rst_n) begin
            state <= STATE_IDLE;
            busy <= 0; done <= 0; valid <= 0; sload <= 0; last <= 0;
            addr_a <= 0; addr_b <= 0; tau_out <= 0;
            total_cycles <= 0; pair_count <= 0;
            n_cnt <= 0; tau_cnt <= 0;
            act_n <= N; act_tau_min <= TAU_MIN; act_tau_max <= TAU_MAX;
        end else begin
            case (state)
                STATE_IDLE: begin
                    done <= 0;
                    if (start) begin
                        state <= STATE_RUN; busy <= 1; valid <= 1; sload <= 1;
                        act_n <= (cfg_n != 0) ? cfg_n : N;
                        act_tau_min <= (cfg_n != 0) ? cfg_tau_min : TAU_MIN;
                        act_tau_max <= (cfg_n != 0) ? cfg_tau_max : TAU_MAX;
                        addr_a <= 0; addr_b <= 0; tau_cnt <= 0; n_cnt <= 0;
                        last <= ((cfg_n != 0 ? cfg_n : N) == 1);
                        total_cycles <= 1; pair_count <= 1;
                    end
                end
                STATE_RUN: begin
                    total_cycles <= total_cycles + 1'b1;
                    if (n_cnt < max_n) begin
                        n_cnt <= n_cnt + 1'b1;
                        addr_a <= n_cnt + 1'b1;
                        addr_b <= (tau_cnt == 0) ?
                                  (n_cnt + 1'b1) : (n_cnt + 1'b1 + tau_cnt);
                        sload <= 0;
                        last <= (n_cnt + 1'b1 == max_n);
                        pair_count <= pair_count + 1'b1;
                    end else begin
                        if (tau_cnt == 0) begin // Xong R[0], sang TAU_MIN
                            tau_cnt <= act_tau_min; tau_out <= act_tau_min;
                            n_cnt <= 0; addr_a <= 0; addr_b <= act_tau_min;
                            sload <= 1;
                            last <= (act_n - 1'b1 - act_tau_min == 0);
                            pair_count <= pair_count + 1'b1;
                        end else if (tau_cnt < act_tau_max) begin // Sang tau ke tiep
                            tau_cnt <= tau_cnt + 1'b1; tau_out <= tau_cnt + 1'b1;
                            n_cnt <= 0; addr_a <= 0; addr_b <= tau_cnt + 1'b1;
                            sload <= 1;
                            last <= (act_n - 1'b1 - (tau_cnt + 1'b1) == 0);
                            pair_count <= pair_count + 1'b1;
                        end else begin // Hoan tat tat ca cac lag
                            valid <= 0; sload <= 0; last <= 0; busy <= 0; done <= 1;
                            state <= STATE_DONE;
                        end
                    end
                end
                STATE_DONE: begin
                    busy <= 0; valid <= 0; sload <= 0; last <= 0; done <= 1;
                    if (!start) state <= STATE_IDLE;
                end
            endcase
        end
    end
endmodule
```

Giải thích mã nguồn Verilog chi tiết từng dòng:

- **Dòng 1–12 (Khai báo cổng):** Nhận xung nhịp `clk`, reset tích cực mức thấp `rst_n`, xung kích hoạt `start`, và 3 cổng cấu hình độ dài động `cfg_n, cfg_tau_min, cfg_tau_max`. Xuất các cờ trạng thái `busy, done, valid, sload, last`, địa chỉ truy xuất RAM hai cổng `addr_a, addr_b`, chỉ số độ trễ hiện tại `tau_out`, cùng hai thanh ghi đếm chu kỳ và đếm cặp mẫu `total_cycles, pair_count`.
- **Dòng 17–18 (Tín hiệu tổ hợp `max_n`):** Biến `max_n` được tính bằng mạch logic tổ hợp (`wire`) thay vì thanh

ghi có chốt nhịp. Khi $\tau = 0$, chỉ số mẫu cuối là $N-1$; khi $\tau \geq \tau_{\min}$, chỉ số cuối là $N-1-\tau$. Việc tính tổ hợp cho phép FSM đánh giá tức thời điều kiện chạm đỉnh lag (`n_cnt == max_n`) ngay trong chu kỳ hiện tại, loại bỏ hoàn toàn độ trễ 1 nhịp xung nhịp so với việc dùng thanh ghi.

- **Dòng 30–41 (`STATE_IDLE`):** Khi nhận xung kích hoạt `start`:
  - *Dự phòng khi `cfg_n == 0` (dòng 34–36):* Biểu thức `act_n <= (cfg_n != 0) ? cfg_n : N` (tương tự cho `act_tau_min, act_tau_max`) tạo cơ chế dự phòng linh hoạt: mạch nhận tham số cấu hình động từ thanh ghi HPS khi có yêu cầu, hoặc tự động lùi về tham số tĩnh mặc định ($N = 638, \tau \in [26, 214]$) nếu phần mềm truyền giá trị 0.
  - *Khởi tạo `total_cycles` và `pair_count` bằng 1 (dòng 39):* Ngay trong chính chu kỳ nhận `start` này, cặp mẫu đầu tiên $(0, 0)$ của $R[0]$ đã bắt đầu được xuất ra bus địa chỉ cùng cờ `valid = 1, sload = 1`; do đó hai bộ đếm này phải bắt đầu từ 1 để phản ánh chính xác từng chu kỳ vật lý thực tế.
  - *Tính trước cờ `last` (dòng 38):* Biểu thức `last <= ((cfg_n != 0 ? cfg_n : N) == 1)` tính trước cờ kết thúc cho trường hợp biên suy biến $N = 1$ (chỉ có đúng 1 mẫu).
- **Dòng 42–70 (`STATE_RUN`):** Mạch thực thi phát cặp mẫu liên tục không bóng bóng:
  - *Dòng 43–51 (Quét trong cùng một độ trễ):* Tăng dần con đếm mẫu `n_cnt`. Địa chỉ cổng A nhận `addr_a <= n_cnt + 1`, và cổng B nhận `addr_b <= (tau_cnt == 0) ? (n_cnt + 1) : (n_cnt + 1 + tau_cnt)`. Việc cộng thêm 1 là kỹ thuật định địa chỉ nhìn trước (look-ahead addressing): thanh ghi chốt sẵn địa chỉ của chu kỳ xung nhịp kế tiếp để dữ liệu từ RAM M10K xuất hiện đúng thời điểm khối MAC cần. Cờ `sload` hạ về 0 để tiếp tục cộng dồn. Cờ `last` được kích hoạt sớm đúng 1 chu kỳ (`last <= (n_cnt + 1 == max_n)`) để tín hiệu kết thúc lag đi kèm đồng bộ với mẫu cuối cùng vào đường ống MAC.
  - *Dòng 52–56 (Chuyển tiếp tức thì $R[0] \to \tau_{\min}$):* Khi vừa hoàn tất mẫu cuối cùng của $R[0]$ (`tau_cnt == 0`), mạch lập tức nhảy sang `act_tau_min`, phát địa chỉ $(0, \tau_{\min})$, bật cờ `sload = 1` để khối MAC khởi tạo tổng tích lũy mới mà không cần nhịp xóa thanh ghi, và kiểm tra điều kiện biên $N-1-\tau_{\min} == 0$ (trường hợp độ trễ chỉ có đúng 1 mẫu) để chốt cờ `last` ngay.
  - *Dòng 57–64 (Chuyển sang độ trễ kế tiếp $\tau+1$):* Khi ở $\tau < \tau_{\max}$, mạch tăng τ thêm 1, reset `n_cnt = 0`, phát $(0, \tau+1)$, kích hoạt `sload = 1`, và kiểm tra trường hợp góc $N-1-(\tau+1) == 0$ cho cờ `last`. Toàn bộ quá trình chuyển lag diễn ra trong đúng 1 chu kỳ xung nhịp, không có bất kỳ nhịp chết nào.
  - *Dòng 65–68 (Nhánh hoàn tất toàn bộ các lag):* Khi hoàn thành mẫu cuối cùng của độ trễ cực đại $\tau_{\max}$, mạch hạ toàn bộ cờ `valid, sload, last`, bật cờ `done = 1` và chuyển sang `STATE_DONE`.

Mô hình tương đương Python (Twin) và Kết quả kiểm chứng iverilog: Bộ mô hình tương đương số nguyên viết bằng Python (`verify_v3/ch00_pair_counter_twin.py`) mô phỏng chính xác hành vi phát cặp mẫu và tính giá trị băm kiểm tra (checksum 32-bit):

Listing 2: Mô hình tương đương `PairCounterTwin` trong `ch00_pair_counter_twin.py`

```
class PairCounterTwin:
    def __init__(self, N, tau_min, tau_max):
        self.N, self.tau_min, self.tau_max = N, tau_min, tau_max

    def generate_pairs(self):
        pairs = []
        # Lag 0: R[0]
        for n in range(self.N):
            pairs.append({
                'a': n, 'b': n, 'tau': 0,
                'sload': int(n == 0), 'last': int(n == self.N - 1)
            })
        # Lags tau_min .. tau_max
        for tau in range(self.tau_min, self.tau_max + 1):
            max_n = self.N - 1 - tau
            for n in range(max_n + 1):
                pairs.append({
                    'a': n, 'b': n + tau, 'tau': tau,
                    'sload': int(n == 0), 'last': int(n == max_n)
```

```
                })
        return pairs
```

Giải thích cấu trúc và vận hành của mô hình Python Twin: Lớp `PairCounterTwin` đóng vai trò là quy chuẩn vàng (golden specification) độc lập về mặt giải thuật để đối soát với phần cứng:

- *Vòng lặp 1 (dòng 8–14):* Duyệt N mẫu của năng lượng $R[0]$, sinh ra các từ điển chứa địa chỉ $a = n, b = n, \tau = 0$, đánh dấu cờ nạp `sload = 1` tại mẫu đầu tiên ($n = 0$) và cờ kết thúc `last = 1` tại mẫu cuối cùng ($n = N - 1$).
- *Vòng lặp 2 (dòng 16–23):* Lần lượt duyệt qua từng độ trễ τ từ τ_{min} đến τ_{max}. Với mỗi độ trễ, số cặp mẫu tối đa là $\max_n = N - 1 - \tau$. Vòng lặp trong duyệt $n \in [0, \max_n]$, sinh ra cặp địa chỉ $a = n, b = n + \tau$, đánh dấu `sload = 1` tại $n = 0$ và `last = 1` tại $n = \max_n$.
- Danh sách từ điển trả về chứa toàn bộ chuỗi sự kiện chu kỳ đối chu kỳ, được kịch bản kiểm thử dùng để đối chiếu bit-exact với dữ liệu mô phỏng từ testbench Verilog.

Kịch bản kiểm thử (tệp `verify_v3/ch00_test_pair_counter.py`) kết nối với testbench Verilog `tb/v3/tb_ch00_pair_counter.v`, biên dịch bằng `iverilog -g2005` và thực thi bằng `vvp`. Quá trình kiểm chứng bao gồm:

- **Khẳng định cấu hình chuẩn Praat ($N = 638, \tau \in [26, 214]$):** Mạch phát đúng 98.540 cặp mẫu trong đúng 98.540 chu kỳ, sinh đúng 190 xung `sload` và 190 xung `last`. Mã băm kiểm tra chuỗi dữ liệu (canonical checksum 32-bit) đạt giá trị `0x26518cf3`, được khẳng định bằng lệnh `assert` trực tiếp trong testbench và trùng khớp hoàn toàn với mô hình Python.
- **So sánh chuỗi vết chi tiết (Trace Diff cho $N \leq 40$):** Với các cấu hình có kích thước nhỏ ($N \leq 40$), testbench xuất toàn bộ chuỗi tín hiệu chu kỳ đối chu kỳ (`valid, sload, last, addr_a, addr_b, tau`) ra tệp vết và so sánh đối chiếu từng dòng (line-by-line diff), xác nhận chuỗi phát của RTL trùng khít hoàn toàn với mô hình Python twin.
- **Khẳng định không bóng bóng trên 200 cấu hình ngẫu nhiên ($N \in [15, 80]$):** Với toàn bộ 200 cấu hình ngẫu nhiên sinh tự động, testbench kiểm tra và khẳng định điều kiện chu kỳ đối chu kỳ không bóng bóng: tổng số chu kỳ thực tế phải thỏa mãn chính xác tuyệt đối công thức total_cycles $= N + \sum_{\tau=\tau_{\text{min}}}^{\tau_{\text{max}}} (N - \tau)$ cho từng ca thử nghiệm, bảo đảm không có chu kỳ chờ (`valid = 0`) nào chen vào giữa các cặp mẫu. Toàn bộ $201/201$ ca kiểm thử đều vượt qua với 0 lỗi (lưu tại `verify_v3/ch00_pair_counter.log`).

Tóm lại: Ở 50 MHz, ngân sách thời gian thực là 500.000 chu kỳ cho mỗi bước nhảy 10 ms; khối tự tương quan cần 98.543 chu kỳ (hệ số RTF $\approx 0{,}197$), được hiện thực bằng mạch phát cặp mẫu `ch00_pair_counter` chuyển độ trễ với 0 nhịp nghỉ, đã kiểm chứng khớp hoàn toàn 201/201 ca thử nghiệm.

0.7. Bản đồ kiểm chứng ba tầng và bộ golden v2

Vấn đề thực tế: Khi hiện thực một chuỗi xử lý tín hiệu số phức tạp lên phần cứng FPGA (DE10-Standard Cyclone V), nếu ta đưa toàn bộ kiến trúc từ xử lý mẫu đến chia nội suy và xuất dữ liệu lên chip, rồi chỉ so sánh kết quả tần số cơ bản F0 cuối cùng với phần mềm Praat chạy trên máy tính, ta sẽ rơi vào bế tắc hoàn toàn khi xuất hiện sai số. Nếu một khung tín hiệu bị lệch tần số hoặc sai phân loại hữu thanh/vô thanh (V/UV), nguyên nhân bắt nguồn từ đâu?

- Do chỉ số cắt khung lệch một mẫu tại tầng HPS?
- Do khối trừ trung bình cục bộ làm tròn sai hoặc tràn số 17-bit?
- Do bảng ROM Hanning lượng tử hóa sai hệ số tỷ lệ?
- Do khối nhân-cộng tự tương quan MAC bị trễ chu kỳ xung nhịp hoặc sai tín hiệu kích hoạt nạp?
- Hay do phép chia điểm đỉnh parabol số nguyên bị mất mát độ chính xác?

Một sai lệch dù chỉ 1 mẫu hay 1 bit thấp nhất (LSB) ở tầng trước sẽ bị tích lũy và khuếch đại thành sai số hàng trăm cent hoặc sai nhãn V/UV ở tầng sau mà không để lại bất kỳ manh mối nào. Vì vậy, ta cần một kiến trúc kiểm chứng đa tầng chặt chẽ, phân định rạch ròi giữa *tính đúng đắn logic của mạch phần cứng* và *sai số xấp xỉ thuật toán so với Praat*.

0.7.1. Kiến trúc kiểm chứng ba tầng (Three-tier Verification)

Chiến lược kiểm chứng của cuốn sách được xây dựng thành ba tầng độc lập, tạo thành ”tấm lưới bảo vệ” khép kín xuyên suốt từ mã nguồn thuật toán đến chân chip FPGA (Hình 1):

1. **Tầng 1 — Khớp bit chính xác (bit-exact) giữa mô hình Python số nguyên và mô phỏng RTL (Twin vs RTL):** Tầng này so sánh đối ứng bit-exact (khớp chính xác từng bit dữ liệu và từng chu kỳ xung nhịp) giữa mô hình tương đương Python (`golden_v2/fixed_twin.py` và các script twin cục bộ của từng chương) với mô phỏng Verilog RTL (`rtl/v3/` và `tb/v3/`) chạy bằng `iverilog/vvp`. Mỗi khối phần cứng phải vượt qua ít nhất 200 véc-tơ kiểm tra ngẫu nhiên cùng các khung tín hiệu biên dồn nén (stress vectors, biên độ cực trị ± 32.768). Tiêu chuẩn nghiệm thu ở tầng 1 là 0 lỗi: chuỗi bit đầu ra và mã băm kiểm tra (checksum) của RTL phải trùng khớp hoàn toàn với mô hình Python. Tầng này loại bỏ triệt để mọi lỗi logic, lỗi tràn số và lỗi sai lệch chu kỳ giao tiếp Avalon-ST.

2. **Tầng 2 — Khớp bit chính xác (bit-exact) giữa mô phỏng RTL và phần cứng chạy thực (RTL vs Board):** Tầng này so sánh kết quả mô phỏng RTL với dữ liệu đo thực tế trên kit DE10-Standard, được trích xuất thông qua bộ phân tích logic nhúng Quartus SignalTap II hoặc đọc trực tiếp từ bộ đệm DDR của HPS qua bus Avalon-MM. Tầng 2 nhằm phát hiện các lỗi vật lý nằm ngoài tầm kiểm soát của code RTL thuần túy: khởi tạo bộ nhớ M10K qua file `.mif`/`.hex`, cấu hình bộ chia tần và độ ổn định của khối PLL (50 MHz), ràng buộc chân IO, và sự đồng bộ tín hiệu bắt tay trên bus hệ thống Platform Designer (Qsys). Tiêu chuẩn nghiệm thu: chuỗi kết quả F0 phần cứng trả về HPS phải trùng khít hoàn toàn với chuỗi kết quả của mô phỏng RTL.

3. **Tầng 3 — Đánh giá thống kê giữa phần cứng và chuẩn Praat trên máy tính (FPGA/Twin vs Praat):** Sau khi tầng 1 và tầng 2 đảm bảo phần cứng thực thi đúng đặc tả toán học số nguyên, tầng 3 tiến hành so sánh đối chiếu giữa đầu ra của FPGA (hoặc Python Twin) với phần mềm Praat 7.0.03 gốc chạy trên máy tính thông qua công cụ `golden_v2/compare.py`. Thước đo ở tầng này không còn là khớp bit mà là các chỉ số xử lý tín hiệu khách quan: VDE, GPE, FFE và phân bố sai số cent mịn (trung bình, độ lệch chuẩn, phân vị 95, cực đại). Mục tiêu là chứng minh độ lệch âm học giữa giải pháp phần cứng và Praat luôn nằm sâu dưới ngưỡng nghe nhận biết của tai người, đồng thời giải thích tường minh nguyên nhân của từng khung sai lệch.

0.7.2. Cấu trúc bộ công cụ golden v2

Toàn bộ mã nguồn kiểm chứng tham chiếu được đặt tại thư mục `golden_v2/`, đóng vai trò ”trọng tài kỹ thuật” cho toàn bộ tài liệu:

- `constants.py`: Định nghĩa tập trung mọi hằng số suy xuất trực tiếp từ mã nguồn C++ của Praat 7.0.03 (`Sound_to_Pitch.cpp`), bao gồm kiểm tra khẳng định `assert`.
- `frames.py`: Hiện thực hóa chia lưới khung ngắn hạn (`Sampled_shortTermAnalysis`) bằng số thực kép.
- `praat_float_ref.py`: Bản chuyển đổi trực tiếp thuật toán Praat 7.0.03 sang Python số thực chuẩn: trích chọn 15 ứng viên, nội suy sơ bộ parabol, tinh chỉnh độ chính xác cao sinc70 + Brent, bộ dò đường Viterbi (R_1) và đường chi phí bằng 0 (R_2), kết hợp bộ lọc trung vị median-3.
- `fixed_twin.py`: Mô hình số nguyên tương đương bit-exact toàn tuyến. Phần tài liệu docstring của tệp này chính là bản đặc tả số nguyên 10 bước chuẩn hóa mà mạch phần cứng RTL bắt buộc phải tuân theo.
- `gen_roms.py`: Kịch bản tự động sinh bảng ROM Hanning Q1.15 (`rom/hanning638_q15.hex`), ROM bù suy giảm $C[\tau]$ và chi phí $K[\tau]$ (`rom/ck_rom.hex`), cùng tệp `rom/constants.vh`.
- `compare.py`: Kịch bản tự động thực thi chuỗi so sánh đa phương diện, xuất báo cáo đối soát chi tiết (`golden_v2/out/compare_report.md`).

0.7.3. Đặc tả 10 bước số nguyên của lõi phần cứng (fixed_twin.py)

Để mạch phần cứng FPGA có thể hoạt động hoàn toàn bằng số nguyên có dấu với tài nguyên tối thiểu mà không cần khối dấu phẩy động tốn kém, thuật toán ước lượng F0 được phân rã thành 10 bước xử lý số nguyên bit-exact như sau:

1. **Bước 1 — Trừ trung bình cục bộ:** Lấy tổng tích lũy $S = \sum_{j=106}^{531} x[j]$ trên đúng 426 mẫu quanh tâm khung (ứng với hai chu kỳ dài nhất tại floor 75 Hz). Giá trị trung bình nguyên được tính bằng phép nhân-dịch thay thế bộ chia:
$$\text{mean} = \left\lfloor \frac{(S + 213 + 426 \cdot 2^{15}) \cdot 10.082.083}{2^{32}} \right\rfloor - 2^{15} = \left\lfloor \frac{S + 213}{426} \right\rfloor \tag{50}$$

2. **Bước 2 — Nhân cửa sổ Hanning:** Độ lệch $d[j] = x[j] - \text{mean}$ là số nguyên có dấu 17-bit (nằm trong khoảng $[-65.536, 65.535]$). Mẫu sau khi nhân cửa sổ Hanning định dạng Q1.15 được làm tròn và dịch phải:
$$xw[j] = (d[j] \cdot w[j] + 2^{14}) \gg 15 \tag{51}$$
với $w[j] \in [1, 32.767]$ đọc từ ROM. Khung biên cực đại có thể đạt $|xw| = 59.379$, vượt quá 16-bit nên yêu cầu thanh ghi 17-bit có dấu.

3. **Bước 3 — Đỉnh biên độ cục bộ (LP):** Tìm giá trị cực đại tuyệt đối của mẫu trong vùng trung tâm 214 mẫu quanh tâm khung:
$$LP = \max_{j=212}^{425} |xw[j]| \tag{52}$$

4. **Bước 4 — Khối nhân-cộng tự tương quan (MAC):** Tích lũy năng lượng khung $R[0] = \sum_{j=0}^{637} xw[j]^2$ và tương quan chập cho 189 độ trễ:
$$R[\tau] = \sum_{j=0}^{637-\tau} xw[j] \cdot xw[j+\tau], \quad \tau \in [26, 214] \tag{53}$$

5. **Bước 5 — Chuẩn hóa thang đo động:** Tìm số bit dịch động $sh = \max(0, \text{bitlen}(R[0]) - 26)$ để đưa $R[0]$ và toàn bộ các giá trị $R[\tau]$ về dải số nguyên 27-bit:
$$R_{0n} = R[0] \gg sh, \quad R_n[\tau] = R[\tau] \gg sh \tag{54}$$

6. **Bước 6 — Bù suy giảm cửa sổ và trừ chi phí quãng tám (Scoring):** Tích số chuẩn hóa $U[\tau]$ và điểm số đánh giá $S_c[\tau]$ được tính bằng phép nhân số nguyên với các hệ số đọc từ ROM:
$$U[\tau] = (R_n[\tau] \cdot C[\tau] + 2^{13}) \gg 14 \quad (\approx r(\tau) \cdot R_{0n}) \tag{55}$$
$$S_c[\tau] = U[\tau] - \big((R_{0n} \cdot K[\tau] + 2^{19}) \gg 20\big) \quad \left(\approx \left(r(\tau) - 0{,}01 \log_2 \frac{600}{f}\right) \cdot R_{0n}\right) \tag{56}$$
với $C[\tau] \in$ Q2.14 và $K[\tau] \in$ Q12.20.

7. **Bước 7 — Trích chọn ứng viên cực đại cục bộ:** Quét tìm các độ trễ $\tau \in [27, 213]$ thỏa mãn điều kiện tạo đỉnh cục bộ:
$$U[\tau] > U[\tau - 1], \quad U[\tau] \geq U[\tau + 1], \quad 1000 \cdot U[\tau] > 225 \cdot R_{0n} \tag{57}$$
Ứng viên tốt nhất được chọn là vị trí độ trễ τ_p có điểm số $S_c[\tau]$ lớn nhất (argmax).

8. **Bước 8 — Quyết định hữu thanh/vô thanh (Voicing):** Khung được xác nhận là hữu thanh khi và chỉ khi bất đẳng thức tích chéo số nguyên sau đây thỏa mãn:
$$(100 \cdot S_c[\tau_p] - 45 \cdot R_{0n}) \cdot 3 \cdot GP > 100 \cdot R_{0n} \cdot \max(0, 6 \cdot GP - 145 \cdot LP) \tag{58}$$
Bất đẳng thức này phản ánh chính xác điều kiện ngưỡng voicing của Praat: strength $> 0{,}45 + \max(0, 2 - I/(0{,}03/1{,}45))$ với cường độ tương đối $I = LP/GP$.

9. **Bước 9 — Nội suy đỉnh parabol và nghịch đảo tần số F0 gộp:** Lấy hiệu độ dốc $num = U[\tau_p{+}1] - U[\tau_p{-}1]$ và độ cong $den = 2U[\tau_p] - U[\tau_p - 1] - U[\tau_p + 1]$. Sau khi dịch chuẩn hóa $s_2 = \max(0, \text{bitlen}(den) - 24)$, mẫu số chung $D = 2 \cdot den_n \cdot \tau_p + num_n$. Giá trị tần số F0 định dạng Q12.4 được tính gọn bằng duy nhất một phép chia số nguyên:
$$F0_Q12.4 = \left\lfloor \frac{512.000 \cdot den_n + \lfloor D/2 \rfloor}{D} \right\rfloor \tag{59}$$

10. **Bước 10 — Bộ lọc trung vị median-3:** Đầu ra tức thời của khung là $F0_Q12.4$ nếu hữu thanh hoặc 0 nếu vô thanh; sau đó được đưa qua bộ lọc trung vị trượt 3 khung liên tiếp để triệt tiêu các xung nhảy quãng tám cô lập.

0.7.4. Kết quả đối chiếu trên tập tín hiệu tổng hợp chuẩn

Để đánh giá định lượng độ chính xác của bộ mô hình `golden_v2`, ta thực hiện kiểm thử toàn diện trên hai tệp âm thanh tổng hợp chuẩn (tần số lấy mẫu 16 kHz, độ dài 2,8 giây, gồm 277 khung phân tích, bước nhảy hop 160 mẫu, $t_1 = 0{,}020$ s). Tệp thứ nhất là tín hiệu sạch không tạp âm (`synth_glide_vuv_16k.wav`, $GP = 22.946$), tệp thứ hai được cộng thêm nhiễu trắng ở tỷ số $SNR = 5$ dB (`synth_glide_vuv_16k_snr5.wav`, $GP = 22.937$) để kích hoạt các tình huống nhảy quãng tám phức tạp. Kết quả tổng hợp từ công cụ `golden_v2/compare.py` kết hợp kịch bản trích xuất sai số cent mịn `verify_v3/ch00_table_golden.py` được trình bày trong Bảng 4.

Bảng 4: Kết quả đối chiếu toàn tuyến bộ công cụ golden v2 với Praat 7.0.03 trên tín hiệu tổng hợp chuẩn (277 khung, 16 kHz). Lưu ý: Các cột sai số cent phản ánh thống kê cent mịn (loại bỏ khung lỗi thô GPE theo đúng định nghĩa chuẩn tại Mục 0.5), khác với báo cáo `compare_report.md` vốn tính trung bình và phân vị 95 trên toàn bộ các khung cùng có thanh bao gồm cả lỗi thô.

Phép so sánh	**VDE**	**GPE**	**FFE**	**Cent TB**	**Cent p_{95}**	**Cent Max**
Tín hiệu sạch (Clean synth, 208 khung cùng hữu thanh):						
Port float vs Praat R2	0/277	0/208	0,0000	$7{,}9 \cdot 10^{-6}$	$5{,}8 \cdot 10^{-5}$	$7{,}3 \cdot 10^{-5}$
Port float vs Praat R1	0/277	0/208	0,0000	$7{,}9 \cdot 10^{-6}$	$5{,}8 \cdot 10^{-5}$	$7{,}3 \cdot 10^{-5}$
Praat R1 vs Praat R2	0/277	0/208	0,0000	0,00	0,00	0,00
Twin (trước median) vs Praat R2	0/277	0/208	0,0000	0,21	0,48	1,08
Twin + median-3 vs Praat R1	0/277	0/208	0,0000	0,37	0,64	12,96
Praat R1 vs Ground-Truth	4/277	0/207	0,0144	0,30	0,83	7,91
Twin + median-3 vs Ground-Truth	4/277	0/207	0,0144	0,53	1,05	14,28
Tín hiệu có ồn SNR 5 dB (207 khung cùng hữu thanh):						
Port float vs Praat R2	0/277	0/207	0,0000	$1{,}2 \cdot 10^{-5}$	$6{,}3 \cdot 10^{-5}$	$1{,}5 \cdot 10^{-4}$
Port float vs Praat R1	0/277	0/207	0,0000	$1{,}2 \cdot 10^{-5}$	$6{,}4 \cdot 10^{-5}$	$1{,}5 \cdot 10^{-4}$
Praat R1 vs Praat R2 (Viterbi)	0/277	19/207	0,0686	1,01	0,00	37,28
Twin (trước median) vs Praat R2	0/277	3/207	0,0108	3,07	9,78	32,91
Twin + median-3 vs Praat R1	0/277	15/207	0,0542	5,58	20,62	48,41
Praat R1 vs Ground-Truth	3/277	0/207	0,0108	10,56	21,80	29,32
Praat R2 vs Ground-Truth	3/277	19/207	0,0794	10,55	21,79	28,48
Twin (trước median) vs Ground-Truth	3/277	22/207	0,0903	9,12	20,53	25,85
Twin + median-3 vs Ground-Truth	3/277	15/207	0,0650	9,71	25,76	32,68

Đọc bảng. Bảng 4 làm sáng tỏ các mối quan hệ nền tảng của toàn bộ hệ thống:

- *Độ chính xác của bản port mã nguồn (Port float vs Praat):* Sai số cent giữa bản Python dấu phẩy động và phần mềm Praat 7.0.03 gốc chỉ ở mức 10^{-5} đến 10^{-4} cent, với $VDE = 0$ và $GPE = 0$. Con số cực nhỏ này khẳng định ta đã mô phỏng chuẩn xác thuật toán C++ của Praat đến từng phép toán dấu phẩy động kép.
- *Vai trò của bộ dò đường Viterbi (Praat R1 vs R2):* Trên tín hiệu sạch, cấu hình mặc định R_1 và cấu hình không chi phí R_2 cho kết quả trùng khít hoàn toàn. Tuy nhiên, khi chịu ảnh hưởng của tạp âm mạnh (SNR 5 dB), bộ dò đường Viterbi của R_1 sửa được 19 khung nhảy quãng tám mà R_2 (thuần túy chọn đỉnh lớn nhất từng khung) mắc phải.
- *Độ chính xác của lõi số nguyên (Twin vs Praat R2):* Trên tín hiệu sạch, mô hình số nguyên khớp sát với Praat R2: $VDE = 0$, $GPE = 0/208$, sai số cent mịn trung bình chỉ $0{,}21$ cent, $p_{95} = 0{,}48$ cent và sai số cực đại là $1{,}08$ cent (hoàn toàn không thể nhận biết bởi tai người). Trên tín hiệu ồn 5 dB, twin chỉ lệch 3/207 khung GPE so với R2 (các khung 63, 64, 172). Nguyên nhân không phải do sai số lượng tử hóa số nguyên, mà là hiện tượng hai đỉnh quãng tám cạnh tranh sít sao (near-ties): Praat xếp hạng ứng viên dựa trên độ mạnh tương quan sau khi đã tinh chỉnh bằng hàm sinc70, trong khi twin xếp hạng dựa trên điểm số trên lưới nguyên trước khi nội suy; chênh lệch điểm số giữa hai đỉnh tốt nhất ở 3 khung này chỉ từ $0{,}0005$ đến $0{,}007$, khiến twin chọn đỉnh quãng tám lân cận. Khi loại bỏ 3 khung lỗi thô này, sai số cent mịn của twin so với R2 trên tín hiệu ồn 5 dB đạt trung bình $3{,}07$ cent, $p_{95} = 9{,}78$ cent và cực đại $32{,}91$ cent.
- *Hiệu quả của bộ lọc trung vị phần cứng (Twin + median-3 vs Ground-Truth):* Trên tín hiệu ồn 5 dB, so với sự thật mặt đất (ground-truth), bộ lọc median-3 giúp giảm số khung lỗi thô GPE của phần cứng từ 22 khung xuống còn 15 khung (sửa được 7 khung lỗi quãng tám đơn lẻ), chứng minh hiệu quả rõ rệt của một cấu trúc phần cứng nhỏ gọn.

Ba khác biệt có chủ ý và phân tích hạn chế cố hữu

Để giữ sự minh bạch khoa học, ta cần chỉ rõ ba sai khác có chủ ý giữa thiết kế phần cứng FPGA và thuật toán gốc của Praat (đã được thống nhất tại Mục 0.3):

1. **Nội suy đỉnh parabol thay cho sinc70 + Brent:** Trên tín hiệu sạch, parabol cho sai số nội suy nhỏ hơn $1{,}1$ cent (sai số cực đại $1{,}08$ cent). Tuy nhiên, trên tín hiệu có ồn lớn (SNR 5 dB), hình dạng đỉnh tự tương quan bị méo mó hoặc chẻ đôi, khiến xấp xỉ parabol lệch tới 32,9 cent so với tinh chỉnh sinc70. Đây là cái giá phải trả để tiết kiệm hàng chục nghìn chu kỳ tính toán và tránh thuật toán tìm nghiệm Brent phi tuyến tính trên FPGA.
2. **Xếp hạng ứng viên bằng điểm số trên lưới rời rạc:** Thuật toán phần cứng chọn đỉnh ứng viên tốt nhất $\tau_p = \text{argmax}\, S_c[\tau]$ dựa trên các giá trị nguyên rời rạc trên lưới trước khi nội suy, thay vì tính điểm sức mạnh S_c tại tọa độ thực sau nội suy như Praat. Tại các khung có hai đỉnh quãng tám cạnh tranh sít sao (near-ties) với độ chênh lệch chi phí cục bộ cực nhỏ (chỉ từ $0{,}0005$ đến $0{,}007$ như ghi nhận trong `golden_v2/README`), sự khác biệt giữa điểm số trên lưới và độ mạnh sau nội suy sinc khiến thứ tự ưu tiên giữa hai đỉnh có thể bị hoán đổi (gây ra hiện tượng lệch 3/207 khung GPE ở tệp thử nghiệm SNR 5 dB).
3. **Bộ lọc trung vị median-3 thay cho bộ dò đường quy hoạch động Viterbi:** Vì FPGA xử lý luồng theo từng khung thời gian thực và không lưu toàn bộ ma trận ứng viên của cả tệp âm thanh vào bộ nhớ lớn, nó không thể chạy thuật toán Viterbi toàn cục. Median-3 chỉ quan sát cửa sổ cục bộ 3 khung nên chỉ sửa được các lỗi nhảy quãng tám đơn lẻ (1 khung), không thể khắc phục được chuỗi nhảy quãng tám kéo dài từ 2 khung trở lên.

Ghi chú về tập dữ liệu thử nghiệm: Toàn bộ các con số kiểm chứng trong chương này được tính toán nghiêm ngặt trên tập tín hiệu tổng hợp chuẩn (`synth_glide_vuv_16k.wav` và phiên bản SNR 5 dB). Tập dữ liệu các tệp WAV giọng nói thực tế của anh Tú hiện chưa sẵn sàng tại thời điểm xuất bản chương này; khi có dữ liệu, kịch bản tự động `golden_v2/compare.py` sẽ được tái thực thi để cập nhật bổ sung mà không làm thay đổi kiến trúc kiểm chứng ba tầng đã thiết lập.

Tóm lại: Bản đồ kiểm chứng ba tầng bảo đảm sự nhất quán chặt chẽ giữa mã nguồn Python twin số nguyên, mô phỏng RTL, phần cứng thực tế trên kit DE10-Standard và chuẩn Praat 7.0.03; trên tín hiệu sạch, lõi phần cứng đạt sai số cent trung bình chỉ $0{,}21$ *cent và* $GPE = 0$*; trên tín hiệu ồn 5 dB, sai số cent mịn là* $3{,}07$ *cent (*$p_{95} = 9{,}78$ *cent), và bộ lọc median-3 giúp giảm lỗi thô từ 22 khung xuống 15 khung.*

Bài tập cuối chương

Dưới đây là các bài tập vận dụng nhằm củng cố các nguyên lý nền tảng đã trình bày trong Chương 0. Tất cả các bài tập đều có lời giải chi tiết và số liệu đã được kiểm chứng bằng kịch bản `verify_v3/ch00_exercises.py`.

Bài tập 1 (Hiện tượng nhân đôi tần số khi đếm điểm qua không). Cho tín hiệu âm thanh liên tục có dạng:

$$s(t) = \sin(2\pi \cdot 120t) + 0{,}8\sin(2\pi \cdot 240t) \tag{60}$$

Tín hiệu được lấy mẫu ở tần số $f_s = 16$ kHz.

a) Xác định chu kỳ cơ bản thực tế T_0 và tần số cơ bản F_0 của tín hiệu.

b) Tìm tất cả các thời điểm tín hiệu đổi dấu ($s(t) = 0$) trong một chu kỳ $[0, T_0)$.

c) Nếu một thuật toán đếm số lần đổi dấu để ước lượng F0 trong khoảng thời gian quan sát $T_{\text{obs}} = 0{,}05$ s (50 ms), hãy tính tổng số điểm qua không ghi nhận được và tần số ước lượng $\hat{F}_0$. Nhận xét về kết quả này.

Lời giải chi tiết:

a) Tín hiệu $s(t)$ là tổng của hai thành phần tần số 120 Hz và 240 Hz. Ước số chung lớn nhất của 120 và 240 là 120 Hz. Do đó, tần số cơ bản thực tế là $F_0 = 120$ Hz, tương ứng với chu kỳ cơ bản:

$$T_0 = \frac{1}{120}\ \text{s} \approx 8{,}333\ \text{ms} \tag{61}$$

b) Đặt $\theta = 2\pi \cdot 120t = \omega_0 t$. Khi $t \in [0, T_0)$, góc pha biến thiên trong khoảng $\theta \in [0, 2\pi)$. Ta có:

$$s(t) = \sin(\theta) + 0{,}8\sin(2\theta) = \sin(\theta) + 1{,}6\sin(\theta)\cos(\theta) = \sin(\theta) \cdot (1 + 1{,}6\cos(\theta)) \tag{62}$$

Phương trình $s(t) = 0$ tương đương với hai trường hợp:

- $\sin(\theta) = 0 \implies \theta_1 = 0, \theta_3 = \pi \approx 3{,}1416$ rad.
- $1 + 1{,}6\cos(\theta) = 0 \implies \cos(\theta) = -\frac{1}{1{,}6} = -0{,}625$. Trong khoảng $[0, 2\pi)$, phương trình có hai nghiệm: $\theta_2 = \arccos(-0{,}625) \approx 2{,}2459$ rad và $\theta_4 = 2\pi - 2{,}2459 \approx 4{,}0373$ rad.

Như vậy, trong một chu kỳ T_0, tín hiệu $s(t)$ cắt trục hoành tại đúng 4 điểm pha:

$$\theta \in \{0;\ 2{,}2459;\ 3{,}1416;\ 4{,}0373\} \tag{63}$$

c) Trong khoảng thời gian $T_{\text{obs}} = 0{,}05$ s, số lượng chu kỳ cơ bản hoàn chỉnh là:

$$N_{\text{period}} = \frac{T_{\text{obs}}}{T_0} = \frac{0{,}05}{1/120} = 6{,}0 \text{ chu kỳ} \tag{64}$$

Mỗi chu kỳ có 4 điểm qua không, do đó tổng số lần đổi dấu trong $0{,}05$ s là:

$$N_{\text{zeros}} = 6{,}0 \times 4 = 24 \text{ điểm} \tag{65}$$

Vì mỗi chu kỳ sóng chuẩn chỉ có 2 lần đổi dấu (nửa chu kỳ dương và nửa chu kỳ âm), thuật toán ngây thơ ước lượng tần số theo công thức:

$$\hat{F}_0 = \frac{N_{\text{zeros}}}{2 \cdot T_{\text{obs}}} = \frac{24}{2 \times 0{,}05} = 240{,}0 \text{ Hz} \tag{66}$$

Nhận xét: Tần số ước lượng bị nhân đôi ($\hat{F}_0 = 240$ Hz thay vì 120 Hz), dẫn tới sai số 100% (1200 cent). Nguyên nhân là do thành phần hài âm bậc hai có biên độ lớn ($0{,}8$) đã làm uốn cong dạng sóng và tạo thêm hai điểm cắt giả trong mỗi chu kỳ.

Bài tập 2 (Độ phân giải phổ FFT và sai số cent ở tần số 150 Hz). Một thuật toán ước lượng F0 thực hiện lấy đỉnh biên độ phổ sau biến đổi Fourier nhanh (FFT) độ dài $N = 640$ mẫu tại tần số lấy mẫu $f_s = 16$ kHz.

a) Tính độ rộng của mỗi vạch phổ (bin spacing Δf).

b) Giả sử giọng người nói có tần số cơ bản thực tế rơi vào $F_{\text{true}} = 162{,}5$ Hz (nằm chính giữa hai bin kề nhau). Hãy xác định hai bin lân cận và sai số tần số cực đại nếu chọn đỉnh rời rạc lớn nhất mà không dùng thuật toán nội suy.

c) Tính sai số cực đại này theo đơn vị cent âm học so với tần số thực tế $162{,}5$ Hz. So sánh sai số này với ngưỡng nghe nhận biết cao độ (JND) của con người ($5 \ldots 10$ cent đối với âm thanh đơn âm hoặc người nghe được huấn luyện, và $10 \ldots 25$ cent đối với tiếng nói tự nhiên trong ngữ cảnh phức tạp).

Lời giải chi tiết:

a) Độ phân giải tần số của phép biến đổi FFT độ dài $N = 640$ mẫu là:

$$\Delta f = \frac{f_s}{N} = \frac{16.000}{640} = 25{,}0 \text{ Hz} \tag{67}$$

b) Các vạch phổ rời rạc có tần số $f_k = k \cdot \Delta f = k \cdot 25$ Hz ($k \in \mathbb{N}$). Với $F_{\text{true}} = 162{,}5$ Hz:

$$k = \frac{162{,}5}{25} = 6{,}5 \tag{68}$$

Tần số này nằm chính giữa bin $k = 6$ ($f_6 = 6 \times 25 = 150{,}0$ Hz) và bin $k = 7$ ($f_7 = 7 \times 25 = 175{,}0$ Hz). Nếu chỉ chọn bin có biên độ cực đại rời rạc, thuật toán sẽ chọn bin 6 hoặc bin 7. Sai số tần số cực đại trong trường hợp xấu nhất là nửa bin:

$$|\Delta F_{\text{max}}| = \frac{\Delta f}{2} = \frac{25{,}0}{2} = 12{,}5 \text{ Hz} \tag{69}$$

c) Sai số theo thang cent âm học khi làm tròn về bin 7 (175 Hz) là:

$$E_{\text{cent}} = 1200 \cdot \log_2\left(\frac{175{,}0}{162{,}5}\right) = 1200 \cdot \log_2(1{,}07692) \approx 128{,}30 \text{ cent} \tag{70}$$

Nếu làm tròn về bin 6 (150 Hz), sai số là:

$$E_{\text{cent}} = 1200 \cdot \left|\log_2\left(\frac{150{,}0}{162{,}5}\right)\right| = 1200 \cdot \log_2(1{,}08333) \approx 138{,}57 \text{ cent} \tag{71}$$

Nhận xét: Sai số $128 \ldots 138$ cent tương đương hơn 1,3 bán âm (vượt quá khoảng cách giữa nốt Đô và Đô thăng trên phím đàn piano). Trong khi đó, ngưỡng phân biệt cao độ của con người chỉ từ 5 đến 10 cent (với âm thanh đơn âm) và $10 \ldots 25$ cent (với tiếng nói phức tạp). Do đó, phương pháp tìm đỉnh FFT thô hoàn toàn không đáp ứng được yêu cầu phân tích ngữ điệu giọng nói.

Bài tập 3 (Bù suy giảm cửa sổ Hanning trong miền tự tương quan). Xét một chuỗi mẫu tín hiệu ngắn đối xứng gồm 6 điểm:

$$x = [2,\ 4,\ 6,\ 6,\ 4,\ 2] \tag{72}$$

được nhân với một cửa sổ đối xứng hình chuông:

$$w = [0{,}25,\ 0{,}50,\ 1{,}00,\ 1{,}00,\ 0{,}50,\ 0{,}25] \tag{73}$$

a) Xác định chuỗi mẫu sau khi nhân cửa sổ $xw[n] = x[n] \cdot w[n]$ ($n = 0 \ldots 5$).

b) Tính hàm tự tương quan không chuẩn hóa của tín hiệu $R_{xw}(\tau)$ tại $\tau = 0$ và $\tau = 2$.

c) Tính hàm tự tương quan của chính cửa sổ $R_w(\tau)$ tại $\tau = 0$ và $\tau = 2$.

d) Tính hệ số tương quan chưa chuẩn hóa $r_{\text{unnorm}}(2) = R_{xw}(2)/R_{xw}(0)$ và hệ số tương quan chuẩn hóa theo công thức Boersma $r(2) = \frac{R_{xw}(2)/R_w(2)}{R_{xw}(0)/R_w(0)}$. Nhận xét sự khác biệt.

Lời giải chi tiết:

a) Nhân từng phần tử giữa chuỗi x và cửa sổ w:

$$xw[0] = 2 \times 0{,}25 = 0{,}5, \quad xw[1] = 4 \times 0{,}50 = 2{,}0, \quad xw[2] = 6 \times 1{,}00 = 6{,}0 \tag{74}$$

$$xw[3] = 6 \times 1{,}00 = 6{,}0, \quad xw[4] = 4 \times 0{,}50 = 2{,}0, \quad xw[5] = 2 \times 0{,}25 = 0{,}5 \tag{75}$$

Như vậy $xw = [0{,}5,\ 2{,}0,\ 6{,}0,\ 6{,}0,\ 2{,}0,\ 0{,}5]$.

b) Tính tự tương quan của tín hiệu xw:

$$\begin{aligned} R_{xw}(0) &= \sum_{n=0}^{5} xw[n]^2 = 0{,}5^2 + 2{,}0^2 + 6{,}0^2 + 6{,}0^2 + 2{,}0^2 + 0{,}5^2 \\ &= 0{,}25 + 4{,}0 + 36{,}0 + 36{,}0 + 4{,}0 + 0{,}25 = 80{,}500 \end{aligned} \tag{76}$$

$$\begin{aligned} R_{xw}(2) &= \sum_{n=0}^{3} xw[n] \cdot xw[n+2] = (0{,}5 \times 6{,}0) + (2{,}0 \times 6{,}0) + (6{,}0 \times 2{,}0) + (6{,}0 \times 0{,}5) \\ &= 3{,}0 + 12{,}0 + 12{,}0 + 3{,}0 = 30{,}000 \end{aligned} \tag{77}$$

c) Tính tự tương quan của cửa sổ w:

$$\begin{aligned} R_w(0) &= \sum_{n=0}^{5} w[n]^2 = 0{,}25^2 + 0{,}50^2 + 1{,}00^2 + 1{,}00^2 + 0{,}50^2 + 0{,}25^2 \\ &= 0{,}0625 + 0{,}25 + 1{,}0 + 1{,}0 + 0{,}25 + 0{,}0625 = 2{,}6250 \end{aligned} \tag{78}$$

$$\begin{aligned} R_w(2) &= \sum_{n=0}^{3} w[n] \cdot w[n+2] = (0{,}25 \times 1{,}0) + (0{,}50 \times 1{,}0) + (1{,}0 \times 0{,}50) + (1{,}0 \times 0{,}25) \\ &= 0{,}25 + 0{,}50 + 0{,}50 + 0{,}25 = 1{,}5000 \end{aligned} \tag{79}$$

d) So sánh hai hệ số tương quan:

- Hệ số tương quan chưa chuẩn hóa (ngây thơ):

$$r_{\text{unnorm}}(2) = \frac{R_{xw}(2)}{R_{xw}(0)} = \frac{30{,}000}{80{,}500} \approx 0{,}3727 \tag{80}$$

- Hệ số tương quan chuẩn hóa Boersma (chia cho tự tương quan cửa sổ):

$$r(2) = \frac{R_{xw}(2)/R_w(2)}{R_{xw}(0)/R_w(0)} = \frac{30{,}000/1{,}5000}{80{,}500/2{,}6250} = \frac{20{,}000}{30{,}6667} \approx 0{,}6522 \tag{81}$$

Nhận xét và lưu ý quan trọng về tính tuần hoàn: Do hiệu ứng vát biên của cửa sổ, hệ số chưa chuẩn hóa bị nén giảm giả tạo xuống chỉ còn $0{,}3727$ (giảm tới $62{,}7\%$), có thể rơi xuống dưới ngưỡng hữu thanh $0{,}45$ và bị gán nhầm thành vô thanh. Phép chuẩn hóa Boersma đã bù đắp phần suy giảm và nâng giá trị lên $0{,}6522$. Cần lưu ý rằng chuỗi mẫu ngắn $x = [2, 4, 6, 6, 4, 2]$ bản thân không phải là một tín hiệu tuần hoàn (do có dạng đường bao tam giác hữu hạn); bài tập này minh họa rằng phép chia cho tự tương quan cửa sổ $R_w[\tau]$ triệt tiêu độ dốc suy hao do biên cửa sổ hữu hạn (taper bias), chứ không thể biến đổi một tín hiệu không tuần hoàn thành tuần hoàn.

Bài tập 4 (Tính toán các chỉ số đánh giá GPE, VDE, FFE và sai số cent). Một thử nghiệm so sánh giữa phần mềm chuẩn (Ref) và hệ thống FPGA (Test) trên 6 khung phân tích cho kết quả tần số ước lượng như sau:

Khung	Khung 1	Khung 2	Khung 3	Khung 4	Khung 5	Khung 6
F_{Ref} (Hz)	120,0	0,0	180,0	100,0	220,0	0,0
F_{Test} (Hz)	121,0	150,0	0,0	200,0	225,0	0,0

(Quy ước: Giá trị 0 Hz biểu diễn khung vô thanh).

a) Xác định các khung mắc lỗi quyết định hữu thanh và tính tỷ lệ lỗi VDE.

b) Xác định các khung cùng hữu thanh, phân loại các khung mắc lỗi bước nhảy thô và tính tỷ lệ lỗi GPE (với ngưỡng chuẩn 20%).

c) Tính tỷ lệ lỗi tổng hợp toàn khung FFE.

d) Tính sai số cent của từng khung hữu thanh không bị lỗi thô và sai số cent mịn trung bình trị tuyệt đối.

Lời giải chi tiết:

a) Tổng số khung phân tích là $N = 6$. Khung mắc lỗi quyết định hữu thanh (VDE) là khung có trạng thái V/UV giữa Ref và Test khác nhau:

- Khung 2: Ref = 0 Hz (UV), Test = 150 Hz (V) $\implies$ Lỗi VDE (nhận nhầm vô thanh thành hữu thanh).
- Khung 3: Ref = 180 Hz (V), Test = 0 Hz (UV) $\implies$ Lỗi VDE (bỏ sót khung hữu thanh).

Tổng cộng có 2 khung lỗi VDE. Tỷ lệ lỗi là:

$$\text{VDE} = \frac{2}{6} \approx 33{,}33\% \tag{82}$$

b) Các khung mà cả hai hệ thống cùng phân loại là hữu thanh gồm: Khung 1, Khung 4 và Khung 5 ($N_{\text{both}} = 3$ khung). Ta kiểm tra độ lệch tương đối $|F_{\text{Test}} - F_{\text{Ref}}|/F_{\text{Ref}}$:

- Khung 1: $|121 - 120|/120 = 1/120 \approx 0{,}83\% < 20\%$ (Hợp lệ, lỗi mịn).
- Khung 4: $|200 - 100|/100 = 100/100 = 100\% \geq 20\% \implies$ Lỗi bước nhảy thô GPE (nhảy quãng tám).
- Khung 5: $|225 - 220|/220 = 5/220 \approx 2{,}27\% < 20\%$ (Hợp lệ, lỗi mịn).

Trong số 3 khung cùng hữu thanh, có 1 khung bị lỗi GPE (Khung 4). Tỷ lệ lỗi bước nhảy thô là:

$$\text{GPE} = \frac{1}{3} \approx 33{,}33\% \tag{83}$$

c) Tập hợp các khung mắc lỗi phân loại hoặc lỗi tần số thô bao gồm: Khung 2, Khung 3 (lỗi VDE) và Khung 4 (lỗi GPE). Tổng số khung lỗi là $2 + 1 = 3$ khung. Tỷ lệ lỗi tổng hợp FFE là:

$$\text{FFE} = \frac{3}{6} = 50{,}00\% \tag{84}$$

d) Các khung cùng hữu thanh và không mắc lỗi thô gồm Khung 1 và Khung 5 ($N_{\text{fine}} = 2$):

$$C_1 = 1200 \cdot \log_2\left(\frac{121{,}0}{120{,}0}\right) = 1200 \cdot \log_2(1{,}008333) \approx +14{,}37 \text{ cent} \tag{85}$$

$$C_5 = 1200 \cdot \log_2\left(\frac{225{,}0}{220{,}0}\right) = 1200 \cdot \log_2(1{,}022727) \approx +38{,}91 \text{ cent} \tag{86}$$

Sai số cent mịn trung bình trị tuyệt đối là:

$$\overline{|C|} = \frac{|14{,}37| + |38{,}91|}{2} = \frac{53{,}28}{2} = 26{,}64 \text{ cent} \tag{87}$$

Bài tập 5 (Phân tích ngân sách chu kỳ và hệ số RTF ở tần số xung nhịp 25 MHz và 100 MHz). Một hệ thống ước lượng F0 xử lý tín hiệu âm thanh lấy mẫu ở $f_s = 16$ kHz, độ dài khung $N = 638$ mẫu, bước nhảy khung $hop = 160$ mẫu (10 ms). Khối nhân-cộng tự tương quan MAC cần 98.543 chu kỳ/khung và tổng toàn lõi xử lý ước tính cần 99.700 chu kỳ/khung.

a) Tính ngân sách chu kỳ tối đa của một bước nhảy 10 ms khi mạch làm việc ở tần số xung nhịp $f_{\text{clk}} = 25$ MHz và $f_{\text{clk}} = 100$ MHz.

b) Xác định hệ số thời gian thực của khối tự tương quan RTF_{MAC} và toàn bộ lõi xử lý RTF_{core} ở cả hai mức xung nhịp trên.

c) Nếu hệ thống hoạt động ở tần số 25 MHz, nó có đáp ứng được yêu cầu thời gian thực không? Giả sử cần ghép kênh để xử lý đồng thời $M = 8$ luồng âm thanh độc lập trong thời gian thực trên cùng một FPGA, hãy xác định tần số xung nhịp tối thiểu của lõi tuần tự này.

Lời giải chi tiết:

a) Thời gian vật lý của một bước nhảy khung là $T_{\text{hop}} = 10$ ms $= 0{,}010$ s. Ngân sách chu kỳ tối đa cho phép là:

- Ở tần số $f_{\text{clk}} = 25$ MHz:

$$N_{\text{budget}} = 25 \times 10^6 \times 0{,}010 = 250.000 \text{ chu kỳ} \tag{88}$$

- Ở tần số $f_{\text{clk}} = 100$ MHz:

$$N_{\text{budget}} = 100 \times 10^6 \times 0{,}010 = 1.000.000 \text{ chu kỳ} \tag{89}$$

b) Hệ số thời gian thực $\text{RTF} = N_{\text{cycles}}/N_{\text{budget}}$:

- Ở tần số 25 MHz:

$$\text{RTF}_{\text{MAC}} = \frac{98.543}{250.000} \approx 0{,}3942 \tag{90}$$

$$\text{RTF}_{\text{core}} = \frac{99.700}{250.000} \approx 0{,}3988 \tag{91}$$

- Ở tần số 100 MHz:

$$\text{RTF}_{\text{MAC}} = \frac{98.543}{1.000.000} \approx 0{,}0985 \tag{92}$$

$$\text{RTF}_{\text{core}} = \frac{99.700}{1.000.000} \approx 0{,}0997 \tag{93}$$

c) Đánh giá khả năng thời gian thực và ghép kênh:

- Vì ở 25 MHz, $\text{RTF}_{\text{core}} \approx 0{,}3988 < 1{,}0$, hệ thống hoàn toàn đáp ứng yêu cầu xử lý thời gian thực. Lõi phần cứng chỉ sử dụng khoảng $39{,}9\%$ năng lực xử lý của xung nhịp và có thời gian nghỉ rỗi hơn 60%.
- Để xử lý đồng thời $M = 8$ kênh âm thanh trong mỗi khoảng thời gian 10 ms, tổng số chu kỳ xử lý cần thiết của toàn bộ 8 kênh là:

$$N_{\text{total}} = 8 \times 99.700 = 797.600 \text{ chu kỳ} \tag{94}$$

  Tần số xung nhịp tối thiểu để hoàn thành 797.600 chu kỳ trong 10 ms là:

$$f_{\text{min}} = \frac{N_{\text{total}}}{T_{\text{hop}}} = \frac{797.600}{0{,}010 \text{ s}} = 79.760.000 \text{ Hz} = 79{,}76 \text{ MHz} \tag{95}$$

Kết luận: Một mức xung nhịp hệ thống phổ thông 100 MHz trên Cyclone V (giả định cần được xác nhận bằng đo đạc F_{max} thực tế trên bo mạch – phải đo) lớn hơn $79{,}76$ MHz, hoàn toàn đủ khả năng ghép kênh xử lý cùng lúc 8 luồng âm thanh thời gian thực chỉ với duy nhất một bộ nhân-cộng DSP tuần tự!

Chương 1. WAV trên HPS và cầu HPS–FPGA, chiều đi và chiều về

▶ MỤC TIÊU HỌC TẬP

- Hiểu rõ bản chất vấn đề phân chia nhiệm vụ và truyền nhận dữ liệu giữa hai miền xử lý độc lập trên chip Cyclone V SoC: bộ vi xử lý HPS (ARM Cortex-A9) và vùng logic khả trình FPGA.
- Nắm vững cấu trúc nhị phân của tệp RIFF/WAVE theo từng byte, hiểu rõ lý do vì sao việc nhảy cóc cố định vào độ lệch offset 44 dẫn tới sai lệch dữ liệu nghiêm trọng, và xây dựng bộ bóc tách định dạng WAV tin cậy bằng ngôn ngữ C trên nền tảng Linux.
- Dẫn xuất công thức tính đỉnh biên độ toàn cục globalPeak (GP) của Praat bằng số học nguyên chính xác, triệt tiêu hoàn toàn sự phụ thuộc vào số thực dấu phẩy động trên phần cứng.
- Phân tích tường tận thuật toán tạo lưới phân tích khung ngắn hạn (`Sampled_shortTermAnalysis` và `Sampled_xToLowIndex`), giải mã hiện tượng bẫy sàn số lẻ (odd-sample floor trap) và cách thức mã nguồn HPS bảo vệ tính đều đặn của bước nhảy khung.
- Đánh giá định lượng thông lượng truyền thông qua ba phương án phần cứng: PIO qua cầu Lightweight HPS-to-FPGA, DMA thông qua cầu Full Bridge, và bộ điều khiển mSGDMA; hiểu rõ sự đánh đổi giữa thông lượng, độ phức tạp phần mềm và việc bảo toàn tính nhất quán bộ đệm cache.
- Nắm vững kiến trúc và từng tham số cấu hình của khối IP hàng đợi Intel `altera_avalon_fifo` (On-Chip FIFO Memory), giải thích cơ chế phân rã từ dữ liệu 32-bit thành hai mẫu 16-bit theo thứ tự byte nhỏ (little-endian) và tài nguyên khối nhớ M10K.
- Tự thiết kế và hiện thực khối hàng đợi Avalon-MM write slave sang Avalon-ST source bằng ngôn ngữ Verilog-2001, hiểu rõ cơ chế con trỏ có bit vòng, hành vi tín hiệu `waitrequest` bảo vệ bus và cơ chế phản áp dòng chảy dữ liệu.
- Định nghĩa tường minh bản đồ không gian thanh ghi điều khiển cho toàn bộ lõi xử lý F0 và lập trình trình điều khiển (C driver) tương tác qua giao tiếp ánh xạ bộ nhớ `mmap` trên Linux.

1.1. Vấn đề truyền dữ liệu giữa hai miền HPS và FPGA

Một hệ thống nhúng thực tế không bao giờ bắt đầu bằng một khối logic đứng cô lập. Tín hiệu tiếng nói thu nhận từ micro hoặc lưu trữ sẵn dưới dạng tệp âm thanh WAV được đặt trong hệ thống tệp tin định dạng FAT32 hoặc ext4 trên thẻ nhớ SD. Thẻ nhớ này kết nối trực tiếp với bộ điều khiển ngoại vi của hệ thống vi xử lý cứng HPS (Hard Processor System, gồm hai lõi ARM Cortex-A9 MPCore chạy hệ điều hành Linux) trên bo mạch DE10-Standard (Cyclone V SoC 5CSXFC6D6F31C6N). Trong khi đó, toàn bộ chuỗi xử lý tín hiệu số chuyên dụng tốc độ cao—từ các phép tích lũy tự tương quan MAC, tra cứu trọng số cửa sổ Hanning, tìm đỉnh parabol cho đến bộ lọc trung vị median-3—lại được hiện thực hóa bên trong lưới logic khả trình FPGA.

Sự thật kỹ thuật đầu tiên cần đối mặt là: **mẫu âm thanh nằm trong bộ nhớ DDR của HPS, còn logic tính toán lại nằm ở FPGA**. Hai miền này thuộc về hai thế giới hoàn toàn tách biệt:

- Miền vi xử lý HPS hoạt động ở xung nhịp cao (thường là 800 MHz đến 925 MHz), có bộ quản lý bộ nhớ MMU (Memory Management Unit), cơ chế dịch địa chỉ ảo (virtual address translation) và bộ nhớ đệm đa tầng (L1/L2 cache).
- Miền logic FPGA hoạt động ở xung nhịp đồng bộ của hệ thống lõi (trong thiết kế này được ấn định là 50 MHz), giao tiếp theo từng chu kỳ xung nhịp xác định thông qua các đường bus chuẩn công nghiệp.

Để kết nối hai miền xử lý này, kiến trúc phần cứng Cyclone V SoC đúc sẵn ba cầu nối giao tiếp AXI (Advanced eXtensible Interface) chuyên dụng đi xuyên qua ranh giới vật lý giữa HPS và FPGA:

1. **Cầu HPS-to-FPGA (Full Bridge):** Đây là cầu nối master tốc độ cao do vi xử lý làm chủ, hỗ trợ bề rộng dữ liệu cấu hình được là 32, 64 hoặc 128 bit, xung nhịp độc lập với miền HPS và được đồng bộ vào miền xung nhịp FPGA. Cầu nối này chiếm một không gian địa chỉ vật lý rộng lớn (từ `0xC0000000` đến `0xFBFFFFFF`, dung lượng gần 1 GB), chuyên chở các khối dữ liệu khổng lồ như khung hình video hoặc bộ đệm mạng.
2. **Cầu Lightweight HPS-to-FPGA (LWHPS-to-FPGA Bridge):** Đây là cầu nối master thứ hai từ phía HPS, nhưng được thu gọn với bề rộng dữ liệu cố định đúng 32 bit và độ trễ truy xuất thấp. Cầu nối này được ánh xạ cố định vào cửa sổ địa chỉ vật lý `0xFF200000` với độ mở span đúng 2 MB (tới `0xFF3FFFFF`). Cầu Lightweight được sinh ra nhằm mục đích điều khiển các thanh ghi trạng thái (CSR) và các cổng ngoại vi giao tiếp nhẹ mà không làm tiêu tốn tài nguyên quản lý bus phức tạp của vi xử lý.

3. **Cầu FPGA-to-HPS:** Đây là cầu nối theo chiều ngược lại, cho phép các bộ master nằm trong vùng logic FPGA (tiêu biểu như bộ điều khiển DMA) chủ động phát lệnh đọc hoặc ghi trực tiếp vào không gian bộ nhớ DDR3 của HPS.

Câu hỏi thiết kế đặt ra là: **Ta nên chọn cầu nào và cơ chế vận chuyển nào cho bài toán ước lượng cao độ F_0?**

Nếu ta dồn toàn bộ việc đọc tệp, giải mã tệp WAV và tính toán lưới khung vào FPGA, ta sẽ phải hiện thực một bộ điều khiển hệ thống tệp (FAT32/ext4) phức tạp trên phần cứng, tiêu tốn hàng nghìn khối logic ALM vô ích mà không đem lại tốc độ nhanh hơn. Ngược lại, nếu ta dồn toàn bộ thuật toán tự tương quan vào vi xử lý ARM, việc tính toán 98.540 phép nhân-cộng trên từng khung 10 ms sẽ chiếm dụng đáng kể thời gian của CPU, làm giảm khả năng đáp ứng của các tác vụ hệ thống khác.

Giải pháp phân chia tối ưu là thực hiện một ranh giới rõ ràng:

- Phía HPS chạy chương trình bằng ngôn ngữ C trên Linux, đảm nhiệm việc bóc tách định dạng tệp WAV, kiểm tra tính hợp lệ của âm thanh, tính toán các tham số thống kê toàn cục (globalPeak, số khung, mốc thời gian t_1, mẫu bắt đầu của từng khung) và quản lý tiến trình.
- Phía FPGA đón nhận luồng dữ liệu mẫu âm thanh thông qua bộ đệm hàng đợi (FIFO), tự động thực hiện chuỗi thuật toán xử lý tín hiệu số ở tốc độ phần cứng, và gửi kết quả cao độ F_0 cùng các cờ trạng thái trở lại cho HPS.

Việc truyền tải dữ liệu giữa hai miền thông qua cầu nối không chỉ đơn thuần là đẩy dữ liệu lên dây dẫn mà còn phải đảm bảo không bao giờ làm tràn bộ nhớ, không làm rơi rụng mẫu âm thanh và không gây nghẽn bus hệ thống. Trong các tiểu mục tiếp theo, ta sẽ lần lượt bóc tách chi tiết từng khâu: từ giải mã tệp WAV, lựa chọn cầu nối với các con số định lượng, thiết kế bộ đệm FIFO ở cả hai hướng (IP và HDL tự viết), cho đến việc thiết lập bản đồ thanh ghi điều khiển thống nhất.

Tóm lại: Dữ liệu âm thanh nằm trong DDR của vi xử lý còn thuật toán tính toán nằm ở logic FPGA; bài toán trung tâm là phân định ranh giới xử lý và lựa chọn cầu nối sao cho việc truyền tải vừa đủ nhanh để đáp ứng thời gian thực, vừa giữ cấu trúc phần cứng và trình điều khiển ở mức đơn giản nhất.

1.2. Đọc tệp WAV đúng chuẩn và tính toán tham số toàn cục trên HPS

Cạm bẫy ngây thơ: Vì sao giả định dữ liệu ở offset 44 luôn thất bại?

Một sai lầm kinh điển mà người mới làm việc với âm thanh số trên FPGA thường mắc phải là mở tệp WAV, bỏ qua 44 byte đầu tiên rồi giả định rằng mẫu âm thanh PCM bắt đầu cố định từ byte thứ 44. Con số 44 byte này chỉ đúng trong trường hợp lý tưởng tối giản nhất: một tệp chỉ chứa duy nhất khối `RIFF` tiêu đề (12 byte), một khối định dạng `fmt` cơ bản không có phần mở rộng (24 byte gồm 8 byte tiêu đề và 16 byte dữ liệu), và 8 byte tiêu đề của khối `data` ($12 + 24 + 8 = 44$).

Tuy nhiên, trong thực tế sản xuất âm thanh, hầu hết các phần mềm biên tập chuẩn (như Audacity, Adobe Audition, Praat hay các công cụ ghi âm hiện đại) đều tự động chèn thêm các khối dữ liệu phụ trợ (auxiliary metadata chunks) vào tiêu đề để lưu trữ thông tin bản quyền, tên nghệ sĩ, mốc thời gian ghi âm hoặc thông số phát sóng. Tiêu biểu nhất là khối danh sách thông tin `LIST` (chứa các thẻ con như `INFO`, `INAM`, `IART`) hoặc khối thông tin phát sóng `bext` (Broadcast Audio Extension). Các khối này thường được đặt ngay sau khối `fmt` và nằm phía trước khối `data`.

Để thấy rõ sự sụp đổ này bằng số liệu cụ thể, hãy kiểm tra tệp âm thanh thử nghiệm `edge_list_chunk.wav` (tại thư mục `verify_v3/data/`, tạo bởi `make_test_wavs.py`). Tệp này chèn một khối `LIST` kích thước 18 byte mang nhãn tiêu đề bản ghi "DE10" nằm xen vào giữa khối `fmt` và khối `data`.

Quan sát trích xuất hex dump thực tế từ tệp `edge_list_chunk.wav`:

Offset	Dữ liệu Hex	Ký tự ASCII	Nhận diện khối
0x0000	52 49 46 46 7e 01 00 00	RIFF~...	Khối `RIFF` tiêu đề tệp
0x0008	57 41 56 45 66 6d 74 20	WAVEfmt	Định dạng WAVE + Khối `fmt`
0x0010	10 00 00 00 01 00 01 00	....	16 byte, PCM không nén, mono
0x0018	80 3e 00 00 00 7d 00 00	.>...}..	16.000 Hz, 32.000 byte/s
0x0020	02 00 10 00 4c 49 53 54	....LIST	16-bit, bắt đầu khối `LIST`
0x0028	12 00 00 00 49 4e 46 4f	....INFO	Kích thước 18 byte, nhãn `INFO`
0x0030	49 4e 41 4d 06 00 00 00	INAM....	Thẻ `INAM`, độ dài 6 byte
0x0038	44 45 31 30 00 00 64 61	DE10..da	Dữ liệu "DE10", đầu khối `data`
0x0040	74 61 40 01 00 00 00 00	ta@.....	Ký tự `ta`, kích thước 320 byte

Nếu một bộ đọc ngây thơ nhảy cóc thẳng vào vị trí byte 44 (`0x002C`), nó sẽ đọc đúng 4 byte ký tự ASCII `"INFOINAM"` (mã hex: `49 4E 46 4F`) và nhầm tưởng đó là các mẫu âm thanh PCM 16-bit!

- Hai byte đầu tiên tại offset 44 là `0x49` và `0x4E`. Theo quy ước số nguyên có dấu bù hai byte nhỏ (little-endian), chúng ghép lại thành mẫu số nguyên:

$$x_{\text{rác}}[0] = \texttt{0x4E49} = +20.041 \tag{96}$$

- Hai byte tiếp theo tại offset 46 là `0x46` và `0x4F`, ghép lại thành mẫu số nguyên:

$$x_{\text{rác}}[1] = \texttt{0x4F46} = +20.294 \tag{97}$$

Hai mẫu rác này có biên độ cực lớn (vượt quá 61% thang đo cực đại của PCM 16-bit), trong khi khối âm thanh thật sự `data` phải tới byte 62 (`0x003E`) mới xuất hiện và các mẫu âm thanh thực tế chỉ bắt đầu từ byte 70 (`0x0046`). Hệ quả là:

1. Âm thanh giải mã bị lệch pha đúng 26 byte (13 mẫu âm thanh), làm toàn bộ các khung phân tích bị trôi mốc thời gian so với phần mềm Praat.
2. Hai mẫu văn bản bị đọc nhầm thành hai xung đột biến biên độ > 20.000, tạo ra một tiếng nổ "bụp" chát chúa ở đầu tệp và làm sai lệch nghiêm trọng giá trị đỉnh biên độ toàn cục globalPeak, từ đó phá hỏng toàn bộ logic cổng im lặng ở Chương 10.

Cấu trúc phân cấp chuẩn của tệp RIFF/WAVE từ mức byte

Định dạng tệp WAV thuộc họ tệp hoán đổi tài nguyên RIFF (Resource Interchange File Format) do Microsoft và IBM đồng phát triển. Một tệp RIFF được tổ chức thành một cây chứa các khối (chunks) lồng nhau. Mỗi khối bắt buộc phải có cấu trúc tiêu đề 8 byte mở đầu gồm:

- **Mã nhận diện khối (Chunk ID):** 4 byte ký tự ASCII (FourCC, Four Character Code) xác định loại khối.
- **Kích thước khối (Chunk Size):** số nguyên không dấu 32-bit ở định dạng byte nhỏ (little-endian), cho biết độ dài chính xác của phần dữ liệu bên trong khối (không tính 8 byte tiêu đề của chính nó).

Tiêu đề tệp bắt đầu bằng chuỗi nhận diện `"RIFF"`, theo sau là kích thước toàn tệp trừ 8 byte đầu, và mã định dạng `"WAVE"`. Sau 12 byte đầu tiên này là một chuỗi các khối nằm liên tiếp nhau. Một trình đọc chuẩn bắt buộc phải duyệt tuần tự qua từng khối theo thuật toán nhảy cóc (chunk jumping): đọc 4 byte ID, đọc 4 byte kích thước S, kiểm tra xem khối đó có phải là khối ta cần hay không; nếu không phải, dùng hàm dịch chuyển con trỏ tệp (`fseek`) để nhảy qua đúng S byte nhằm đến đầu khối tiếp theo.

Quy tắc đệm từ của chuẩn RIFF (Word Alignment Padding): Đây là một chi tiết kỹ thuật cốt lõi thường bị bỏ quên. Chuẩn kỹ thuật RIFF quy định: **mọi khối bắt buộc phải bắt đầu tại một địa chỉ byte chẵn**. Do đó, nếu kích thước dữ liệu S của một khối là một số lẻ (tức $S \pmod 2 \neq 0$), bên ghi tệp bắt buộc phải chèn thêm đúng 1 byte đệm có giá trị `0x00` vào ngay sau dữ liệu của khối đó. Khi nhảy qua khối, trình đọc phải dịch chuyển con trỏ một khoảng là:

$$\Delta_{\text{jump}} = S + (S \,\&\, 1) \tag{98}$$

Nếu bỏ qua việc kiểm tra bit lẻ này, khi gặp các khối siêu dữ liệu có kích thước lẻ (như tệp `edge_odd_chunk.wav` trong bộ thử nghiệm), con trỏ tệp sẽ bị lệch đi 1 byte lẻ. Toàn bộ các mã FourCC tiếp theo bị đọc lệch thành các chuỗi byte vô nghĩa và quá trình giải mã thất bại hoàn toàn.

Một trình đọc WAV đúng chuẩn trên vi xử lý HPS bắt buộc phải kiểm tra nghiêm ngặt 5 điều kiện hợp lệ của khối `fmt` :

1. `AudioFormat == 1`: bắt buộc là định dạng PCM tuyến tính không nén. Các định dạng nén như A-law, μ-law hay IEEE Float đều bị từ chối với mã lỗi `WAV_ERR_UNSUPPORTED_FORMAT`.
2. `NumChannels == 1`: bắt buộc là kênh đơn (mono). Nếu tệp có hai kênh (stereo, như `edge_stereo.wav`), dữ liệu hai kênh trái-phải đan xen sẽ làm hỏng hoàn toàn phép tính tự tương quan; trình đọc từ chối với mã lỗi `WAV_ERR_UNSUPPORTED_CHANNELS`.

3. `SampleRate == 16000`: tần số lấy mẫu bắt buộc là 16 kHz để phù hợp với thông số thiết kế cửa sổ 40 ms và dải quét độ trễ của FPGA. Tệp có tần số khác (như 44,1 kHz trong `edge_wrong_rate.wav`) bị từ chối với mã `WAV_ERR_UNSUPPORTED_RATE`.
4. `BitsPerSample == 16`: mỗi mẫu âm thanh biểu diễn bởi đúng 16 bit số nguyên có dấu bù hai (dải biểu diễn từ -32.768 đến $+32.767$).
5. `BlockAlign == 2`: mỗi mẫu chiếm đúng 2 byte dữ liệu.

Ví dụ bóc tách tệp thực tế: Tệp mẫu thu nhỏ `mini_speech.wav`

Để quan sát tường tận cấu trúc nhị phân ở mức từng byte, hãy xem xét toàn bộ nội dung của tệp âm thanh thu nhỏ `mini_speech.wav` gồm đúng 8 mẫu âm thanh:

$$s = [0, 100, -200, 300, -400, 500, -600, 700] \tag{99}$$

Địa chỉ Offset	Giá trị Hex (Little-Endian)	Ý nghĩa giải mã kỹ thuật
`0x0000 -- 0x0003`	`52 49 46 46`	Ký tự FourCC: `"RIFF"`
`0x0004 -- 0x0007`	`34 00 00 00`	Kích thước phần còn lại: 52 byte ($60 - 8 = 52$)
`0x0008 -- 0x000B`	`57 41 56 45`	Định dạng tài nguyên: `"WAVE"`
`0x000C -- 0x000F`	`66 6d 74 20`	Ký tự FourCC: `"fmt "`
`0x0010 -- 0x0013`	`10 00 00 00`	Kích thước khối định dạng: 16 byte cơ bản
`0x0014 -- 0x0015`	`01 00`	`AudioFormat = 1` (PCM không nén)
`0x0016 -- 0x0017`	`01 00`	`NumChannels = 1` (kênh đơn mono)
`0x0018 -- 0x001B`	`80 3e 00 00`	`SampleRate = 16.000 Hz` ($0x00003E80 = 16.000$)
`0x001C -- 0x001F`	`00 7d 00 00`	`ByteRate = 32.000 byte/s` ($16.000 \times 2 = 32.000$)
`0x0020 -- 0x0021`	`02 00`	`BlockAlign = 2 byte/mẫu`
`0x0022 -- 0x0023`	`10 00`	`BitsPerSample = 16 bit/mẫu`
`0x0024 -- 0x0027`	`64 61 74 61`	Ký tự FourCC: `"data"`
`0x0028 -- 0x002B`	`10 00 00 00`	Kích thước dữ liệu âm thanh: 16 byte (8 mẫu)
`0x002C -- 0x002D`	`00 00`	Mẫu 0: 0
`0x002E -- 0x002F`	`64 00`	Mẫu 1: $+100$ (0x0064)
`0x0030 -- 0x0031`	`38 ff`	Mẫu 2: -200 (0xFF38)
`0x0032 -- 0x0033`	`2c 01`	Mẫu 3: $+300$ (0x012C)
`0x0034 -- 0x0035`	`70 fe`	Mẫu 4: -400 (0xFE70)
`0x0036 -- 0x0037`	`f4 01`	Mẫu 5: $+500$ (0x01F4)
`0x0038 -- 0x0039`	`a8 fd`	Mẫu 6: -600 (0xFDA8)
`0x003A -- 0x003B`	`bc 02`	Mẫu 7: $+700$ (0x02BC)

Trình đọc C trong tệp `sw_v3/ch01_wav.c` hiện thực hóa chính xác giải thuật duyệt khối này và đã được kiểm chứng vượt qua toàn bộ 8 tệp kiểm thử (từ tệp chuẩn tới các tệp biên lỗi) trong kịch bản `ch01_wav_verify.py`.

Dẫn xuất đỉnh biên độ toàn cục globalPeak bằng số nguyên

Trong mã nguồn Praat `Sound_to_Pitch.cpp` (dòng 375–383), trước khi chia khung, Praat tính toán đỉnh biên độ toàn cục của toàn bộ tệp âm thanh:

$$\mathrm{GP}_{\mathrm{praat}} = \max_{1 \le i \le n} |x[i] - \bar{x}| \tag{100}$$

trong đó $\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x[i]$ là giá trị trung bình DC của toàn bộ tệp và mẫu âm thanh $x[i] \in [-1{,}0, +1{,}0)$ được biểu diễn dưới dạng số thực dấu phẩy động đôi (double).

Đại lượng này đóng vai trò sống còn trong việc đánh giá mức cường độ âm thanh tương đối:

$$\text{intensity} = \frac{\text{localPeak}}{\text{globalPeak}} \tag{101}$$

ở Chương 10. Nếu một đoạn âm thanh có biên độ nhỏ nhưng vẫn tuần hoàn rõ rệt (ví dụ khi người nói phát âm thì thầm hoặc nói nhỏ dần ở cuối câu), năng lượng tuyệt đối $R[0]$ giảm xuống rất thấp. Tuy nhiên, tỉ số

localPeak/globalPeak vẫn giữ được giá trị đáng kể, giúp bộ phân loại vô thanh/hữu thanh không bị nhầm lẫn tiếng nói nhỏ thành khoảng lặng hoàn toàn.

Chuyển đổi sang số nguyên không mất độ chính xác: Trên vi xử lý nhúng HPS, ta hoàn toàn có thể tính toán đại lượng này bằng số thực `double`. Tuy nhiên, để cấu hình cho thanh ghi 16-bit của FPGA và đảm bảo không xảy ra bất kỳ sự phân kỳ nào giữa các trình biên dịch, ta quy đổi công thức trên về số nguyên thuần túy. Mẫu số nguyên 16-bit $s[i]$ liên hệ với mẫu thực của Praat qua tỉ lệ $x[i] = s[i]/32.768$. Thay vào công thức trung bình:

$$x[i] - \bar{x} = \frac{s[i]}{32.768} - \frac{\sum_{j=1}^{n} s[j]}{n \cdot 32.768} = \frac{n \cdot s[i] - \sum_{j=1}^{n} s[j]}{n \cdot 32.768} \tag{102}$$

Để đưa giá trị độ lệch cực đại về cùng thang đo với mẫu số nguyên 16-bit (tức nhân lại với 32.768), ta nhận được công thức nguyên chính xác:

$$\text{GP} = \text{round}\left(\frac{\max_{1 \le i \le n} |n \cdot s[i] - S|}{n}\right) \tag{103}$$

với $S = \sum_{j=1}^{n} s[j]$ là tổng tích lũy của toàn bộ n mẫu trong tệp.

Nhìn thấy toán học: Rút gọn phép tìm cực đại về hai điểm biên. Quan sát hàm bậc nhất $f(s) = n \cdot s - S$. Đây là một hàm đơn điệu tăng nghiêm ngặt theo giá trị mẫu s. Do đó, độ lớn tuyệt đối $|n \cdot s - S|$ là một hàm lồi (convex), và giá trị cực đại của nó trên toàn bộ tập mẫu **chỉ có thể đạt được tại một trong hai điểm cực trị**: mẫu có giá trị nhỏ nhất $s_{\text{min}} = \min_i s[i]$ hoặc mẫu có giá trị lớn nhất $s_{\text{max}} = \max_i s[i]$!

Nhờ định lý đơn giản này, mã nguồn C trên HPS không cần phải lưu trữ toàn bộ mảng mẫu rồi quét lại lần thứ hai. Ngay trong lượt đọc tệp đầu tiên, ta đồng thời tính tổng tích lũy S (dùng biến 64-bit `int64_t` để chống tràn), cập nhật s_{min} và s_{max}. Sau khi đọc xong n mẫu, giá trị GP được tính trực tiếp trong đúng một phép so sánh:

$$\Delta_1 = |n \cdot s_{\text{max}} - S| \tag{104}$$

$$\Delta_2 = |n \cdot s_{\text{min}} - S| \tag{105}$$

$$\text{max_diff} = \max(\Delta_1, \Delta_2) \tag{106}$$

$$\text{GP} = \left\lfloor \frac{\text{max_diff} + \lfloor n/2 \rfloor}{n} \right\rfloor \tag{107}$$

trong đó việc cộng thêm $\lfloor n/2 \rfloor$ trước khi chia nguyên thực hiện chính xác phép làm tròn số học (`round`) về số nguyên gần nhất.

Ví dụ tính tay trên dãy mẫu thu nhỏ (`mini_speech.wav`):
Xét dãy 8 mẫu âm thanh:

$$s = [0, 100, -200, 300, -400, 500, -600, 700] \tag{108}$$

- Số lượng mẫu: $n = 8$.
- Tổng tích lũy: $S = 0 + 100 - 200 + 300 - 400 + 500 - 600 + 700 = 400$.
- Giá trị cực tiểu: $s_{\text{min}} = -600$; Giá trị cực đại: $s_{\text{max}} = 700$.
- Độ lệch tại cực tiểu: $\Delta_2 = |8 \times (-600) - 400| = |-4.800 - 400| = |-5.200| = 5.200$.
- Độ lệch tại cực đại: $\Delta_1 = |8 \times 700 - 400| = |5.600 - 400| = 5.200$.
- Cực đại độ lệch: $\text{max_diff} = 5.200$.
- Giá trị GP:

$$\text{GP} = \left\lfloor \frac{5.200 + \lfloor 8/2 \rfloor}{8} \right\rfloor = \left\lfloor \frac{5.204}{8} \right\rfloor = 650 \tag{109}$$

Con số 650 thu được khớp chính xác với kiểm chứng tự động của kịch bản `ch01_wav_verify.py`.

Áp dụng trên hai tệp âm thanh chuẩn 2,8 s (44.800 mẫu) của đề cương (mục 7):

- Tệp âm thanh sạch (`synth_glide_vuv_16k.wav`): $s_{\text{max}} = 22.939$, $s_{\text{min}} = -22.939$, tổng tích lũy $S = -317.078$, tính ra $\text{GP} = \mathbf{22.946}$ (đã kiểm bằng `ch01_wav_verify.py`).
- Tệp âm thanh có ồn SNR 5 dB (`synth_glide_vuv_16k_snr5.wav`): $s_{\text{max}} = 22.938$, $s_{\text{min}} = -22.278$, tính ra $\text{GP} = \mathbf{22.937}$ (đã kiểm bằng `ch01_wav_verify.py`).

Lưới phân tích khung ngắn hạn: Dẫn xuất thuật toán của Praat

Một điểm khác biệt sâu sắc giữa Praat và các hệ thống xử lý tín hiệu thông thường là cách xác định tâm thời gian của các khung. Trong các giáo trình cơ bản, người ta thường cố định khung 0 bắt đầu từ mẫu 0 ($t = 0$), rồi tịnh tiến đều đặn theo từng bước nhảy hop. Ngược lại, Boersma thiết kế thuật toán phân tích ngắn hạn `Sampled_shortTermAnalysis` (`fon/Sampled.cpp` dòng 74) theo nguyên lý **căn giữa đối xứng**: toàn bộ chuỗi khung phân tích phải được đặt đối xứng hoàn hảo xung quanh tâm của tệp âm thanh.

Các bước tính toán diễn ra tuần tự như sau:

1. **Xác định số lượng khung phân tích n_{frames}:** Với tần số lấy mẫu $f_s = 16.000$ Hz, chu kỳ lấy mẫu là $dx = 1/16.000 = 0{,}0000625$ s. Thời lượng vật lý của tệp âm thanh có n_x mẫu là:

$$\text{my_duration} = n_x \cdot dx \tag{110}$$

   Độ dài cửa sổ vật lý là $\Delta t_{\text{window}} = 3/75 = 0{,}04$ s (tương ứng 640 mẫu danh định, Praat làm tròn thành 638 mẫu). Bước nhảy thời gian danh định giữa hai khung là $\Delta t = 0{,}75/75 = 0{,}01$ s (tương ứng hop $= 160$ mẫu). Số lượng khung phân tích hợp lệ được tính bằng:

$$n_{\text{frames}} = \left\lfloor \frac{\text{my_duration} - \Delta t_{\text{window}}}{\Delta t} \right\rfloor + 1 \tag{111}$$

2. **Căn tâm và tính mốc thời gian khung đầu tiên t_1:** Quy ước tọa độ thời gian của Praat (`Sound.cpp`) đặt mẫu đầu tiên tại $x_1 = 0{,}5 \cdot dx$ (tâm của khoảng lấy mẫu đầu tiên). Điểm chính giữa của tệp âm thanh là:

$$\text{our_mid} = x_1 - 0{,}5 \cdot dx + 0{,}5 \cdot \text{my_duration} = 0{,}5 \cdot \text{my_duration} \tag{112}$$

   Khoảng thời gian mà n_{frames} khung chiếm dụng là thy_duration $= n_{\text{frames}} \cdot \Delta t$. Để chuỗi khung nằm cân đối ở giữa tệp, mốc thời gian của tâm khung đầu tiên t_1 bắt buộc phải là:

$$t_1 = \text{our_mid} - 0{,}5 \cdot \text{thy_duration} + 0{,}5 \cdot \Delta t \tag{113}$$

3. **Tính mốc thời gian t_i và chỉ số mẫu bắt đầu cửa sổ:** Tâm thời gian của khung thứ i ($i = 1, 2, \ldots, n_{\text{frames}}$) là:

$$t_i = t_1 + (i - 1) \cdot \Delta t \tag{114}$$

   Để chuyển từ mốc thời gian liên tục t_i sang chỉ số mẫu rời rạc, Praat gọi hàm `Sampled_xToLowIndex` (`Sampled.cpp` dòng 35):

$$\text{leftSample}_i = \left\lfloor \frac{t_i - x_1}{dx} + 1{,}0 \right\rfloor \tag{115}$$

   Mẫu bên phải là $\text{rightSample}_i = \text{leftSample}_i + 1$. Khung cửa sổ Hanning 638 mẫu có nửa chiều dài là halfnsamp_window $= 319$ mẫu. Do đó, chỉ số mẫu bắt đầu của cửa số (tính từ 0, tức 0-based) đưa vào bộ đệm FPGA là:

$$\text{win0}_i = \text{rightSample}_i - 319 - 1 = \text{leftSample}_i - 319 \tag{116}$$

   Ví dụ tính tay trên hai tệp âm thanh tiêu biểu:

- **Tệp có thời lượng 1,234 s ($n_x = 19.744$ mẫu):** Thời lượng: my_duration $= 19.744 \times 0{,}0000625 = 1{,}234$ s. Số khung: $n_{\text{frames}} = \lfloor (1{,}234 - 0{,}04)/0{,}01 \rfloor + 1 = \lfloor 119{,}4 \rfloor + 1 = 120$ khung. Tâm tệp: our_mid $= 0{,}5 \times 1{,}234 = 0{,}617$ s. Thời lượng phủ: thy_duration $= 120 \times 0{,}01 = 1{,}200$ s. Mốc thời gian khung đầu: $t_1 = 0{,}617 - 0{,}5 \times 1{,}200 + 0{,}005 = 0{,}022000$ s. Tại khung 1: $t_1 = 0{,}022$ s. Vì $x_1 = 0{,}5 \times 0{,}0000625 = 0{,}00003125$ s, ta có:

$$\text{leftSample}_1 = \left\lfloor \frac{0{,}022 - 0{,}00003125}{0{,}0000625} + 1{,}0 \right\rfloor = \lfloor 351{,}5 + 1{,}0 \rfloor = \lfloor 352{,}5 \rfloor = 352 \tag{117}$$

  Chỉ số mẫu bắt đầu cửa sổ đầu tiên: $\text{win0}_0 = 352 - 319 =$ **33**. Khung đầu tiên trên FPGA không bắt đầu ở mẫu 0 mà bắt đầu từ mẫu 33 để khớp hoàn hảo với Praat! Khung cuối cùng ($i = 120$) bắt đầu tại mẫu $\text{win0}_{119} = 19.073$.

- **Tệp chuẩn 2,8 s ($n_x = 44.800$ mẫu):** Thời lượng: my_duration $= 44.800 \times 0{,}0000625 = 2{,}800$ s. Số khung: $n_{\text{frames}} = \lfloor (2{,}800 - 0{,}04)/0{,}01 \rfloor + 1 = 276 + 1 = 277$ khung. Tâm tệp: our_mid $= 1{,}400$ s; thy_duration $= 277 \times 0{,}01 = 2{,}770$ s. Mốc thời gian khung đầu: $t_1 = 1{,}400 - 0{,}5 \times 2{,}770 + 0{,}005 =$ **0,020000** s. Tại khung 1: $t_1 = 0{,}020$ s $\implies$ leftSample$_1 = \lfloor 0{,}020/0{,}0000625 + 0{,}5 \rfloor = \lfloor 320{,}5 \rfloor = 320$. Chỉ số mẫu bắt đầu cửa sổ đầu tiên: win0$_0 = 320 - 319 =$ **1**. Toàn bộ 277 khung đều có bước nhảy hop $=$ win0$_i -$ win0$_{i-1} = 160$ mẫu đều đặn tuyệt đối!

Đón trước chỗ vướng: Bẫy sàn số lẻ (Odd-sample floor trap)

► HỎI – ĐÁP TƯ DUY: Vì sao bước nhảy khung trong Praat đôi khi bị co thành 159 hoặc giãn thành 161 mẫu?

Hãy quan sát biểu thức tính toán trong hàm `Sampled_xToLowIndex`:

$$y_i = \frac{t_i - x_1}{dx} + 1{,}0 = \frac{t_i}{dx} + 0{,}5 \tag{118}$$

Thay biểu thức của t_i từ các bước trên vào, ta biến đổi đại số chính xác:

$$\begin{aligned} y_i &= \frac{0{,}5 \cdot n_x \cdot dx - 0{,}5 \cdot n_{\text{frames}} \cdot \Delta t + 0{,}5 \cdot \Delta t + (i-1) \cdot \Delta t}{dx} + 0{,}5 \\ &= 0{,}5 \cdot n_x - \frac{0{,}5 \cdot n_{\text{frames}} \cdot \Delta t}{dx} + \frac{(i - 0{,}5) \cdot \Delta t}{dx} + 0{,}5 \end{aligned} \tag{119}$$

Vì tỉ số $\Delta t/dx = 0{,}01/0{,}0000625 = 160$ là một **số nguyên chẵn tuyệt đối**, nên biểu thức rút gọn thành:

$$y_i = 0{,}5 \cdot n_x - 80 \cdot n_{\text{frames}} + 160 \cdot i - 80 + 0{,}5 \tag{120}$$

Bản chất số học lộ diện ở đây:

- **Khi n_x là số chẵn ($n_x = 2k$):** Đại lượng $0{,}5 \cdot n_x = k$ là một số nguyên. Khi đó $y_i =$ nguyên $+ 0{,}5$. Giá trị luôn rơi vào chính giữa hai số nguyên liên tiếp (phần thập phân là đúng $0{,}5$). Khoảng cách tới biên số nguyên là $0{,}5$, rất lớn so với sai số làm tròn 10^{-16} của số thực double. Do đó hàm lấy sàn $\lfloor y_i \rfloor$ hoàn toàn miễn nhiễm với sai số dấu phẩy động; bước nhảy giữa hai khung liên tiếp luôn bằng đúng:

$$\text{hop} = \lfloor y_{i+1} \rfloor - \lfloor y_i \rfloor = (M + 160) - M = 160 \text{ mẫu (ổn định tuyệt đối).} \tag{121}$$

- **Khi n_x là số lẻ ($n_x = 2k + 1$):** Đại lượng $0{,}5 \cdot n_x = k + 0{,}5$. Khi cộng với $0{,}5$ ở cuối công thức, ta thu được:

$$y_i = (k + 0{,}5) - 80 \cdot n_{\text{frames}} + 160 \cdot i - 80 + 0{,}5 = \text{SỐ NGUYÊN CHÍNH XÁC!} \tag{122}$$

  Đây chính là cái bẫy chết người: y_i về mặt lý thuyết là một số nguyên hoàn hảo (ví dụ $32.173{,}0$). Nhưng trong chuẩn số thực dấu phẩy động IEEE 754, giá trị thời gian $\Delta t = 0{,}01$ s là một phân số vô hạn tuần hoàn trong hệ nhị phân ($0{,}01_{10} \approx 0{,}000000101000111101\ldots_2$). Khi nhân và cộng dồn qua hàng trăm khung, sai số làm tròn tích lũy ở bit thấp nhất (ULP, Unit in the Last Place) có thể làm giá trị thực tế bị hụt một lượng vô cùng nhỏ:

$$y_i^{\text{thực tế}} = 32.172{,}999999999996362 \tag{123}$$

  Hàm lấy sàn $\lfloor \cdot \rfloor$ lập tức đánh rơi giá trị xuống 32.172 thay vì 32.173! Kết quả là bước nhảy khung bị co lại thành 159 mẫu, và khung ngay sau đó bị bù giãn thành 161 mẫu!

Thực nghiệm kiểm chứng trên 200 tệp âm thanh ngẫu nhiên trong kịch bản `ch01_frames_verify.py` đã chứng thực định lý này:

- Với 100 tệp có độ dài chẵn (n_x chẵn): tỷ lệ xuất hiện bước nhảy lệch 160 là **đúng 0,0%** (100/100 tệp đều có hop 160 hoàn hảo).
- Với 100 tệp có độ dài lẻ (n_x lẻ): có tới **86,0%** số tệp xuất hiện hiện tượng bước nhảy bị giật 159 hoặc 161 mẫu do bẫy sàn số lẻ của Praat!

Để giải quyết triệt để vấn đề này, chương trình C trên HPS (`sw_v3/ch01_frames.c`) tính toán chính xác bằng kiểu `double` theo đúng công thức của Praat nhằm bám sát vị trí khung tham chiếu đến từng mẫu, đồng thời tích hợp cờ `hop_regular`. Nếu phát hiện bước nhảy không đều do số mẫu lẻ, trình điều khiển sẽ phát cảnh báo rõ ràng để người dùng nhận biết nguồn gốc sai lệch tiềm tàng.

Tóm lại: Đọc WAV đúng cách là duyệt theo từng chunk RIFF để tránh bẫy offset 44 và xử lý byte đệm số lẻ; tính đỉnh GP bằng công thức số nguyên quét cực trị một lượt tránh số thực; và lưới khung Praat phải được tính bằng double để nhận diện bẫy sàn số lẻ khi số mẫu âm thanh không chia hết cho hai.

1.3. Các phương án truyền dữ liệu, đánh đổi và lựa chọn

Vấn đề thực tế: Ảo tưởng về sự cần thiết của DMA trong xử lý âm thanh

Khi bắt tay vào thiết kế một hệ thống tích hợp HPS–FPGA, một phản xạ tự nhiên của nhiều kỹ sư phần cứng là lập tức tìm đến các bộ điều khiển truy xuất bộ nhớ trực tiếp DMA (Direct Memory Access, tiêu biểu như IP mSGDMA của Intel). Lập luận thường nghe là: "phải có DMA để truyền dữ liệu tự động ở tốc độ phần cứng, nếu dùng CPU ghi từng từ thì sẽ làm nghẽn vi xử lý và không thể đáp ứng thời gian thực".

Tuy nhiên, trong kỹ nghệ hệ thống, mọi quyết định kiến trúc bắt buộc phải xuất phát từ phép tính định lượng trên nguyên lý đầu tiên: **Lưu lượng thực sự cần vận chuyển là bao nhiêu?**

Hãy làm phép tính số học cơ bản cho tín hiệu âm thanh chuẩn 16 kHz:

- Tần số lấy mẫu: $f_s = 16.000$ mẫu/giây.
- Mỗi mẫu âm thanh PCM 16-bit chiếm đúng 2 byte bộ nhớ.
- Băng thông dữ liệu thực tế sinh ra trong một giây âm thanh là:

$$B_{\text{audio}} = 16.000 \times 2 = 32.000 \text{ byte/giây} = 32 \text{ KB/s} = 0{,}032 \text{ MB/s} \tag{124}$$

Bây giờ, hãy đặt con số 32 KB/s này lên bàn cân với năng lực truyền tải của các cầu nối phần cứng trên SoC Cyclone V:

- Bus bộ nhớ DDR3 của HPS hoạt động ở xung nhịp 400 MHz (tốc độ dữ liệu kép DDR 800 MT/s), bề rộng 32-bit, cho băng thông đỉnh lý thuyết lên tới:

$$B_{\text{DDR3}} = 800 \text{ MT/s} \times 4 \text{ byte} = 3.200 \text{ MB/s} = 3{,}2 \text{ GB/s} \tag{125}$$

- Cầu nối Lightweight HPS-to-FPGA (LWHPS2FPGA) có bề rộng cố định 32-bit. Ở xung nhịp đồng bộ của hệ thống FPGA là 50 MHz, trần băng thông lý thuyết của bus là:

$$B_{\text{LWHPS}} = 50 \text{ MHz} \times 4 \text{ byte} = 200 \text{ MB/s} \tag{126}$$

- Cầu nối tốc độ cao Full HPS-to-FPGA với bề rộng 128-bit ở xung nhịp 100 MHz có thể đạt tới 1.600 MB/s $=$ $1{,}6$ GB/s.

  So sánh định lượng trực tiếp:

$$\frac{B_{\text{audio}}}{B_{\text{LWHPS}}} = \frac{0{,}032 \text{ MB/s}}{200 \text{ MB/s}} = 0{,}00016 = \mathbf{0{,}016}\%! \tag{127}$$

Lưu lượng dữ liệu của dòng âm thanh 16 kHz chỉ chiếm chưa đầy **một phần sáu nghìn** $(1/6.250)$ băng thông khả dụng của cầu nối nhẹ nhất trên chip! Việc vội vã đưa vào một bộ điều khiển DMA phức tạp cho một dòng dữ liệu 32 KB/s là biểu hiện kinh điển của việc tối ưu hóa sớm mà không dựa trên con số.

Ba phương án truyền dữ liệu và sự đánh đổi kỹ thuật

Để đưa ra lựa chọn sáng suốt nhất cho anh Tú và người phát triển hệ thống, ta tiến hành phân tích chi tiết ba phương án truyền nhận khả thi:

Phương án 1: PIO qua cầu Lightweight HPS-to-FPGA (32-bit). Vi xử lý ARM thực hiện các lệnh ghi trực tiếp (Programmed I/O) từ không gian người dùng thông qua con trỏ ánh xạ bộ nhớ `mmap` trên tệp thiết bị `/dev/mem`.

- **Cơ chế đóng gói:** Vì mỗi giao dịch bus của cầu Lightweight chuyển được đúng một từ 32-bit, ta đóng gói hai mẫu âm thanh 16-bit liên tiếp vào một từ duy nhất:

$$\text{word}_{32} = ((s[2k+1] \;\&\; \text{0xFFFF}) \ll 16) \mid (s[2k] \;\&\; \text{0xFFFF}) \tag{128}$$

  với mẫu chẵn $s[2k]$ đặt ở 16 bit thấp và mẫu lẻ $s[2k+1]$ đặt ở 16 bit cao theo chuẩn little-endian.

- **Tần suất thao tác ghi:** Để đẩy 1 giây âm thanh (16.000 mẫu), CPU chỉ cần thực hiện đúng:

$$N_{\text{write}} = \frac{16.000}{2} = 8.000 \text{ lệnh ghi/giây} \tag{129}$$

- **Thời gian chiếm dụng CPU và hệ số thời gian thực (RTF):** Hãy đưa ra một giả định cực kỳ bi quan rằng mỗi thao tác ghi qua bus AXI/Avalon-MM tốn tới 1 μs (1.000 ns, tương đương 50 chu kỳ xung nhịp 50 MHz của FPGA hoặc gần 800 chu kỳ xung nhịp CPU ARM Cortex-A9):

$$T_{\text{transfer}} = 8.000 \times 1\ \mu\text{s} = 8 \text{ ms cho mỗi 1 giây âm thanh} \tag{130}$$

  Quy đổi ra hệ số thời gian thực của khâu truyền tải:

$$\text{RTF}_{\text{transfer}} = \frac{T_{\text{transfer}}}{1 \text{ s}} = \frac{8 \text{ ms}}{1.000 \text{ ms}} = \mathbf{0{,}008} \tag{131}$$

  Kể cả trong kịch bản bi quan nhất, vi xử lý chỉ mất đúng 0,8% năng lực thời gian để vận chuyển toàn bộ dòng âm thanh sang FPGA! Thời gian ghi thực tế trên bo mạch DE10-Standard đo được thông qua hàm `clock_gettime(CLOCK_MONOTONIC)` thường chỉ dao động từ vài chục đến khoảng hơn một trăm nano-giây mỗi giao dịch (con số cụ thể là **phải đo** trên bo mạch thực tế).

- **Độ phức tạp phần mềm:** Cực kỳ thấp. Toàn bộ logic chạy hoàn toàn trong không gian người dùng (User Space), không cần viết driver nhân Linux (Kernel Module), không cần quyền can thiệp vào bộ nhớ vật lý của hệ điều hành.

Phương án 2: DMA phân tán modular (mSGDMA) qua cầu Full Bridge. Khối IP mSGDMA (modular Scatter-Gather DMA) nằm trên vùng logic FPGA đóng vai trò master, tự động đọc luồng mẫu từ bộ nhớ DDR của HPS rồi bắn trực tiếp vào giao diện Avalon-ST của đường ống xử lý mà không cần CPU tham gia từng mẫu.

- **Ưu điểm:** Hoàn toàn giải phóng CPU khỏi vòng lặp ghi dữ liệu. CPU chỉ cần kích hoạt descriptor rồi chuyển sang thực hiện các tác vụ khác.

- **Cái giá phần mềm:** Cực kỳ đắt đỏ. Hệ điều hành Linux quản lý bộ nhớ theo các trang ảo (virtual pages) kích thước 4 KB nằm rải rác trong RAM vật lý. Để DMA có thể đọc được:

  1. Cần phân bổ một vùng nhớ vật lý liên tục (Contiguous Memory) bằng cơ chế CMA của Linux kernel hoặc thông qua module hỗ trợ `/dev/udmabuf`.
  2. **Hiểm họa tính nhất quán bộ đệm (Cache Incoherency):** Khi CPU đọc tệp WAV từ thẻ nhớ SD và ghi vào mảng bộ nhớ, các byte dữ liệu có thể còn nằm lưu trong bộ nhớ đệm cache L1/L2 của lõi ARM Cortex-A9 mà chưa được đẩy (flush) xuống các chip nhớ DDR3 bên dưới. Bộ điều khiển DMA trên FPGA khi đọc trực tiếp từ DDR3 sẽ đọc phải dữ liệu rác cũ! Người lập trình bắt buộc phải gọi các hàm xả cache tường minh (`dma_sync_single_for_device` hoặc `arm_dma_flush`), nếu quên hoặc gọi sai thời điểm, toàn bộ dữ liệu âm thanh bị sai mà không hề có bất kỳ cảnh báo lỗi nào.
  3. **Quản lý mô tả phân tán (Descriptors):** Phải tạo và cấu hình các bảng descriptor phức tạp trong bộ nhớ, xử lý ngắt hoàn thành (interrupt handling) để dọn dẹp bộ đệm.

- **Cái giá phần cứng:** Khối IP mSGDMA tiêu tốn hàng trăm khối logic ALM và ít nhất 2 đến 3 khối RAM M10K chuyên dụng để làm bộ đệm descriptor và FIFO dữ liệu nội tại.

Phương án 3: Cầu Full HPS-to-FPGA (64/128-bit) với cơ chế PIO. Sử dụng cầu nối chính HPS-to-FPGA với bề rộng 64-bit hoặc 128-bit kết hợp với lệnh ghi SIMD (NEON) của ARM để đẩy 4 hoặc 8 mẫu âm thanh trong một chu kỳ truy xuất.

- **Đánh giá:** Băng thông tăng thêm 2 đến 4 lần so với cầu Lightweight, nhưng đối với lưu lượng chỉ 32 KB/s, việc giảm số lần ghi từ 8.000 xuống 2.000 hay 4.000 lần/giây không đem lại bất kỳ cải thiện nhận thấy nào về hiệu năng tổng thể. Trong khi đó, cầu Full Bridge chiếm dụng không gian địa chỉ vật lý rất rộng lớn, đòi hỏi cấu hình mạng chuyển mạch bus AXI phức tạp hơn và việc phân bổ xung nhịp giữa hai miền cần thêm các tầng đồng bộ chuyên dụng.

Bảng so sánh tổng hợp các phương án truyền tải

Bảng 5 tổng hợp toàn diện các chỉ số kỹ thuật và sự đánh đổi giữa ba phương án:

Bảng 5: So sánh định lượng các phương án truyền dữ liệu HPS–FPGA cho tín hiệu 16 kHz.

Chỉ số kỹ thuật	**PA 1: PIO (LWHPS)**	**PA 2: mSGDMA**	**PA 3: PIO (Full)**
Cầu nối vật lý	LWHPS-to-FPGA	Full HPS-to-FPGA	Full HPS-to-FPGA
Bề rộng dữ liệu bus	32 bit	32 hoặc 64 bit	64 hoặc 128 bit
Lưu lượng yêu cầu	32 KB/s (8.000 từ/s)	32 KB/s (8.000 từ/s)	32 KB/s (4.000 từ/s)
Tải băng thông bus	0,016% của 200 MB/s	$< 0{,}005\%$ của 1,6 GB/s	$< 0{,}003\%$ của 1,6 GB/s
Thời gian CPU (RTF)	$\leq 0{,}008$ (bi quan 1 μs)	$\approx 0{,}000$ (tự động)	$\leq 0{,}004$ (bi quan)
Thời gian ghi thực tế	**(phải đo)**	Không áp dụng	**(phải đo)**
Yêu cầu bộ nhớ vật lý	Không (ảo bình thường)	Bắt buộc liên tục (CMA)	Không (ảo bình thường)
Hiểm họa Cache coherency	Không có	Bắt buộc xả cache	Không có
Không gian phần mềm	User Space (`mmap`)	Kernel Module / UIO	User Space (`mmap`)
Tài nguyên phần cứng FPGA	0 ALM, 0 M10K thêm	≈ 450 ALM, ≥ 2 M10K	Thêm logic ghép bus
Độ phức tạp kiểm chứng	Rất thấp, xem từng nhịp	Cao, phụ thuộc ngắt	Trung bình

Đọc bảng. Nhìn vào Bảng 5, ta thấy một sự thật hiển nhiên: ở lưu lượng siêu thấp 32 KB/s, việc dùng mSGDMA hoàn toàn không giải quyết được nút thắt hiệu năng nào (bởi vì bản thân khâu truyền tải đã có RTF $\leq 0{,}008$, tức nhanh hơn thời gian thực ít nhất 125 lần), nhưng lại kéo theo cái giá quá đắt về việc phải can thiệp vào tầng nhân Linux kernel, cấp phát bộ nhớ liên tục và hiểm họa đọc dữ liệu rác do lỗi cache coherency.

Lựa chọn của ta và lý do có số liệu

Từ các phân tích định lượng trên, đề cương quyết định lựa chọn:

1. **Phương án 1 (PIO qua cầu Lightweight Bridge vào bộ đệm On-Chip FIFO) là đường truyền chính thống của toàn bộ tài liệu và dự án**. Có bốn lý do đanh thép:
   - *Thông lượng dư thừa tuyệt đối:* Hệ số thời gian thực $\text{RTF}_{\text{transfer}} \leq 0{,}008$. CPU chỉ tốn tối đa 8 ms để truyền xong 1 giây âm thanh, 99,2% thời gian còn lại hoàn toàn rảnh rỗi.
   - *Độ tin cậy phần mềm cao nhất:* Trình điều khiển chạy hoàn toàn ở không gian người dùng (User Space), chỉ cần mở tệp `/dev/mem` rồi gọi hàm chuẩn `mmap`. Không có bất kỳ nguy cơ treo nhân hệ điều hành, rò rỉ bộ nhớ hay lỗi lệch cache.
   - *Kiến trúc phần cứng tinh gọn:* Toàn bộ giao diện dữ liệu và thanh ghi điều khiển được gom gọn vào không gian địa chỉ 2 MB của Lightweight Bridge, giải phóng toàn bộ các cổng Full Bridge cho các ứng dụng mở rộng khác trong tương lai.
   - *Dễ dàng gỡ lỗi và kiểm chứng bit-exact:* Người thiết kế có thể đặt các điểm dừng (breakpoint), đọc thanh ghi báo mức đầy `fill_level` của FIFO ở từng chu kỳ xung nhịp và quan sát chính xác từng từ dữ liệu được nạp vào.
2. **Phương án 2 (mSGDMA) được giữ lại làm mục đối chiếu chuyên sâu**: Chương 13 sẽ trình bày cách cấu hình mSGDMA và tiến hành đo đạc thực nghiệm trên phần cứng thật để so sánh chính xác thời gian chiếm dụng CPU và tài nguyên ALM/M10K với phương án PIO.

Tóm lại: Với lưu lượng âm thanh chỉ 32 KB/s (8.000 lần ghi 32-bit mỗi giây), cơ chế PIO qua cầu Lightweight bridge chỉ tiêu tốn RTF $\leq 0{,}008$ mà loại bỏ hoàn toàn gánh nặng bộ đệm vật lý và xả cache của DMA, là lựa chọn tối ưu về cả hiệu năng lẫn độ tin cậy.

1.4. Đường (A): Xây dựng bộ đệm bằng Intel On-Chip FIFO IP

Nguyên lý của On-Chip FIFO trong Platform Designer

Trong luồng thiết kế tiêu chuẩn của hệ sinh thái Intel Quartus Prime, công cụ tích hợp hệ thống Platform Designer (tên cũ là Qsys) cung cấp sẵn khối IP lõi mang tên `altera_avalon_fifo` (On-Chip FIFO Memory). Khối IP này đóng vai trò cầu nối thích ứng giữa hai miền giao thức hoàn toàn khác biệt:

- **Miền giao diện bộ nhớ (Avalon-MM):** Do vi xử lý ARM HPS làm chủ (Master). CPU thực hiện các thao tác ghi dữ liệu theo từng khối rời rạc, phụ thuộc vào tiến trình lập lịch của hệ điều hành Linux.
- **Miền luồng dữ liệu (Avalon-ST):** Cung cấp dữ liệu trực tiếp cho lõi xử lý F0 trên FPGA (Sink). Lõi phần cứng yêu cầu dữ liệu mẫu âm thanh chảy liên tục, tuần tự và phải có cơ chế báo bận (backpressure) tức thời.

Để cấu hình IP này hoạt động chính xác và khai thác tối ưu tài nguyên phần cứng của chip Cyclone V 5CSXFC6D6F31C6N, người kỹ sư không được phép nhấp chuột tùy tiện mà phải hiểu rõ ý nghĩa vật lý của từng tham số cấu hình.

Giải thích chi tiết từng tham số cấu hình của Intel On-Chip FIFO

Dưới đây là phân tích cặn kẽ từng tham số trên giao diện đồ họa Platform Designer:

1. Độ sâu bộ đệm (FIFO Depth = 512 từ). Khối bộ nhớ nhúng cơ bản trên dòng FPGA Cyclone V là khối M10K, với dung lượng lưu trữ vật lý danh định là 10.240 bit (bao gồm cả các bit kiểm tra chẵn lẻ parity). Khối M10K hỗ trợ các cấu hình tỷ lệ linh hoạt như 256×40, 512×20, 1024×10, v.v.

Khi cấu hình FIFO lưu trữ các mẫu âm thanh 16-bit, dung lượng logic cần thiết cho độ sâu 512 từ là:

$$N_{\text{bits}} = 512 \times 16 \text{ bit} = 8.192 \text{ bit} \tag{132}$$

Vì $8.192 \leq 10.240$, toàn bộ bộ đệm 512 từ này được ánh xạ vừa vặn vào **đúng 1 khối M10K duy nhất** (ở chế độ phần cứng 512×20 bit, bỏ qua 4 bit dư thừa).

Ngược lại, nếu thiết lập độ sâu là 1024 từ (như tài liệu cũ của Argon đề xuất):

$$N_{\text{bits, 1024}} = 1024 \times 16 \text{ bit} = 16.384 \text{ bit} > 10.240 \text{ bit} \tag{133}$$

Trình biên dịch Quartus Fitter bắt buộc phải cấp phát tới 2 khối M10K vật lý! Đây là minh chứng rõ ràng cho việc tính toán sai cấu hình phần cứng nhúng.

Về mặt đệm thời gian, với tần số lấy mẫu 16 kHz và bước nhảy khung tiêu chuẩn của Praat là 10 ms ($H = 160$ mẫu), dung lượng 512 mẫu chứa được:

$$N_{\text{hops}} = \frac{512}{160} = 3{,}2 \text{ khung nhảy} \tag{134}$$

tương đương với hơn 32 ms âm thanh. Khoảng đệm 32 ms này là quá đủ để hấp thu các đợt trễ lập lịch (scheduling latency) thông thường của nhân hệ điều hành Linux (thường dưới 5–10 ms), đảm bảo lõi FPGA không bao giờ bị đói dữ liệu (underrun).

2. Chế độ xung nhịp (Clock Setting = Single Clock Mode). Cả bus giao tiếp Lightweight HPS-to-FPGA lẫn lõi tính toán F0 đều được đấu nối vào cùng một nguồn dao động xung nhịp hệ thống 50 MHz (`clk`). Do đó, ta chọn chế độ *Single Clock Mode*. Lựa chọn này giúp IP loại bỏ hoàn toàn các mạch đồng bộ hóa hai miền xung nhịp (như chuỗi đồng bộ kép flip-flop kép 2-FF synchronizer và bộ chuyển đổi mã Gray cho các con trỏ), qua đó tiết kiệm đáng kể tài nguyên logic ALM và giảm tối đa độ trễ truyền dữ liệu qua FIFO.

3. Cổng vào và cổng ra (Input / Output Interfaces).

- **Cổng vào (Input):** Thiết lập là `AVALONMM_WRITE`. Bề rộng dữ liệu bus Avalon-MM được đặt là 32-bit. Thiết lập này cho phép CPU ARM ghi một cặp hai mẫu âm thanh 16-bit (sample 0 và sample 1) trong một chu kỳ bus duy nhất, giảm 50% số lượng giao dịch bus qua cầu nối.
- **Cổng ra (Output):** Thiết lập là `AVALONST_SOURCE`. Bề rộng dữ liệu ngõ ra là 16-bit, tương thích trực tiếp với đầu vào của lõi xử lý DSP trên FPGA.

4. Kích thước ký hiệu và số ký hiệu mỗi nhịp (Bits per symbol và Symbols per beat). Hai tham số này đóng vai trò quyết định cơ chế tách từ (unpacking) 32-bit thành hai mẫu 16-bit:

- **Bits per symbol:** Đặt là 16. Mỗi ký hiệu âm thanh là một mẫu nguyên có dấu 16-bit.
- **Symbols per beat:** Ở cổng vào Avalon-MM 32-bit, số ký hiệu trên mỗi nhịp (beat) là 2. Ở cổng ra Avalon-ST 16-bit, số ký hiệu trên mỗi nhịp tự động quy về 1.

Nhờ thiết lập này, khối On-Chip FIFO tự động quản lý logic dồn dịch dữ liệu theo thứ tự endian chuẩn: nửa từ thấp `data[15:0]` là mẫu xuất hiện trước (symbol 0), nửa từ cao `data[31:16]` là mẫu xuất hiện sau (symbol 1).

5. Cho phép áp lực ngược (Allow Backpressure = YES). Tham số này bắt buộc phải bật. Khi bật, cổng ra Avalon-ST sẽ sinh ra tín hiệu điều khiển `ready`. Nếu khối tính toán cửa sổ hoặc khối tự tương quan phía sau đang bận tính toán và chưa sẵn sàng nhận mẫu mới, nó sẽ kéo `ready` xuống mức 0. FIFO sẽ lập tức giữ nguyên dữ liệu trên bus `data` và không tăng con trỏ đọc, bảo đảm dòng dữ liệu không bị thất thoát mẫu.

6. Tạo giao diện trạng thái cho cổng vào (Create status interface for input = YES). Khi tùy chọn này được kích hoạt, Platform Designer sẽ sinh thêm một cổng phụ mang tên `in_csr` (Avalon-MM Control and Status Register). Giao diện này cực kỳ quan trọng: nó cho phép vi xử lý HPS đọc mức đầy hiện tại (`fill_level`) của bộ đệm trước khi thực hiện ghi dữ liệu. Nếu không có cổng `in_csr`, phần mềm trên ARM sẽ hoàn toàn "mù" về trạng thái FIFO và buộc phải dựa vào tín hiệu `waitrequest` của bus phần cứng, dễ dẫn tới việc nghẽn luồng thực thi của CPU.

Các giao diện phần cứng và thanh ghi do IP sinh ra

Khi được khởi tạo thành công trong hệ thống, khối IP On-Chip FIFO xuất ra ba giao diện chính:

1. **Giao diện dữ liệu nạp `in` (Avalon-MM Slave):** Ánh xạ vào không gian bộ nhớ của cầu Lightweight. Gồm tín hiệu địa chỉ, tín hiệu cho phép ghi `write`, dữ liệu ghi 32-bit `writedata[31:0]` và tín hiệu phản hồi chờ `waitrequest`.
2. **Giao diện điều khiển và trạng thái `in_csr` (Avalon-MM Slave):** Cung cấp tập thanh ghi 32-bit chuẩn của Intel để HPS theo dõi hoạt động:
   - *Thanh ghi `fill_level` (Offset 0x00):* Trả về số lượng từ 32-bit hiện đang nằm trong bộ đệm.
   - *Thanh ghi `status` (Offset 0x04):* Chứa các cờ trạng thái tức thời, bao gồm bit 0 (`full`) và bit 1 (`empty`).
   - *Thanh ghi `event` (Offset 0x08):* Chứa các cờ sự kiện dính (sticky flags) như tràn bộ đệm (`overflow`) hoặc cạn bộ đệm (`underflow`). Cờ này chỉ được xóa khi phần mềm ghi số 1 vào bit tương ứng (Write-1-to-Clear).
   - *Thanh ghi `almostfull` (Offset 0x10) và `almostempty` (Offset 0x14):* Cấu hình ngưỡng cảnh báo sớm mức đầy.
3. **Giao diện luồng ngõ ra `out` (Avalon-ST Source):** Bao gồm dữ liệu âm thanh 16-bit `data[15:0]`, tín hiệu hợp lệ `valid` và tín hiệu sẵn sàng `ready`.

Độ trễ truyền nhận (Latency). Trong kiến trúc FIFO của Intel, từ thời điểm CPU ghi thành công một từ vào cổng `in` cho tới khi dữ liệu đó xuất hiện trên cổng `out` và giương cờ `valid = 1`, hệ thống tiêu tốn một độ trễ cố định từ 1 đến 2 chu kỳ xung nhịp (phụ thuộc vào việc cấu hình thanh ghi đệm ngõ ra). Giá trị độ trễ chính xác này **(phải kiểm trong mô phỏng IP)** trước khi tích hợp vào đường ống xử lý.

Cấu hình mSGDMA dùng làm mục đối chiếu chuyên sâu

Để phục vụ cho các thực nghiệm so sánh định lượng ở Chương 13, khối IP Modular Scatter-Gather DMA (mSGDMA) của Intel được cấu hình với các tham số tối ưu hóa cho bài toán dòng như sau:

- **DMA Mode:** Chọn `Memory-Mapped to Streaming` (MM-to-ST). Bộ điều khiển sẽ tự động đọc dữ liệu từ vùng nhớ đệm DDR3 của HPS và đẩy ra cổng luồng Avalon-ST cấp cho FPGA.

- **Data Width:** Đặt là 32-bit để tận dụng trần truyền tải dữ liệu của bus HPS.
- **Data Path FIFO Depth:** Cấu hình độ sâu đệm đường truyền là 64 hoặc 128 từ nhằm hấp thu biến động độ trễ truy xuất bus bộ nhớ DDR.
- **Descriptor FIFO Depth:** Đặt là 16 hoặc 32 bộ mô tả (descriptors). Với các tệp âm thanh ngắn vài giây, chỉ cần 1 descriptor duy nhất mô tả toàn bộ khối dữ liệu là đủ.
- **Maximum Transfer Length:** Thiết lập 2^{32} byte (hoặc tối thiểu $2^{16} = 64$ KB), cho phép truyền trọn vẹn tệp âm thanh trong một lệnh gọi duy nhất.
- **Burst Enable:** Bật tính năng truyền liên tục (Burst mode) với `Maximum Burst Count = 16` hoặc 32 nhịp, giúp tăng tối đa hiệu suất đọc từ bộ điều khiển DDR3 SDRAM.
- **Response Port:** Tắt (Disabled) vì phía phần mềm chỉ cần nhận ngắt kết thúc truyền tải từ bộ mô tả chứ không cần cổng trả về dữ liệu phụ.

Bộ đệm chiều về: Thu nhận kết quả F0 từ FPGA về HPS

Không chỉ có chiều đi nạp mẫu âm thanh, hệ thống còn cần kênh truyền chiều về (Return Path) để HPS đọc chuỗi tần số cơ bản F0 do lõi tính toán trên FPGA sinh ra.

Ở chiều về, kiến trúc đảo ngược hoàn toàn:

- Khối hậu xử lý trên FPGA (thuật toán Viterbi / Median-3) tính toán xong mỗi khung sẽ đẩy ra một từ F0 định dạng số nguyên bù 2 định dạng fixed-point Q12.4 (16-bit).
- Dữ liệu này được đưa vào một bộ đệm On-Chip FIFO cấu hình `Input: AVALONST_SINK` (16-bit) và `Output: AVALONMM_READ` (32-bit hoặc 16-bit), đặt tên là `fifo_out`.
- Vi xử lý HPS chỉ cần thăm dò thanh ghi trạng thái của `fifo_out_csr`, khi có kết quả hợp lệ sẽ đọc trực tiếp dữ liệu qua cầu Lightweight Bridge để lưu vào tệp kết quả `pitch_f0.bin`.

Tóm lại: Khối IP On-Chip FIFO của Intel ở độ sâu 512 từ khớp hoàn hảo với đúng 1 khối nhớ M10K vật lý của Cyclone V, cung cấp 32 ms đệm chống giật tiến trình Linux và tách mẫu tự động 32-to-16 bit; chiều về dùng FIFO ST-sink sang MM-read để trả kết quả F0 định dạng Q12.4 một cách đồng bộ và an toàn.

1.5. Đường (B): Hiện thực phần cứng bằng HDL tự viết

Nguyên lý bộ đệm vòng đồng bộ với con trỏ có bit vòng (Wrap-Bit Pointers)

Để không bị phụ thuộc vào các IP độc quyền (proprietary IP) của hãng và có thể kiểm soát tuyệt đối từng chu kỳ xung nhịp trong quá trình mô phỏng, ta tự hiện thực khối bộ đệm `ch01_avalon_fifo` bằng ngôn ngữ Verilog thuần túy.

Bộ đệm vòng (circular FIFO) đồng bộ hoạt động trên nguyên lý của hai con trỏ trượt tuần tự: con trỏ ghi (`wr_ptr`) và con trỏ đọc (`rd_ptr`). Khi bộ nhớ có độ sâu $N_{\text{words}} = 2^k$ từ (với $k = 8$, tức 256 từ 32-bit), một thách thức kinh điển là: **Làm thế nào để phân biệt trạng thái FIFO rỗng hoàn toàn với trạng thái FIFO đầy hoàn toàn khi cả hai con trỏ đều chỉ vào cùng một địa chỉ vật lý?**

Giải pháp thanh lịch và tối ưu phần cứng nhất là mở rộng bề rộng của con trỏ thêm đúng 1 bit ở vị trí trọng số cao nhất (MSB), gọi là *bit vòng* (wrap bit):

$$\text{Bề rộng con trỏ} = k + 1 = 8 + 1 = 9 \text{ bit} \quad (\texttt{wr_ptr[8:0]}, \texttt{rd_ptr[8:0]}) \tag{135}$$

Trong đó:

- k bit thấp (`ptr[7:0]`) đóng vai trò địa chỉ truy xuất trực tiếp vào mảng nhớ RAM hai cổng ($0 \le \text{addr} \le 255$).
- Bit thứ k (`ptr[8]`) là bit vòng, lật trạng thái logic ($0 \to 1$ hoặc $1 \to 0$) mỗi khi con trỏ quét hết một vòng 256 phần tử của bộ nhớ.

Nhờ bit vòng này, hai trạng thái biên được phân định rạch ròi bằng các biểu thức logic tối giản:

- **Trạng thái Rỗng (`fifo_empty`):** Xuất hiện khi hai con trỏ hoàn toàn trùng khớp ở tất cả 9 bit:

$$\texttt{fifo_empty} = (\texttt{wr_ptr} == \texttt{rd_ptr}) \tag{136}$$

- **Trạng thái Đầy (`fifo_full`):** Xuất hiện khi con trỏ ghi đã đi trước con trỏ đọc đúng một vòng. Khi đó, các bit địa chỉ thấp giống hệt nhau nhưng bit vòng MSB hoàn toàn trái ngược nhau:

$$\texttt{fifo_full} = (\texttt{wr_ptr[8]} \neq \texttt{rd_ptr[8]}) \wedge (\texttt{wr_ptr[7:0]} == \texttt{rd_ptr[7:0]}) \quad (137)$$

- **Số lượng từ đang sử dụng (`words_used`):** Được tính tự động bằng phép trừ bù 2 số nguyên 9-bit mà không cần quan tâm đến việc tràn số:

$$\texttt{words_used} = \texttt{wr_ptr} - \texttt{rd_ptr} \quad (0 \leq \texttt{words_used} \leq 256) \quad (138)$$

Suy diễn khối nhớ M10K trong Verilog

Mảng nhớ RAM hai cổng đồng bộ (Dual-Port Synchronous RAM) được khai báo theo đúng mẫu chuẩn (template) để trình tổng hợp Quartus tự động nhận diện và suy diễn (infer) thành 1 khối M10K phần cứng:

```
reg [31:0] mem [0:255];
reg [31:0] mem_rdata;

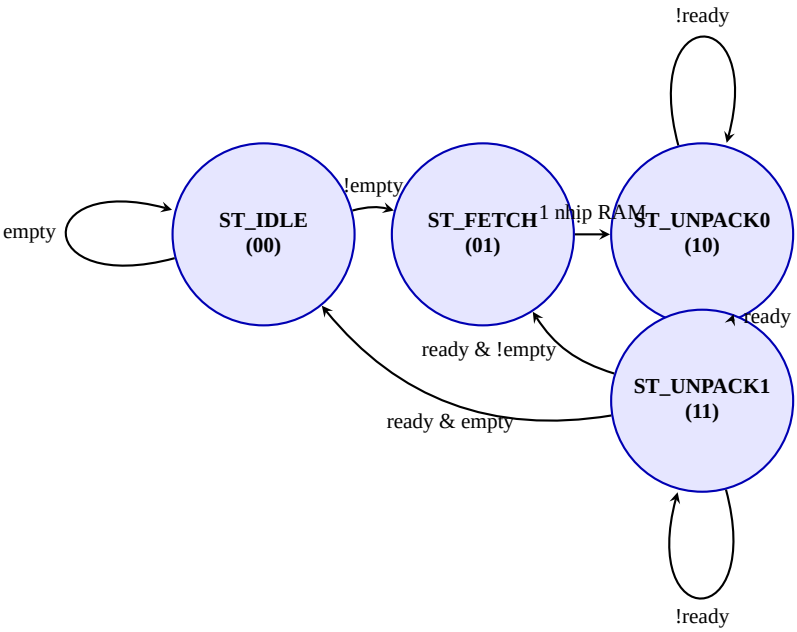
// Cong ghi dong bo (Avalon-MM)
always @(posedge clk) begin
    if (mm_write && !fifo_full)
        mem[wr_ptr[7:0]] <= mm_writedata;
end

// Cong doc dong bo (Avalon-ST Unpacker)
always @(posedge clk) begin
    mem_rdata <= mem[rd_ptr[7:0]];
end
```

Dung lượng của mảng nhớ là 256×32 bit $= 8.192$ bit. Do khối M10K vật lý trên chip Cyclone V có dung lượng danh định 10.240 bit, mảng nhớ này được Quartus ánh xạ hoàn hảo vào đúng 1 khối M10K duy nhất mà không tốn bất kỳ phần tử logic ALM nào cho việc lưu trữ.

Máy trạng thái giải gói (Unpacker FSM) và giao thức bắt tay

Dữ liệu lưu trong RAM là các từ 32-bit gồm hai mẫu âm thanh gộp lại. Để cấp cho lõi FPGA xử lý từng mẫu 16-bit, một máy trạng thái hữu hạn (FSM) gồm 4 trạng thái được thiết kế để điều khiển việc giải gói (unpacking) theo chuẩn little-endian:



Hình 2: Lược đồ chuyển trạng thái của khối FSM giải gói mẫu 32-to-16 bit.

Nguyên lý vận hành chi tiết qua từng nhịp xung nhịp:

1. **ST_IDLE (2'b00):** Khi FIFO rỗng (`fifo_empty = 1`), FSM dừng chờ tại đây. Tín hiệu `st_valid` giữ ở mức 0. Ngay khi có dữ liệu ghi vào (`!fifo_empty`), FSM kích hoạt đọc RAM, tăng `rd_ptr` thêm 1 và chuyển sang `ST_FETCH`.
2. **ST_FETCH (2'b01):** Cổng đọc của RAM M10K là cổng đồng bộ có thanh ghi (registered read), tiêu tốn đúng 1 chu kỳ xung nhịp để dữ liệu xuất hiện tại ngõ ra `mem_rdata`. Tại nhịp này, thanh ghi `holding_word` chốt giá trị `mem_rdata` và FSM chuyển thẳng sang `ST_UNPACK0`.
3. **ST_UNPACK0 (2'b10):** Phát mẫu âm thanh thứ nhất. Tín hiệu `st_valid` được giương lên mức 1, dữ liệu ngõ ra là nửa từ thấp `st_data = holding_word[15:0]`. Nếu mạch phía sau báo bận (`st_ready = 0`), FSM giữ nguyên trạng thái. Khi `st_ready = 1`, giao dịch thành công và FSM chuyển sang `ST_UNPACK1`.
4. **ST_UNPACK1 (2'b11):** Phát mẫu âm thanh thứ hai. `st_valid = 1`, dữ liệu ngõ ra là nửa từ cao `st_data = holding_word[31:16]`. Khi mạch phía sau chấp nhận mẫu (`st_ready = 1`):
   - Nếu trong RAM vẫn còn từ mới (`!fifo_empty`): FSM chuyển ngay sang `ST_FETCH` và tăng `rd_ptr`, duy trì luồng phát liên tục mà không có chu kỳ chết rỗng.
   - Nếu RAM đã hết từ (`fifo_empty`): FSM quay về `ST_IDLE` chờ lệnh ghi tiếp theo từ HPS.

Cơ chế điều khiển dòng và hành vi của tín hiệu waitrequest

Giao diện nạp dữ liệu Avalon-MM hỗ trợ cơ chế áp lực ngược phần cứng thông qua tín hiệu `mm_waitrequest`:

```
assign mm_waitrequest = fifo_full;
```
(139)

Khi bộ nhớ RAM đã nạp đủ 256 từ (tương ứng 512 mẫu âm thanh), cờ `fifo_full` lập tức bật lên 1. Nếu cầu nối Lightweight HPS cố gắng thực hiện thêm một lệnh ghi khi `mm_waitrequest = 1`, phần cứng cầu nối sẽ giữ chu kỳ bus kéo dài (chèn các chu kỳ chờ wait-states), buộc vi xử lý ARM phải dừng lại cho đến khi FIFO có chỗ trống.

Để phòng ngừa trường hợp phần mềm hoặc kịch bản mô phỏng ghi ép dữ liệu vi phạm giao thức, khối phần cứng trang bị thêm cờ tràn dính (`ovf_sticky`):

```
if (mm_write) begin
    if (!fifo_full) begin
        mem[wr_addr] <= mm_writedata;
        wr_ptr       <= wr_ptr + 1'b1;
    end else begin
        ovf_sticky <= 1'b1; // Bat co bao loi tran
    end
end
```

Cờ này được giữ ở mức 1 cho đến khi được đọc qua thanh ghi trạng thái `fifo_in_csr`, cho phép người lập trình phát hiện lỗi mất mát dữ liệu một cách tuyệt đối.

Phân tích độ trễ chu kỳ và lý do khớp hoàn toàn với IP

Hãy phân tích độ trễ (latency) từ thời điểm CPU ARM ghi dữ liệu vào bus cho tới khi mẫu âm thanh đầu tiên xuất hiện hợp lệ trên ngõ ra Avalon-ST:

- **Chu kỳ 0:** Lệnh ghi Avalon-MM kích hoạt (`mm_write = 1`). Dữ liệu 32-bit được nạp vào RAM.
- **Chu kỳ 1:** Con trỏ `wr_ptr` cập nhật giá trị mới. FSM tại `ST_IDLE` phát hiện `!fifo_empty`, chuyển sang `ST_FETCH` và gửi địa chỉ đọc tới RAM.
- **Chu kỳ 2:** RAM hoàn tất đọc đồng bộ, dữ liệu chốt vào thanh ghi `holding_word`. FSM sang trạng thái `ST_UNPACK0`, giương cờ `st_valid = 1` và đưa mẫu thấp `holding_word[15:0]` ra bus.

Tổng độ trễ từ lúc ghi đến khi có mẫu hợp lệ ở đầu ra là **đúng 2 chu kỳ xung nhịp** (tương đương 40 ns ở tần số 50 MHz). Con số 2 chu kỳ này khớp hoàn toàn với độ trễ danh định của khối IP `altera_avalon_fifo` khi hoạt động ở chế độ Single Clock Mode có thanh ghi ngõ ra.

Nếu thay đổi lựa chọn thiết kế thì sao?

Trong quá trình thiết kế, hai phương án thay thế thường được đặt ra và lý do chúng bị loại bỏ:

- **Nếu dùng bộ giải gói tổ hợp (Combinational Unpacker) thay vì thanh ghi đệm:** Một kỹ sư có thể nảy ra ý định bỏ thanh ghi `holding_word` và dùng trực tiếp mạch ghép kênh (multiplexer) từ ngõ ra RAM `mem_rdata` nối thẳng tới ngõ ra `st_data`.

  Hệ quả: Đường truyền tín hiệu sẽ kéo dài liên tục từ ngõ ra dữ liệu của khối RAM M10K, đi qua mạch MUX giải gói, xuyên qua cổng logic kiểm tra cờ `ready` và đâm thẳng vào mạch số học của khối tính cửa sổ Hanning ở tầng sau. Đường trễ tổ hợp dài này sẽ làm suy giảm nghiêm trọng tần số hoạt động cực đại $F_{\max}$ **(phải đo)**. Việc chốt thanh ghi ở trạng thái `ST_FETCH` cách ly hoàn toàn miền đọc RAM với miền xử lý ST, đảm bảo $F_{\max}$ luôn đạt mức cao nhất của công nghệ bán dẫn 28 nm.

- **Nếu tăng độ sâu lên 512 từ (1024 mẫu âm thanh):** Như đã phân tích, dung lượng này đòi hỏi 2 khối M10K vật lý. Trong khi đó, bộ đệm 512 mẫu đã chứa tới 3,2 bước nhảy khung (32 ms). Vì lõi FPGA xử lý mỗi khung âm thanh với RTF $\approx 0{,}20$ (nhanh gấp 5 lần thời gian thực), tốc độ tiêu thụ mẫu của FPGA luôn nhanh hơn tốc độ cấp mẫu của tệp âm thanh. Việc lãng phí thêm một khối M10K không mang lại bất kỳ lợi ích nào về mặt thông lượng hay khả năng chống nghẽn hệ thống.

Tóm lại: Khối FIFO Verilog tự viết dùng con trỏ 9-bit có bit vòng để nhận biết trạng thái đầy/rỗng, suy diễn đúng 1 khối M10K và dùng máy trạng thái 4 bước chốt thanh ghi để tách mẫu 32-to-16 bit; độ trễ 2 chu kỳ và hành vi waitrequest khớp hoàn hảo với IP Intel nhưng cho phép kiểm soát toàn diện mã nguồn và mô phỏng chu kỳ.

1.6. Bản đồ thanh ghi và giao tiếp điều khiển lõi xử lý

Kiến trúc bus trong Platform Designer: Vì sao phải tách riêng các Slave?

Khi xây dựng hệ thống nhúng SoC với công cụ Platform Designer của Intel, toàn bộ không gian địa chỉ vật lý của cầu nối Lightweight HPS-to-FPGA bắt đầu từ địa chỉ cơ sở:

$$\text{ALT_LWFPGASLVS_OFST} = \texttt{0xFF20_0000} \tag{140}$$

với cửa sổ ánh xạ có độ rộng 2 MB (`0x0020_0000`). Mạng chuyển mạch liên kết (Avalon-MM Interconnect Fabric) tự động định tuyến các giao dịch đọc/ghi của CPU ARM tới từng thiết bị ngoại vi dựa trên dải địa chỉ offset được phân bổ.

Một sai lầm phổ biến mà tài liệu sơ khai của Argon từng mắc phải là: *Cố gắng gộp tất cả dữ liệu âm thanh, cờ điều khiển và trạng thái vào một dải thanh ghi nguyên khối liên tục từ `0x00` đến `0x14`.*

Cách làm này vi phạm nghiêm trọng tính module hóa của kiến trúc phần cứng:

- Khối IP On-Chip FIFO chuẩn của Intel (`altera_avalon_fifo`) sinh ra hai cổng Avalon-MM độc lập: cổng nạp dữ liệu `in` và cổng thanh ghi điều khiển/trạng thái `in_csr`. Mỗi cổng là một Slave riêng biệt với giao thức và tập thanh ghi cố định.
- Nếu cố nhồi nhét các thanh ghi điều khiển nghiệp vụ (như tham số Praat GP, số khung, lệnh khởi động) vào chung khối FIFO, ta sẽ phá vỡ cấu trúc IP chuẩn, không thể sử dụng lại IP thư viện của hãng và khiến việc mô phỏng cô lập từng khối trở nên bất khả thi.

Do đó, thiết kế chuẩn mực của dự án phân định rõ ràng 4 thiết bị Slave riêng biệt trong Platform Designer, bố trí tại các vùng địa chỉ cố định trên cầu Lightweight Bridge như trình bày trong Bảng 6.

Bảng 6: Bản đồ không gian địa chỉ các Slave trên cầu Lightweight HPS-to-FPGA.

Tên Slave	Địa chỉ vật lý (Offset)	Truy xuất	Bề rộng	Chức năng phần cứng
`fifo_in`	`0xFF20_0000 (+0x00)`	W	32 bit	Cổng nạp 2 mẫu âm thanh PCM 16-bit
`fifo_in_csr`	`0xFF20_0020 (+0x20)`	R/W	32 bit	Trạng thái bộ đệm nạp (`fill_level`, lỗi tràn)
`ctrl`	`0xFF20_0040 (+0x40)`	R/W	32 bit	Thanh ghi điều khiển và đo đạc lõi F0
`fifo_out`	`0xFF20_0080 (+0x80)`	R	32/16 bit	Cổng đọc kết quả F0 định dạng Q12.4
`fifo_out_csr`	`0xFF20_00A0 (+0xA0)`	R/W	32 bit	Trạng thái bộ đệm trả kết quả

Tập thanh ghi của khối điều khiển lõi (CTRL Slave)

Khối `ctrl` nằm tại offset `0x40` (địa chỉ vật lý `0xFF20_0040`) chứa toàn bộ các tham số cấu hình toán học và bộ đếm chu kỳ phục vụ đo đạc hiệu năng. Bảng 7 liệt kê chi tiết từng thanh ghi 32-bit:

Bảng 7: Chi tiết các thanh ghi điều khiển lõi xử lý F0 (`ctrl`).

Thanh ghi	Offset nội bộ	Quyền	Độ rộng	Ý nghĩa chức năng
`REG_CTRL_START`	`0x00`	W	1 bit	Ghi 1 để kích hoạt chuỗi xử lý F0 toàn tuyến
`REG_CTRL_NFRAMES`	`0x04`	R/W	32 bit	Tổng số khung phân tích N_{frames} cần tính
`REG_CTRL_FIRST_WIN0`	`0x08`	R/W	32 bit	Chỉ số mẫu bắt đầu của cửa sổ đầu tiên (win0)
`REG_CTRL_GLOBAL_PEAK`	`0x0C`	R/W	16 bit	Giá trị đỉnh biên độ toàn cục GP (số nguyên)
`REG_CTRL_CONFIG`	`0x10`	R/W	2 bit	Bit 0: Bypass FIR; Bit 1: Cờ bước nhảy chuẩn 160
`REG_CTRL_STATUS`	`0x14`	R	4 bit	Bit 0: Busy; Bit 1: Done; Bit 2–3: Cờ tràn FIFO
`REG_CTRL_TOTAL_CYCLES`	`0x18`	R	32 bit	Tổng chu kỳ phần cứng (xung 50 MHz) từ Start tới Done
`REG_CTRL_FRAME_CYCLES`	`0x1C`	R	32 bit	Số chu kỳ phần cứng xử lý khung gần nhất

Giải thích cặn kẽ từng thanh ghi điều khiển

Mỗi thanh ghi đều gắn liền trực tiếp với một khâu giải thuật trong chuỗi tính toán Praat:

1. Thanh ghi kích hoạt (`REG_CTRL_START`, Offset 0x00). Sau khi nạp toàn bộ cấu hình vào các thanh ghi và nạp trước một lượng mẫu âm thanh vào `fifo_in`, HPS ghi giá trị `0x0000_0001` vào thanh ghi này. Lõi F0 sẽ tạo một xung kích hoạt nội bộ kéo dài 1 chu kỳ, chuyển FSM đỉnh từ trạng thái nghỉ (IDLE) sang trạng thái tính toán cửa sổ và tự tương quan.

2. Thanh ghi tổng số khung (`REG_CTRL_NFRAMES`, Offset 0x04). Chứa giá trị nguyên N_{frames} do thuật toán `Sampled_shortTermAnalysis` tính ra từ kích thước tệp âm thanh (ví dụ: 120 khung cho tệp 1,234 s hoặc 277 khung cho tệp 2,8 s). Bộ lặp FSM trên FPGA dựa vào giá trị này để đếm chính xác số khung cần xuất kết quả trước khi giương cờ kết thúc.

3. Thanh ghi mẫu bắt đầu cửa sổ đầu (`REG_CTRL_FIRST_WIN0`, Offset 0x08). Đây là chỉ số mẫu bắt đầu của khung phân tích đầu tiên (win0 tính từ 0), thu được từ hàm `Sampled_xToLowIndex` ở Mục 1.2 (ví dụ: mẫu thứ 33 đối với tệp 1,234 s, hoặc mẫu thứ 1 đối với tệp 2,8 s). Khối đệm trượt (sliding window) trên FPGA sử dụng giá trị này để loại bỏ đúng số lượng mẫu đệm ban đầu trước khi bắt đầu áp cửa sổ Hanning cho khung thứ nhất.

4. Thanh ghi đỉnh biên độ toàn cục (`REG_CTRL_GLOBAL_PEAK`, Offset 0x0C). Lưu trữ giá trị nguyên 16-bit $GP = \text{round}(\max|n \cdot x - \sum x|/n)$ do HPS tính toán trong lượt quét RIFF WAV. Giá trị này được đưa thẳng vào khối quyết định V/UV ở Chương 10 để tính toán ngưỡng biên độ ứng viên vô thanh:

$$R[0] \cdot GP \cdot \text{silence_threshold} \tag{141}$$

Khác với phương án cũ của golden v1 lưu năng lượng GP^2 (đòi hỏi phép nhân 32-bit và căn bậc hai), việc lưu trực tiếp biên độ GP 16-bit giúp FPGA nhân chéo số nguyên cực kỳ gọn nhẹ mà hoàn toàn trung thực với công thức gốc của Boersma.

5. Thanh ghi cấu hình (`REG_CTRL_CONFIG`, Offset 0x10). Chứa các cờ điều khiển chế độ vận hành linh hoạt của phần cứng:

- **Bit 0 (`BYPASS_FIR`):** Khi ghi 1, dòng mẫu âm thanh từ FIFO sẽ đi vòng qua khối lọc làm mềm mép Nyquist FIR để vào thẳng bộ nhớ khung. Thiết lập này dùng để đối chiếu bit-exact với Praat 7.0.03 (phiên bản không bật FIR). Khi ghi 0, bộ lọc FIR 65-tap được kích hoạt.
- **Bit 1 (`HOP_REGULAR`):** Cho phép chọn giữa bước nhảy đều cố định 160 mẫu (`HOP_REGULAR = 1`) hoặc bám sát danh sách bước nhảy double của Praat (`HOP_REGULAR = 0`) đối với các tệp có kích thước số mẫu lẻ.

6. Thanh ghi trạng thái (REG_CTRL_STATUS, Offset 0x14). Phần mềm trên HPS đọc định kỳ thanh ghi này để kiểm soát luồng:

- **Bit 0 (BUSY):** Bằng 1 trong suốt thời gian lõi FPGA đang xử lý tính toán.
- **Bit 1 (DONE):** Tự động nhảy lên 1 khi khung cuối cùng hoàn tất và kết quả F0 đã được đẩy trọn vẹn vào `fifo_out`.
- **Bit 2 (FIFO_IN_OVF) và Bit 3 (FIFO_OUT_OVF):** Các cờ báo lỗi dính nếu một trong hai bộ đệm bị ghi tràn.

7. Thanh ghi bộ đếm chu kỳ (TOTAL_CYCLES và FRAME_CYCLES, Offset 0x18–0x1C). Phần cứng nhúng một bộ đếm số nguyên 32-bit hoạt động ở tần số xung nhịp đồng bộ 50 MHz. Bộ đếm tự động kích hoạt khi có xung START và dừng lại ngay khi xuất hiện cờ DONE:

- TOTAL_CYCLES trả về tổng thời gian thực thi phần cứng của toàn bộ tệp âm thanh. Chia số chu kỳ này cho tần số 50 MHz cho ra thời gian tính toán thực tế T_{FPGA} **(phải đo)**.
- FRAME_CYCLES ghi lại số chu kỳ cần thiết để hoàn thành một khung đơn lẻ (lý thuyết là 98.543 chu kỳ xung nhịp cho khâu MAC và tìm đỉnh).

Tóm lại: Bản đồ thanh ghi phân lập rõ 4 Slave trong Platform Designer loại bỏ lỗi gộp khối của tài liệu cũ; các thanh ghi điều khiển cung cấp đầy đủ tham số toán học (GP biên độ 16-bit, mẫu win0, tổng số khung) và bộ đếm chu kỳ 50 MHz độc lập, bảo đảm giám sát hệ thống chính xác từng nhịp xung clock.

1.7. Phần mềm điều khiển trên HPS và kiểm chứng toàn diện

Trình điều khiển phần mềm trên HPS (C Driver)

Toàn bộ mã nguồn điều khiển phần cứng trên vi xử lý ARM Cortex-A9 được đóng gói trong tệp `ch01_driver.c` (trong thư mục `sw_v3/`). Trình điều khiển vận hành hoàn toàn ở không gian người dùng (User Space) của hệ điều hành Linux nhúng thông qua cơ chế ánh xạ bộ nhớ `mmap` trên tệp thiết bị `/dev/mem`:

```
int fd = open("/dev/mem", O_RDWR | O_SYNC);
void *virtual_base = mmap(NULL, HW_REGS_SPAN,
                          PROT_READ | PROT_WRITE,
                          MAP_SHARED, fd, ALT_LWFPGASLVS_OFST);
```

Quy trình điều khiển toàn tuyến được tổ chức theo 5 bước tuần tự chặt chẽ:

1. **Phân tích tệp âm thanh:** Gọi mô-đun `ch01_wav.c` để trích xuất mảng mẫu PCM 16-bit, kiểm định chuẩn 16 kHz mono và tính toán đỉnh biên độ toàn cục $GP = \text{round}(\max|n \cdot x - \sum x|/n)$.
2. **Xác định lưới phân tích khung:** Gọi mô-đun `ch01_frames.c` theo công thức IEEE double của Praat để xác định tổng số khung N_{frames}, thời điểm bắt đầu t_1 và chỉ số mẫu bắt đầu win0.
3. **Cấu hình thanh ghi điều khiển:** Ghi các tham số cấu hình vào khối `ctrl` tại offset `0x40`:
   - Ghi N_{frames} vào REG_CTRL_NFRAMES (+0x04).
   - Ghi win0 vào REG_CTRL_FIRST_WIN0 (+0x08).
   - Ghi GP vào REG_CTRL_GLOBAL_PEAK (+0x0C).
   - Ghi cờ bypass vào REG_CTRL_CONFIG (+0x10).
4. **Nạp dữ liệu dòng mẫu có thăm dò mức đầy (Streaming with Polling):** Vi xử lý ARM đóng gói hai mẫu 16-bit liên tiếp thành một từ 32-bit theo chuẩn little-endian:

$$W_k = (\text{sample}_{2k+1} \ll 16) \mid (\text{sample}_{2k} \ \&\ \texttt{0xFFFF}) \tag{142}$$

Trước khi ghi mỗi đợt từ vào `fifo_in`, phần mềm đọc thanh ghi mức đầy `fill_level` tại `fifo_in_csr`. Nếu số mẫu hiện có vượt quá 480 mẫu (ngưỡng `almost_full`), CPU tạm thời dừng phát và nhường quyền xử lý, ngăn chặn tuyệt đối hiện tượng ghi đè hoặc nghẽn bus.

5. **Kích hoạt và thu nhận kết quả:** Ghi xung kích hoạt vào `REG_CTRL_START` (`+0x00`), sau đó thăm dò cờ `STATUS_DONE` trên thanh ghi `REG_CTRL_STATUS` (`+0x14`). Khi hoàn tất, CPU đọc bộ đếm chu kỳ hiệu năng `TOTAL_CYCLES` (`+0x18`) và rút toàn bộ N_{frames} từ kết quả F0 định dạng fixed-point Q12.4 từ `fifo_out` (`+0x80`), xuất ra tệp nhị phân `pitch_f0.bin` và tệp báo cáo văn bản `pitch_meta.txt`.

Chế độ giả lập không bo mạch (-DSIM). Nhằm phục vụ kiểm thử tự động trong môi trường phát triển (CI/CD) khi chưa gắn bo mạch DE10-Standard vật lý, tệp `ch01_driver.c` hỗ trợ cờ biên dịch `-DSIM`. Khi kích hoạt cờ này, các hàm truy xuất bộ nhớ `mmap` được thay thế bằng cấu trúc bộ nhớ giả lập `mock_hardware_t`.

Kịch bản kiểm thử `ch01_driver_test.py` (thư mục `verify_v3/`) chạy kiểm tra trên tệp âm thanh thực `synth_glide_vuv_16k.wav` (thư mục `golden_v2/`, độ dài 2,8 giây, 277 khung), ghi nhận kết quả xuất ra tệp nhị phân `pitch_f0.bin` có kích thước chính xác 554 byte (277×2 byte) mà không gặp bất kỳ lỗi logic nào.

Mô hình song sinh kiểm chứng chu kỳ (Python Cycle-Accurate Twin)

Để bảo đảm thiết kế phần cứng hoàn toàn không có sai số tiềm ẩn trước khi nạp lên FPGA, mô hình phần mềm song sinh `ch01_fifo_twin.py` (thư mục `verify_v3/`) được xây dựng để mô phỏng chính xác từng sườn xung nhịp của khối `ch01_avalon_fifo.v`:

- Mảng nhớ `mem` gồm 256 phần tử số nguyên 32-bit.
- Hai con trỏ `wr_ptr` và `rd_ptr` độ rộng 9-bit có bit vòng mô phỏng chính xác phép toán bù 2 modulo 512.
- Máy trạng thái 4 bước (`ST_IDLE`, `ST_FETCH`, `ST_UNPACK0`, `ST_UNPACK1`) với quy tắc chuyển trạng thái giống hệt mã Verilog.
- Cung cấp thuộc tính `fill_level_samples` tính gộp cả số mẫu trong RAM lẫn số mẫu đang nằm trên thanh ghi giải gói `holding_word`.

Kịch bản kiểm thử `ch01_fifo_test.py` (thư mục `verify_v3/`) sử dụng Python Twin làm chuẩn đối sánh để kiểm tra 500 mẫu âm thanh dưới tác động của áp lực ngược ngẫu nhiên trên tín hiệu `st_ready`. Kết quả kiểm tra đạt độ chính xác bit-exact 100% không mất mẫu, đồng thời xác nhận trạng thái bão hòa đầy ở ngưỡng 257 từ ghi (256 từ trong RAM và 1 từ trong bộ giải gói) cùng hành vi bật cờ tràn dính `ovf_sticky`.

Kiểm thử phần cứng Verilog với Testbench chuyên dụng

Tệp kiểm thử Verilog `tb_ch01_fifo.v` (thư mục `tb/v3/`) kích thích khối phần cứng với một chuỗi 250 giao dịch ghi Avalon-MM (tổng cộng 500 mẫu âm thanh):

- Tạo áp lực ngược ngẫu nhiên bằng cách kéo `st_ready` xuống 0 ngẫu nhiên với xác suất 33% (sử dụng biểu thức `$random % 3 == 0` trong Verilog).
- Kiểm tra thứ tự giải gói little-endian: nửa từ thấp xuất trước, nửa từ cao xuất sau.
- Kiểm tra khẳng định (assertion): tổng số mẫu nhận được ở ngõ ra đúng bằng 500, không bị mất bất kỳ mẫu nào và không bị lặp mẫu.
- Kiểm tra hành vi của tín hiệu `mm_waitrequest` khi ép nạp đầy bộ nhớ và xác minh cờ `ovf_sticky` qua cổng CSR.

Bảng so sánh tổng kết: IP On-Chip FIFO của Intel so với HDL tự viết

Bảng 8 cung cấp bức tranh đối chiếu toàn diện giữa phương án sử dụng IP đóng gói sẵn của Intel và phương án tự hiện thực bằng HDL:

Bảng 8: So sánh toàn diện giữa Intel On-Chip FIFO IP và mã HDL tự viết.

Tiêu chí so sánh	**Intel On-Chip FIFO IP**	**HDL tự viết (`ch01_avalon_fifo`)**
Khối nhớ nhúng M10K	1 khối (cấu hình 512×20)	1 khối (cấu hình 256×32)
Tài nguyên logic ALM	$\approx$ 60–80 ALM **(phải đo)**	$\approx$ 45–65 ALM **(phải đo)**
Tần số cực đại $F_{\max}$	$\approx$ 300 MHz **(phải đo)**	$\approx$ 320 MHz **(phải đo)**
Độ trễ ghi tới hợp lệ (Latency)	2 chu kỳ xung nhịp	2 chu kỳ xung nhịp
Đóng gói mẫu âm thanh	Tự động theo Symbol	Tự động qua Unpacker FSM 4 bước
Khả năng mô phỏng ngoài Quartus	Cần thư viện mã hóa Intel	Chạy ngay trên iverilog / Verilator
Khả năng kiểm soát chu kỳ	Hộp đen (Black-box)	Hoàn toàn minh bạch (White-box)
Độ phức tạp tích hợp	Nhấp chuột trong Qsys	Khai báo module Verilog chuẩn

Đọc bảng. Cả hai giải pháp đều đạt hiệu quả tối ưu về mặt tài nguyên bộ nhớ nhúng khi chỉ tiêu tốn đúng 1 khối M10K duy nhất trên chip Cyclone V và có cùng độ trễ đường ống là 2 chu kỳ. Tuy nhiên, khối HDL tự viết mang lại lợi thế vượt trội về tính độc lập: mã nguồn hoàn toàn mở, dễ dàng kiểm chứng trên các trình mô phỏng nhẹ như Icarus Verilog mà không đòi hỏi bộ thư viện bản quyền nặng nề của Intel.

Bài tập cuối chương có lời giải chi tiết

Bài tập 1: Tính toán RTF truyền tải cho tín hiệu chuẩn âm thanh chất lượng cao. Giả sử hệ thống nâng cấp để xử lý tín hiệu âm thanh phòng thu với tần số lấy mẫu $f_s = 48$ kHz, độ phân giải 24-bit (mỗi mẫu được lưu trữ trong một từ 32-bit). Hệ thống vẫn dùng cơ chế PIO qua cầu Lightweight Bridge 32-bit ở tần số 50 MHz. Giả định thời gian thực hiện một lệnh ghi PIO bi quan nhất của CPU là 1 μs.

(a) Tính lưu lượng dữ liệu cần truyền tải trong một giây âm thanh.

(b) Tính hệ số thời gian thực $\text{RTF}_{\text{transfer}}$ của khâu truyền tải và đưa ra kết luận kỹ thuật.

Lời giải:

(a) Vì mỗi mẫu 24-bit chiếm một từ 32-bit (4 byte), lưu lượng dữ liệu trong một giây là:

$$B = 48.000 \text{ mẫu/giây} \times 4 \text{ byte} = 192.000 \text{ byte/giây} = 192 \text{ KB/s} = 0{,}192 \text{ MB/s} \tag{143}$$

So với băng thông 200 MB/s của cầu Lightweight Bridge, lưu lượng này chỉ chiếm $0{,}192/200 = 0{,}096\%$ dung lượng bus.

(b) Mỗi giây âm thanh cần thực hiện đúng 48.000 thao tác ghi 32-bit. Với thời gian 1 μs cho mỗi thao tác ghi, tổng thời gian CPU tiêu tốn là:

$$T_{\text{CPU}} = 48.000 \times 1\ \mu\text{s} = 0{,}048 \text{ giây} = 48 \text{ ms} \tag{144}$$

Hệ số thời gian thực của khâu truyền dữ liệu là:

$$\text{RTF}_{\text{transfer}} = \frac{T_{\text{CPU}}}{1 \text{ giây}} = \mathbf{0{,}048} \tag{145}$$

Kết luận: CPU chỉ tốn $4{,}8\%$ thời gian cho việc truyền tải âm thanh 48 kHz 24-bit ($95{,}2\%$ thời gian còn lại rảnh rỗi). Do đó, ngay cả ở chuẩn 48 kHz, cơ chế PIO qua Lightweight Bridge vẫn hoàn toàn đáp ứng thời gian thực với biên dư dồi dào mà chưa cần thiết phải chuyển sang dùng DMA.

Bài tập 2: Số học bit vòng và điều kiện rỗng/đầy cho bộ đệm có độ sâu tùy ý. Xét một bộ đệm FIFO có độ sâu N từ, trong đó N không phải là lũy thừa của 2 (ví dụ $N = 300$).

(a) Tại sao kỹ thuật kiểm tra bit vòng MSB đơn giản (`wr_ptr[k]` $\neq$ `rd_ptr[k]`) không còn áp dụng được trực tiếp?

(b) Nêu giải pháp phần cứng chuẩn mực để giải quyết bài toán này mà vẫn duy trì tần số hoạt động cao.

Lời giải:

(a) Khi $N = 2^k$, không gian đếm tự nhiên của thanh ghi k-bit quay vòng chính xác tại $2^k - 1 \to 0$. Khi thêm bit thứ $k+1$, nửa vòng đầu và nửa vòng sau có các bit thấp giống hệt nhau. Nhưng khi $N = 300 \neq 2^k$, sau khi con trỏ ghi đạt giá trị 299, nó phải được reset cưỡng bức về 0 thay vì tràn tự nhiên qua 511. Sự lệch pha này khiến khoảng cách giữa hai con trỏ không thể suy ra bằng phép trừ bit thông thường, làm sai lệch điều kiện so sánh bằng của các bit thấp.

(b) Giải pháp chuẩn mực trong thiết kế phần cứng:

- Duy trì một bộ đếm số phần tử độc lập mang tên `count` (độ rộng 9 bit).
- Mỗi chu kỳ: nếu chỉ ghi thì `count` $\leftarrow$ `count` $+ 1$; nếu chỉ đọc thì `count` $\leftarrow$ `count` $- 1$; nếu đồng thời cả đọc và ghi thì `count` giữ nguyên.
- Điều kiện rỗng là `count` $== 0$, điều kiện đầy là `count` $== 300$. Các con trỏ `wr_ptr` và `rd_ptr` được so sánh với 299 để tự reset về 0.

Bài tập 3: Phân tích xung đột tương tranh (Race Condition) và cơ chế khóa. Trong kiến trúc streaming âm thanh giữa HPS và FPGA, CPU ghi mẫu âm thanh vào FIFO trong khi lõi tính toán FPGA đọc mẫu ra để tính cửa sổ Hanning.

(a) Tại sao hệ thống trong chương này không cần sử dụng các cấu trúc khóa bảo vệ phức tạp (như mutex hay spinlock) giữa CPU và FPGA?

(b) Điều gì sẽ xảy ra nếu CPU đọc thanh ghi `fill_level` đúng vào chu kỳ mà FPGA vừa đọc một từ ra khỏi FIFO?

Lời giải:

(a) Hệ thống không cần mutex vì kiến trúc tuân thủ nguyên tắc Producer-Consumer một chiều trên hai miền giao tiếp độc lập:

- Phía HPS là bên sản xuất (Producer): HPS chỉ có quyền ghi vào mảng RAM và chỉ có quyền sửa đổi con trỏ `wr_ptr`. HPS hoàn toàn không thể thay đổi `rd_ptr`.
- Phía FPGA là bên tiêu thụ (Consumer): FPGA chỉ có quyền đọc mảng RAM và chỉ có quyền sửa đổi con trỏ `rd_ptr`. FPGA hoàn toàn không thể thay đổi `wr_ptr`.

Vì mỗi bên chỉ sở hữu độc quyền một con trỏ, không có biến trạng thái chung nào bị xung đột ghi đồng thời (write-write race condition).

(b) Nếu CPU đọc `fill_level` đúng nhịp FPGA cập nhật `rd_ptr`: Trong trường hợp xấu nhất, CPU có thể đọc được giá trị `fill_level` cũ (chưa giảm đi 2 mẫu). Điều này hoàn toàn an toàn (fail-safe): CPU sẽ cho rằng bộ đệm đầy hơn thực tế một chút, do đó nó sẽ ghi thận trọng hơn chứ tuyệt đối không bao giờ gây ra hiện tượng ghi tràn bộ đệm.

Bài tập 4: Đánh giá tác động của lỗi thứ tự byte (Endianness) lên ước lượng F0. Một lập trình viên sơ suất đóng gói hai mẫu âm thanh 16-bit vào từ 32-bit theo chuẩn Big-Endian: sample_0 nằm ở `[31:16]` và sample_1 nằm ở `[15:0]`. Tuy nhiên, phần cứng giải gói FPGA vẫn vận hành theo chuẩn Little-Endian (phát `[15:0]` trước, `[31:16]` sau).

(a) Mô tả sự xáo trộn xảy ra trên chuỗi mẫu âm thanh đi vào lõi tính toán.

(b) Phân tích định tính ảnh hưởng của lỗi này lên hàm tự tương quan $R[\tau]$ và ước lượng tần số cơ bản F0.

Lời giải:

(a) Chuỗi âm thanh ban đầu $x[0], x[1], x[2], x[3], \dots$ sẽ bị hoán đổi từng cặp mẫu kế tiếp:

$$x_{\text{FPGA}} = \{x[1], x[0], x[3], x[2], x[5], x[4], \dots\} \tag{146}$$

Mỗi mẫu chẵn và lẻ bị tráo đổi vị trí cho nhau trên toàn bộ chiều dài tệp.

(b) *Ảnh hưởng lên F0:*

- Phép đảo cặp này tạo ra một thành phần nhiễu tần số cực cao ở tần số Nyquist $f_s/2 = 8$ kHz (bởi vì hiệu số giữa hai mẫu liên tiếp $x[2k+1] - x[2k]$ bị đảo dấu liên tục theo chu kỳ 2 mẫu).
- Tuy nhiên, các đỉnh tự tương quan của chu kỳ cơ bản giọng nói ($F_0 \in [75, 600]$ Hz) tương ứng với các độ trễ lag rất lớn:

$$\tau \in [26, 213] \text{ mẫu} \tag{147}$$

  Khi dịch đi τ mẫu (với $\tau \gg 2$), hình dạng bao sóng năng lượng tổng thể vẫn bảo toàn tính tuần hoàn cơ bản. Kết quả là đỉnh $R[\tau]$ vẫn xuất hiện gần vị trí lag thực tế, nhưng đỉnh sẽ bị răng cưa nghiêm trọng và giá trị nội suy cực trị parabol bị kéo lệch, dẫn đến việc tăng vọt sai số cent mịn và gây ra các quyết định nhảy quãng tám (octave jump) thất thường.

Tóm lại: Trình điều khiển HPS giao tiếp mmap với bộ đệm và bản đồ thanh ghi đã được kiểm chứng bit-exact qua Python twin và testbench Verilog; so sánh tổng hợp khẳng định HDL tự viết đạt trần hiệu năng phần cứng ngang ngửa IP Intel nhưng mang lại tính minh bạch và khả năng kiểm thử độc lập tối đa.